

PIC-tutor

从动理学模型到 WarpX 源码的 Particle-In-Cell 讲解

原生 LaTeX 版本 (Phase 1 样章)

基线书稿: v0.110 (Git 标签 v1.0)

PIC-tutor 项目

2026

目录

写作说明	1
0.1 这本书为谁而写	1
0.2 如何使用本书	1
0.2.1 四个跨章节术语	2
0.3 按目标选择阅读路线	2
0.3.1 路线 A: 先能运行并解释一个最小 PIC case	2
0.3.2 路线 B: 先读懂一条从粒子到场的源码链	3
0.3.3 路线 C: 先学会设计验证, 而不是只收集输出	3
第一章 动理学模型与 PIC 的基本思想	4
1.1 Vlasov 方程首先是相空间守恒律	4
1.2 碰撞项不是 Vlasov 的一部分, 但必须保留边界	5
1.3 Vlasov-Maxwell: 源项、约束与闭合	6
1.4 Vlasov-Maxwell 的能量与动量守恒边界	7
1.5 Vlasov-Poisson / electrostatic 极限不是另一套世界	7
1.5.1 模型选择与验证卡: Poisson 可解不等于完整电磁问题已被解决	8
1.6 宏粒子不是假粒子, 而是 coarse-grained 分布函数载体	9
1.7 权重可以不同, 但不是没有代价	9
1.8 shape factor 不是插值细节, 而是粒子-网格耦合规则	10
1.9 PIC 的噪声不是 bug, 而是模型代价	11
1.9.1 统计噪声与能量账本验证卡: 能量漂移小不等于热平衡或低噪声	12
1.10 Debye 长度、粒子数与统计时间尺度	13
1.10.1 尺度计算卡: 能量回归通过不等于 Debye 屏蔽已分辨	13
1.11 从连续模型到 PIC 离散变量	14
1.12 这一章对后面源码章节的真正约束	15
1.12.1 跨章交接卡: 从连续对象进入离散循环	16
1.13 证据范围与继续阅读	17
1.14 练习与源码定位	17
第二章 PIC 总循环: 从 Vlasov-Maxwell 到离散时间推进	18
2.0.1 从第 1 章进入本章: 先写状态, 再读调用顺序	18
2.1 连续模型: Vlasov-Maxwell 系统	18

2.2	宏粒子表示与形函数	19
2.3	leapfrog 时间层	20
2.3.1	ω_p 不是背景常数，而是时间离散必须尊重的最快等离子体尺度	21
2.3.2	λ_D 不只是一条长度定义，它直接约束 Δx	21
2.4	FDTD 场更新的数学骨架	22
2.4.1	CFL 不是经验参数，而是离散 Maxwell 更新的因果上界	23
2.4.2	数值色散从这里开始：连续光锥被离散 stencil 改写	24
2.4.3	Yee / Nodal / CKC 的差别本质上是离散色散关系不同	25
2.5	PSATD 与 FDTD 的主循环差异	26
2.6	一个完整 PIC step 的离散组织	27
2.6.1	AMR subcycling：两个时间步不是同一个时间步的重复调用	28
2.6.2	JRhom 与 implicit：同一个外层 step 内部也可能有不同的时间合同	29
2.7	本章后的源码阅读入口	30
2.8	参数示例与最小运行案例	31
2.8.1	第一次运行的读者路线：构建、CTest 与手动分析各自回答什么	32
2.8.2	受控修改路线：一个命令行覆盖会改变哪一份证据	33
2.9	基础文献与证据范围	34
2.10	进一步阅读与练习	35
2.11	本章结论	35
第三章	WarpX 主演化路径：生命周期、初始化与 Evolve()	37
3.0.1	源码定位约定	37
3.1	顶层入口：main.cpp	37
3.2	WarpX 类与单例构造	38
3.3	ReadParameters()：主循环分支的来源	39
3.4	构造函数只建“跨 level 外壳”，不直接建完整网格数据	40
3.5	AllocLevelData() / AllocLevelMFs() 里，四条 solver 分支的落点不同	41
3.6	InitData()：把状态准备到可推进	43
3.7	ComputeDt()：步长不是一个固定常数	44
3.8	Evolve() 外层时间步	45
3.8.1	步末 moving window：连续坐标与整数网格平移	45
3.9	OneStep()：求解器分派器	47
3.10	OneStep_nosub()：显式电磁标准路径	49
3.11	SyncCurrentAndRho()：源项不是沉积完就可用	51
3.12	PushParticlesandDeposit()：进入粒子容器	51
3.13	OneStep_sub1() 与 JRhom 的位置	52
3.13.1	implicit 分支：一次物理步包含多次试探性 source 重建	54
3.13.2	nonlinear solver、JFNK 与 mass-matrix：J 的三种构造层	55
3.13.3	AMR subcycling 修改后的验证卡：先分清时间层、source 和回归	56
3.14	参数示例与最小运行闭环	57

3.14.1	生命周期追踪卡：一项输入何时成为可解释的证据	58
3.15	进一步阅读与练习	59
3.16	本章小结	60
3.16.1	跨章交接卡：从调用图保留到可验证的状态	60
第四章	3A. WarpX 初始化链：从 <code>InitData()</code> 到初始粒子和外部场	62
4.0.1	阅读路线：先构造初态，再追踪分支	62
4.1	3A.1 初始化链为什么值得单独成章	63
4.2	3A.2 启动层先于 <code>InitData()</code> ：MPI、AMReX、FFT、PETSc 与启动前提	63
4.2.1	3A.2.1 参数系统先分命名空间，再分读取语义	64
4.2.2	3A.2.2 文档 alias、AMReX-owned 参数与 WarpX 自有 parser 不是一回事	66
4.3	3A.3 顶层入口：fresh run 与 restart 分叉	74
4.4	3A.4 <code>InitFromScratch()</code> ：AMReX level 与粒子初始化	75
4.5	3A.5 外部场初始化：grid field 与 particle external field	76
4.6	3A.6 species 初始化：PlasmaInjector 是参数总容器	76
4.7	3A.7 密度和动量分布：从文本参数到 functor	77
4.8	3A.8 <code>AddParticles()</code> ：按注入类型进入创建函数	79
4.9	3A.9 体注入 <code>AddPlasma()</code> ：候选粒子、密度、动量和权重	80
4.10	3A.10 Gaussian beam：显式束流列表注入	82
4.11	3A.11 openPMD 粒子文件：文件粒子列表注入	83
4.12	3A.12 Projection divergence cleaning：外部 A/B 场的初始约束修正	84
4.12.1	Laser antenna 与 profile 分派：laser 初始化并不走 PlasmaInjector	85
4.13	3A.13 初始化验证入口：哪些 regressions 真正在兜底	94
4.13.1	3A.13.1 初始化验证卡：分布统计和初始自场是两份合同	97
4.14	3A.14 从 Birdsall 3A ES1 到 WarpX：历史最小程序骨架的现代映射	104
4.15	3A.15 本章小结：初始化状态怎样进入第一步推进	105
4.16	3A.16 练习与最小复现	106
附录 A	符号、时间层与源码变量	107
A.1	A.1 连续模型符号	107
A.1.1	ux, uy, uz 的两层约定	108
A.2	A.2 网格、时间层与离散量	108
A.3	A.3 沉积、推进与场求解记号	109
A.4	A.4 诊断、文件与验证术语	110
A.5	A.5 常用缩写	111
A.6	A.6 使用规则	111

写作说明

0.1 这本书为谁而写

如果你会运行一个 PIC case，却还不能回答“粒子在什么时间层被推进”“电流为什么要沿轨迹沉积”“一个小的 $\text{divE}-\rho$ 残差究竟说明什么”，这本书就是从这里开始的。它不要求读者先熟悉 WarpX 的类层次，也不把源码中的函数名当作知识本身；每个实现细节都先放回物理量、离散方程和可观察结果中解释。

读完主线后，读者应当能够：

- 从连续方程推导出 PIC 中需要保存和交换的离散量；
- 沿着一个输入参数追踪到初始化、时间推进、沉积、场更新和诊断输出；
- 为一个新 case 选择时间步、网格、shape、solver 和诊断，而不是只复制输入文件；
- 看见 regression 通过时，准确说出它证明了什么、还没有证明什么。

0.2 如何使用本书

正文按“模型 -> 离散算法 -> 源码调用链 -> 案例诊断”的顺序展开。章节末的练习要求读者自己定位源码、检查时间层或复现实验；它们不是附加的项目验收清单，而是把阅读变成判断能力的训练。

版本号、运行合同和缺口登记记录的是书稿如何被维护，不是读者必须按时间顺序阅读的内容。正文只在需要解释证据范围时使用它们，不把它们作为学习前提。

本书不是 WarpX 官方文档的翻译，也不是只讲公式的 PIC 理论笔记。它的主线是：先从 Vlasov-Maxwell / Vlasov-Poisson 这类连续模型出发，说明为什么需要宏粒子；再把宏粒子、网格、形函数、沉积和场求解拼成 PIC 算法；最后回到 WarpX 的 Source/、Docs/、Examples/ 和 regression 入口，解释一个现代高性能 PIC 程序如何把这些步骤组织成可运行、可扩展、可验证的模拟软件。

源码定位应优先依赖文件职责、函数名和调用关系，而不是固定 commit 或行号。例如，主循环从 `Evolve()` 与 `OneStep()` 开始，初始化从 `InitData()` 开始，粒子/源项主链从 `PushParticlesandDeposit()` 与 `SyncCurrentAndRho()` 开始。版本信息只用于复现实验和维护记录，不应改变读者对算法关系的理解。

本书的每个技术判断都应尽量落到六类证据：物理方程、离散公式、WarpX 源码路径、输入参数、示例或测试、文献。DeepWiki、Zread 等 AI 解读页面可以用来快速找到模块名，但不能作为最终依据。

遇到一个新的输入或源码分支时，可按四步阅读：先写出要描述连续物理量；再标出它在离散网格和时间层上的表示；随后定位 producer 和 consumer；最后选择一个能区分正确

与错误的 observable，并写下该观察量不能支持的外推。这样，代码阅读始终服务于物理判断，而不是变成函数名清单。

0.2.1 四个跨章节术语

后续章节反复使用下列词，但它们不是函数名、文件名或测试标签的同义词：

术语	本书中的含义	阅读时必须继续追问	不能据此直接推出
source	被某个场或方程更新实际消费的离散源项或状态，例如同步后的 ρ 、 $\mathbf{J}$ 或谱空间历史量	它在哪个时间层、几 何位置、AMR level 和同步阶段成立	任意名为 <code>rho</code> 、 <code>current_*</code> 的数组都已可供 solver 消费
producer	在确定的生命周期阶段创建、推进、沉积、同步或归约某个状态的代码路径	它生成什么量、何时 写入、是否还需要过 滤、通信或边界处理	找到一次写入或对象构造就证明物理状态已经完整
consumer	读取该状态以更新场、写诊断、作比较或产生后续状态的代码路径	它读取的是 producer 的哪一时间层和哪一 种表示	输出文件成功写出就证明 producer 的物理语义正确
observable	为回答一个物理问题而定义、可与解析解、守恒关系、reference 或实验量比较的量	它的比较对象、容 差、几何和不可外推 范围是什么	任意已输出字段、 <code>checksum</code> 或图像都是物理验证

“证据层级”则描述上述链条有多强：源码可以定位职责，指定案例可以检验给定条件下的 observable，全文文献可以支撑公式或机制；它们必须互相独立，不能把其中任一层放大成其余各层。第 3 章把这四个术语接到控制流和状态交接，第 5–7 章说明 source 怎样经过推进、同步和边界，第 8 章说明 consumer 如何形成诊断，第 9 章再判断外部 reference 的证据强度。

本书默认使用以下记号：粒子位置为 $\mathbf{x}_p$ ，粒子动量为 $\mathbf{u}_p = \gamma \mathbf{v}_p$ ，电磁场为 $\mathbf{E}, \mathbf{B}$ ，电荷和电流密度为 $\rho, \mathbf{J}$ ，粒子权重为 w_p ，形函数为 S 。网格量的上标表示时间层，例如 $\mathbf{B}^{n+1/2}$ ；粒子量一般按 leapfrog 交错在位置和动量时间层上。

0.3 按目标选择阅读路线

不必把全书当作一条只能从头到尾走完的线。先选择你要获得的能力，再沿相应路线完成每个终点练习；三条路线共享术语和证据边界，但不能用其中一条的通过替代另一条的结论。

0.3.1 路线 A：先能运行并解释一个最小 PIC case

适合已经会启动程序、但还不能说明输出为何可信的读者。先读第 1.10–1.14，写出 Debye 长度、时间尺度和一个要保留的 observable；再读第 2.6–2.10，把它放进一个完整 PIC step 和最小输入；接着用第 3.12–3.13 追踪输入、`warpx_used_inputs`、输出与 analysis 的交接；最后完成第 8.15 的 Langmuir 或 uniform-plasma 练习。终点不是“作业跑完”，而是能写出该 case 的 producer、consumer、比较量、容差以及它不能覆盖的 geometry、算法和物理范围。

0.3.2 路线 B：先读懂一条从粒子到场的源码链

适合需要修改或审查实现、但不希望先被所有模块细节淹没的读者。以第 2.7 的导航为索引，读第 3.4–3.10 确定初始化、外层时间步和 source 同步的职责；然后依次完成第 4.16、5.16、6.12 的练习，分别追踪粒子状态、 ρ/J 和场更新的时间层。每次定位函数时先写“它消费什么、产生什么、下一位 consumer 是谁”，再阅读分派或 kernel；不能从找到一次数组写入就推断求解器已经消费了物理上完整的 source。

0.3.3 路线 C：先学会设计验证，而不是只收集输出

适合准备建立新 case、改变数值算法或解释异常结果的读者。先读第 1.5.1 与第 1.9.1，区分模型选择、能量账本和统计量；再读第 5.14.2.3 与第 5.14.7，把 source residual、解析场、checksum 和收敛趋势拆成不同 consumer；随后以第 8.14–8.16 设计一张诊断记录卡；最后用第 9.8–9.10 复核文献、源码和运行证据能否支持措辞。终点是一份可审查的验证合同，而不是一张看起来合理的场图或一次 regression PASS。

第一次阅读仍可先走路线 A，再按需要进入 B 或 C。无论选择哪条路线，遇到一个新的输入或源码分支，都回到本节前的四步：连续物理量、离散表示、producer/consumer 和带范围说明的 observable。

第一章 动理学模型与 PIC 的基本思想

PIC 代码不是先有“粒子数组”和“场数组”，再去给它们找物理意义。它的上游模型是动理学方程：对每个物种，真实对象首先是相空间分布函数

$$f_s(\mathbf{x}, \mathbf{p}, t),$$

而不是单个粒子轨道。本章的任务是把这条连续模型主线压实到后面源码会反复用到的几个边界：

1. Vlasov 方程本质上是相空间守恒定律，而不只是“一个偏微分方程”。
2. Vlasov-Maxwell 与 Vlasov-Poisson 不是两套无关模型，而是同一条自洽场闭合在不同物理极限下的分支。
3. 宏粒子、权重和 shape factor 不是数值技巧附会到物理上，而是 coarse-grained kinetic model 的一部分。
4. PIC 的主要误差不是单一来源，而是采样噪声、有限粒子权重、有限网格和离散时间层共同作用的结果。

本章的读者路线

不要把本章当作术语表。先在 1.1–1.4 固定连续对象：什么量在相空间中守恒、哪些矩成为场的 source、能量和动量何时可以交换；再在 1.5 判断问题是否真的允许采用静电近似；随后在 1.6–1.10 判断宏粒子、权重、shape、网格和时间平均会怎样改变可见的统计量；最后用 1.11–1.14 把这些连续对象交给第 2 章的离散时间步。每经过一段，都应能回答三件事：程序实际表示什么、与哪一个连续量比较、该比较还不能保证什么。

本章的阅读支点是：

- Birdsall 1985
- Dawson 1983
- 第 2、3、5、6 章对 WarpX 主循环、沉积和场求解器的实现说明

Hockney–Eastwood 与 Yee 1966 在本书中只作为补充书目信息，而不作为可逐页核对的全文依据；本章不把无法核实的历史细节当作结论。读者应先把这里的连续模型、离散变量和误差边界读清，再进入源码章节。

1.1 Vlasov 方程首先是相空间守恒律

对物种 s ，若忽略碰撞、衰变、电离和其他 source/sink，分布函数满足无碰撞相对论 Vlasov 方程

$$\frac{\partial f_s}{\partial t} + \dot{\mathbf{x}} \cdot \nabla_{\mathbf{x}} f_s + \dot{\mathbf{p}} \cdot \nabla_{\mathbf{p}} f_s = 0.$$

这里采用的归一化是

$$dN_s = f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{x} d\mathbf{p}, \quad n_s(\mathbf{x}, t) = \int f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p}.$$

因此, f_s 不是“某个粒子的概率标签”: 它是对一个相空间体元内物种数目的密度。后面用粒子样本近似它时, 必须同时说明粒子位置、动量、权重和 shape; 只写出一条轨道不能定义 f_s 。

对带电粒子,

$$\dot{\mathbf{x}} = \mathbf{v}, \quad \dot{\mathbf{p}} = q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}),$$

因此可写成更常见的形式

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{p}} f_s = 0.$$

这里真正需要记住的不是式子本身, 而是它的守恒含义。把相空间速度记成

$$\mathbf{Z} = (\mathbf{x}, \mathbf{p}), \quad \dot{\mathbf{Z}} = (\dot{\mathbf{x}}, \dot{\mathbf{p}}),$$

则 Vlasov 方程也可写成

$$\frac{\partial f_s}{\partial t} + \nabla_{\mathbf{Z}} \cdot (f_s \dot{\mathbf{Z}}) = 0.$$

若相空间流满足

$$\nabla_{\mathbf{Z}} \cdot \dot{\mathbf{Z}} = 0,$$

就得到 Liouville 图像: 沿特征线传播时, 分布函数值保持不变, 相空间体积也不被压缩或膨胀。对后面的 PIC 来说, 这一点比公式本身更基础, 因为它解释了为什么:

- 粒子推进器不能随意破坏轨道拓扑;
- 离散时间推进若不尊重相空间结构, 就会把数值伪耗散或伪加热写进分布函数;
- Boris / Vay / Higuera-Cary 这类 pusher 的比较, 不只是在比轨道误差, 而是在比它们怎样离散化相空间流。

1.2 碰撞项不是 Vlasov 的一部分, 但必须保留边界

一旦考虑碰撞、衰变、外部源项或粒子数变化, 更一般的 kinetic equation 应写成

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{p}} f_s = C_s[f] + S_s - L_s.$$

这里:

- $C_s[f]$ 表示碰撞算子，可以是 Boltzmann、Landau、Fokker-Planck 或 Monte-Carlo 近似；
- S_s 表示外加源项，如电离生成、注入、衰变产物；
- L_s 表示损失项，如吸收、衰变消失、边界流出。

这条边界在 WarpX 里很重要，因为：

- 主 PIC loop 默认走的是无碰撞 Vlasov-Maxwell 主线；
- CollisionHandler、field ionization、QED、ContinuousFluxInjection、粒子边界吸收/反射，都是往这条无碰撞主线上附加 C/S/L；
- 它们不该被写成“另一个独立程序”，而应被理解成对同一 kinetic balance 的修改。

因此本书后面凡是讲 collisions、ionization、QED、scraping，都应问两个问题：

1. 它改的是分布函数右端的哪一项？
2. 它是在一个时间步的哪个时间层插入进去？

否则很容易把“物理过程存在”和“离散时间组织正确”混成一件事。

1.3 Vlasov-Maxwell：源项、约束与闭合

相对论动量与速度满足

$$\mathbf{p} = \gamma m_s \mathbf{v}, \quad \gamma = \sqrt{1 + \frac{|\mathbf{p}|^2}{m_s^2 c^2}}, \quad \mathbf{v} = \frac{\mathbf{p}}{\gamma m_s}.$$

分布函数的低阶矩给出 Maxwell 方程右端的源项：

$$\begin{aligned} \rho(\mathbf{x}, t) &= \sum_s q_s \int f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p}, \\ \mathbf{J}(\mathbf{x}, t) &= \sum_s q_s \int \mathbf{v}(\mathbf{p}) f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p}. \end{aligned}$$

然后场满足

$$\begin{aligned} \nabla \cdot \mathbf{E} &= \frac{\rho}{\epsilon_0}, \quad \nabla \cdot \mathbf{B} = 0, \\ \nabla \times \mathbf{E} &= -\frac{\partial \mathbf{B}}{\partial t}, \quad \nabla \times \mathbf{B} = \mu_0 \mathbf{J} + \frac{1}{c^2} \frac{\partial \mathbf{E}}{\partial t}. \end{aligned}$$

这四式里最容易被误写的是：Gauss 定律和 $\text{div } \mathbf{B} = 0$ 不是“额外条件”，而是系统闭合的一部分。只要连续性方程

$$\frac{\partial \rho}{\partial t} + \nabla \cdot \mathbf{J} = 0$$

成立，并且初值满足约束方程，Maxwell 演化就会传播这些约束。反过来，如果离散沉积和离散场推进不一致，那么程序里最先坏掉的往往不是 curl 更新，而是：

- $\text{divE} = \rho / \epsilon_0$
- divB
- 边界附近的伪电荷
- 以及随之而来的非物理电场和数值加热

这正是后面第 5 章和第 6 章为什么要反复围着 source synchronization、current correction、Gauss-law regression 打转。

1.4 Vlasov-Maxwell 的能量与动量守恒边界

在闭域、无外源、忽略边界通量时，连续系统满足总能量守恒：

$$\frac{d}{dt} \left[\sum_s \int \gamma m_s c^2 f_s d\mathbf{x} d\mathbf{p} + \int \left(\frac{\epsilon_0}{2} |\mathbf{E}|^2 + \frac{1}{2\mu_0} |\mathbf{B}|^2 \right) d\mathbf{x} \right] = 0.$$

它说明两件事：

1. 粒子动能和场能不是分开各自守恒，而是可以相互交换。
2. 程序里若只监控粒子能量或只监控场能量，都不足以判断离散系统是否健康。

同理，总动量守恒应理解为“粒子动量 + 电磁场动量 + 边界 Maxwell stress”一起守恒，而不是单看粒子束团动量曲线。对后面的 implicit、hybrid、electrostatic sphere、planar pinch 和 FEL 例子，这个边界都非常关键：自动化案例即使检查完整能量账本，也不能由某个单独变量“看起来没漂”来替代。

1.5 Vlasov-Poisson / electrostatic 极限不是另一套世界

当系统关注的是电荷分离和纵向静电响应，而电磁波传播、辐射和横向磁反馈不是主导效应时，可以把自洽场闭合约化到 Vlasov-Poisson：

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s \mathbf{E} \cdot \nabla_{\mathbf{p}} f_s = 0,$$

$$\mathbf{E} = -\nabla \phi, \quad -\nabla^2 \phi = \frac{\rho}{\epsilon_0}.$$

它不是凭空把 Maxwell 方程换掉，而是对应这样一组物理假设：

- transverse electromagnetic radiation 不是主要自由度；
- 场主要由电荷分离决定；
- 电磁传播时间尺度不是当前主导尺度；
- 所关心的现象更接近 Langmuir、space-charge、electrostatic expansion、Poisson boundary-value problem。

因此：

- electrostatic PIC 仍然是 kinetic PIC；
- 只是“粒子如何给场提供源项、场如何回馈粒子”这一闭合从 Maxwell 变成了 Poisson。

这是一项建模假设，不是看到某一帧电场主要沿一个方向后再补上的解释。WarpX 的 `warpx.do_electrostatic` 参数说明也明确指出：该模式不在每一步用完整 Maxwell 方程更新场，传播的电磁辐射和激光等效效应不被捕获。因此，若研究目标包含传播、辐射、激光注入或横向电磁反馈，就不能用“当前图像看起来近静电”替代模型适用性的论证。

这也是为什么：

- 第 6 章不能把 electrostatic solver 当作“Maxwell solver 的低配版”；
- electrostatic sphere、Pierce diode、effective potential 这些例子要和 Poisson 边界条件、势能账本一起讲；
- `WarpX::OneStep()` 里 electrostatic / hybrid 路线的场解位置会和标准 electromagnetic loop 不同。

1.5.1 模型选择与验证卡：Poisson 可解不等于完整电磁问题已被解决

选择 electrostatic 路线时，先把“模型是否适合”与“这一实现是否通过指定验证”分开。下面的四层顺序可防止把一个收敛的 Poisson solve、一次通过的回归或一幅近纵向电场图误读成完整的 Vlasov–Maxwell 验证。

第一层：先判断问题是否仍需传播电磁自由度。若研究对象包含激光注入、辐射传播、波的有限传播时间、横向电磁反馈或与 PML 吸收有关的反射，`warpx.do_electrostatic` 不是近似程度较高的快捷路径，而是模型对象已经变了；应回到完整 electromagnetic 路线。反之，若目标是给定边界势下的空间电荷、静电平衡或 Poisson 边值问题，才继续问 Poisson 解和边界条件是否与目标相符。参数文档的“没有 timestep 限制”只是在说该模式不由 electromagnetic CFL 限制；它既不免除粒子输运、等离子体频率、统计采样或 Poisson 误差的判断，也不授权把过大的步长写成物理上准确。

第二层：明确实际解的是什么。labframe 以所有物种合成的 ρ 解

$$\nabla^2 \phi = -\rho/\epsilon_0, \quad \mathbf{E} = -\nabla \phi.$$

labframe–electromagnetostatic 还以合成的 $\mathbf{J}$ 解矢势并构造静磁场；labframe–effective–potential 则改写 Poisson 算子以改变大 $\omega_{pe} \Delta t$ 时的数值行为。它们不是同一个开关的三个拼写。读者若只需 electrostatic 的 $\phi, \mathbf{E}$ ，不能因为输出里出现了 B 或因为某个模式能稳定运行，就跳过所选模式的方程和适用假设。

第三层：用一个有解析 reference 的 producer 检查指定对象。官方 `test_3d_electrostatic_sphere_eb` 是启用 EB 时的 3D、2-rank 案例：`max_step = 1`、`warpx.do_electrostatic = labframe`、外边界为零电势的 PEC，半径 $R = 0.1\text{ m}$ 的 embedded sphere 固定为 $\phi_0 = 1\text{ V}$ 。它写出 `Ex/Ey/Ez/rho/phi/eb_covered` 的 Full diagnostics，并另写 `ChargeOnEB` 的全球与八分之一球积分。这里的 producer 很窄：它构造的是给定导体势、特定几何、单步、无传播辐射主张的 Poisson/EB 问题，而不是一般带粒子动力学的 plasma benchmark。

第四层：让 consumer 与所问问题一一对应。该案例的 `analysis.py` 比较全嵌入边界电荷

$$Q_{\text{th}} = 4\pi\epsilon_0\phi_0 R$$

与 `eb_charge.txt` 中的数值积分，并要求相对误差小于 **0.06**；它还把八分之一权重积分与 $Q_{\text{th}}/8$ 比较，并检查 `eb_covered` 在 $[0, 1]$ 内、球内带与球外带分别为 1 与 0。这三类断言分别约束全局边界电荷、加权表面积分和 EB 几何 mask。CTest 随后的 checksum 只回归指定输出基线，不能替代前三个比较。

从执行次序看，初始化阶段和每个 electrostatic 时间步都会经 `ComputeSpaceChargeField()` 组织 Poisson 场；进入该分支后，演化代码会重置 electrostatic 分量，再在粒子下一次 push 前准备可 gather 的场。与此同时，`OneStep_sub1()` 明确拒绝 electrostatic solver 与 AMR subcycling 的组合。因此下面的句子都不成立：“Poisson 残差小，所以激光传播也正确”、“sphere 的电荷通过，所以任意 EB 几何的场都正确”、“没有 electromagnetic CFL，所以时间步不再需要物理判断”，以及“这张单步、固定势测试证明了 electrostatic + subcycling 的路径”。

这张卡的实际交付是一张四行表：模型中保留/删去的自由度、输入 producer、各 consumer 的 reference 与阈值、以及本次结论不能外推的范围。这样才把第 1 章的连续假设接到后续第 3A 章初始化、第 6 章场求解和第 7 章 EB/PML 的不同问题上。

1.6 宏粒子不是假粒子，而是 coarse-grained 分布函数载体

PIC 的核心近似不是把等离子体变成少数真实粒子，而是用有限数量的宏粒子采样分布函数。形式上可写成

$$f_s(\mathbf{x}, \mathbf{p}, t) \approx \sum_{p \in s} w_p S_x(\mathbf{x} - \mathbf{x}_p(t)) S_p(\mathbf{p} - \mathbf{p}_p(t)).$$

这里：

- w_p 是宏粒子权重；
- S_x 是空间形函数；
- S_p 常在实际 PIC 中退化成粒子自身在动量空间的离散采样。

从 Dawson 1983 的角度，更准确的说法是：宏粒子不是“把许多真实粒子团成一个球”的形象化故事，而是 coarse-grained kinetic model 的载体。它的目标是：

1. 用有限自由度代表连续分布；
2. 保住真正重要的低阶矩和 collective behavior；
3. 接受某些细粒度 phase-space 结构会被采样误差和 coarse graining 吞掉。

1.7 权重可以不同，但不是没有代价

宏粒子并不必然等权。Dawson 1983 讨论了

$$q_i = -\alpha_i e, \quad m_i = \alpha_i m, \quad \frac{q_i}{m_i} = -\frac{e}{m}$$

这一类不同电荷和质量、但相同荷质比的电子群，并说明由它们组成的加权分布函数仍满足通常的 Vlasov 方程。

这条结论的意义是：

- weighted macroparticles 从一开始就是合法的 kinetic coarse graining；
- 它允许把 phase-space 分辨率集中到真正需要的区域；
- 但它也会引入新的统计与 collisional side effects。

所以“加权宏粒子”更准确的理解不是“自适应采样免费升级”，而是：

- 你获得了 phase-space resolution redistribution；
- 但要付出更复杂的噪声、散射和统计解释代价。

这条边界对后面理解 WarpX 中：

- species 权重
- Gaussian beam / flux injection
- collision/QED product creation
- reduced-dimension weighting compensation

都很重要。

1.8 shape factor 不是插值细节，而是粒子-网格耦合规则

若把粒子源项沉积到网格单元或网格点 i ，最基本的电荷密度形式是

$$\rho_i^n = \frac{1}{\Delta V_i} \sum_p q_p w_p S_i(\mathbf{x}_p^n).$$

场 gather 则用同一类 shape family 从网格插值回粒子位置：

$$\mathbf{E}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{E}_i^n, \quad \mathbf{B}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{B}_i^n.$$

但 shape factor 的意义远不止“双向插值”：

1. 它定义了宏粒子在空间上的 coarse-grained 电荷云。
2. 它决定了粒子-网格耦合的 stencil 宽度。
3. 它会系统改写短波 aliasing、self-force 和统计噪声。
4. 它直接影响 guard-cell 需求、通信宽度和算子局域性。

Birdsall 1985 与 Dawson 1983 的共同结论都指向这一点: finite-size particles 不是为了把图画得更平滑, 而是为了软化 point-charge 的短程奇异作用、压低非物理 collisionality, 并把系统真正保留成“长程 collective physics + 可控短程误差”。

这也是为什么后面的第 5 章必须把:

- shape factor
- charge/current deposition
- sampled density
- finite-grid effects
- aliasing

放在同一章里讲, 而不是把 shape factor 单独缩成一个插值小节。

1.9 PIC 的噪声不是 bug, 而是模型代价

把连续分布函数换成有限宏粒子之后, 最基本的代价就是采样噪声。它不是代码写坏了才出现, 而是:

- 有限粒子数
- 有限权重
- 有限网格
- 有限时间平均

共同带来的统计涨落。

从 Birdsall 1985 的 thermal-plasma 讨论看, 这种噪声不能只被理解成“粒子数不够大”。更准确的图像是:

1. sampled density 会生成 alias branches;
2. shape factor 会修改 fluctuation spectrum;
3. finite Δx 和 finite Δt 会把 continuum 改写成带离散谱结构和 effective transport 的系统;
4. 若离散耦合处理不好, 噪声会演化成 numerical heating、drag、diffusion, 甚至弱不稳定增长率误判。

因此, 本书后面凡是说“噪声更小”“结果更平滑”, 都不应只停在图像层, 而应继续问:

- 是 modal fluctuation level 变了?
- 是 alias branch 被压了?
- 还是只把可见图像平滑了, 但守恒与统计量并没有更好?

1.9.1 统计噪声与能量账本验证卡：能量漂移小不等于热平衡或低噪声

当修改了宏粒子数、shape、gather、滤波或时间步时，最容易发生的误读是：看到总能量曲线“很平”，便宣称热等离子体保持正确、噪声已经降低。二者回答的不是同一个问题。WarpX 的 `energy_conserving_thermal_plasma` 回归给出了一条很有用、但范围明确的能量账本路线。

第一层：先固定它实际产生了什么。官方 CTest 注册了 1D 和 2D 两个 producer，均使用两个 MPI rank；两份输入都是周期边界、`max_step = 500`、`warpx.do_electrostatic = labframe`、二阶 `algo.particle_shape = 2`、`algo.field_gathering = energy-conserving`，并关闭滤波。电子和质子以相同的常数密度、Gaussian 动量分布和 100 eV 温度初始化。1D 每个 cell 每个方向放 4 个宏粒子，2D 每个方向放 2 个；两者都每 100 步写一次 `ParticleEnergy` 与 `FieldEnergy`。这是一份特定离散设置下的周期、双物种、静电热等离子体账本，不是任意 thermal plasma 的定义。

第二层：再核对 **consumer** 到底比较了什么。`ParticleEnergy` 的第一个能量列是所有物种的加权相对论动能，

$$E_p = \sum_i w_i \left(\sqrt{|\mathbf{p}_i|^2 c^2 + m_i^2 c^4} - m_i c^2 \right),$$

而 `FieldEnergy` 的第一个能量列是各场分量构成的场能

$$E_f = \frac{1}{2} \sum_{\text{cells}} \left(\epsilon_0 |\mathbf{E}|^2 + \frac{|\mathbf{B}|^2}{\mu_0} \right) \Delta V.$$

该 `analysis.py` 读取两张 reduced-diagnostic 表的第 3 列，形成 $E = E_f + E_p$ ，并在所有已记录的時刻断言

$$|E(t) - E(0)| / E(0) < 0.003.$$

这既不是“能量只能增加 0.3%”，也不是“严格守恒”：代码取绝对值，所以正、负漂移都受约束；而阈值只约束 producer 所写出的采样时刻。`energy-conserving gather` 的名字也不能代替这条检查。官方算法说明将它定义为从 grid point 直接 gather，并说明相应的能量守恒性质只在 $\Delta t \rightarrow 0$ 的极限成立，有限时间步通常只是具有更好的相应守恒表现。

第三层：明确这张账本没有测量什么。它不计算温度矩、速度分布是否仍为 Maxwellian、密度或电场的 k -spectrum、alias branch、两点相关或 transport coefficient；也没有让 `momentum-conserving`、不同 particle shape 或不同 particle-per-cell 与当前设置进行对照。因此通过该 consumer 不能推出“噪声更低”“热平衡正确”“每个物种能量正确”“数值严格守恒”，更不能外推到非周期边界、AMR 接口、碰撞、激光传播或其他几何。规则网格上 gather/deposition 的配对性质也不能自动越过网格不规则性；官方 AMR 理论明确指出，规则性或对称性被破坏时，接口附近会出现净的 spurious self-force。

第四层：修改后重新建立两本账。若只改变输出格式或保存位置，先保证 E_f 与 E_p 仍在同一物理时刻、同一归一化下被读取，再继续使用能量 consumer。若改动 gather、shape、滤波、 Δt 、宏粒子数、温度/密度、边界、维度或求解模型，则原来的 0.003 已不再是自动有效的合同：应把新的能量漂移包络、 E_f 与 E_p 的分量曲线作为一份数值账本；再把速度矩/分布、谱或相关函数，或与解析解、收敛研究、实验量的比较作为独立的统计或物理账本。只有两本账分别闭合，才可以分别讨论守恒误差与统计噪声。

这张卡的源码证据由官方 thermal-plasma test 的 CTest 注册、1D/2D 输入与 `analysis.py`，以及 `ParticleEnergy`、`FieldEnergy` reduced diagnostic 和官方参数/算法说明共同组成。

精确路径用于定位职责，而不是替代运行。静态核对不会执行该运行；真实改动仍须由读者运行新 producer，并为新 observable 选择 reference 与容差。

1.10 Debye 长度、粒子数与统计时间尺度

Birdsall 1985 对 sheet model 的讨论给了一个比教科书定义更适合写进程序书的视角。

首先，在线性、近 Maxwellian 的三维等离子体中，热速率 v_t 、等离子体频率和 Debye 长度可写为

$$v_t = \sqrt{\frac{k_B T_s}{m_s}}, \quad \omega_{ps} = \sqrt{\frac{n_s q_s^2}{\epsilon_0 m_s}}, \quad \lambda_{Ds} = \frac{v_t}{\omega_{ps}} = \sqrt{\frac{\epsilon_0 k_B T_s}{n_s q_s^2}}.$$

其中 n_s 是物种数密度， T_s 是温度， k_B 是 Boltzmann 常数。对应的 Debye 球内粒子数近似为

$$N_D \sim (4\pi/3)n_s \lambda_{Ds}^3.$$

这些量的精确定义会随单位制、速度矩约定、磁化程度和 reduced geometry 改变；这里使用它们是为了建立尺度判断，而不是把二维或轴对称计算机械地当成三维 Debye 球。Debye 长度 λ_D 和 Debye 球内粒子数 N_D 因而不是孤立的公式，而是“这个 plasma 是否能被当作 collective medium”与“统计噪声会以什么尺度渗入观测量”的共同边界。

其次，在 reduced model 下，

$$\tau \sim \frac{2N_D}{\omega_p}$$

更适合被理解成：

- randomization time
- correlation time
- 统计独立采样间隔

而不是整个分布完全热化成 Maxwellian 的总弛豫时间。后者通常更慢，量级更接近 N_D^2 。对 PIC 用户来说，这比“记住 Debye 长度定义”更实用，因为它直接影响：

- uniform-plasma 噪声底怎么看；
- reduced diagnostics 应平均多久；
- 弱效应、弱不稳定和 Landau damping 的 measurement window 多大才可信。

1.10.1 尺度计算卡：能量回归通过不等于 Debye 屏蔽已分辨

上一节的官方 energy_conserving_thermal_plasma producer 正好说明，测试名和物理判据必须分开读。其 1D 输入设定 $T_e = 100 \text{ eV}$ 、

$$d_{e0} = c/\omega_{pe}, \quad \Delta t = 0.2/\omega_{pe}, \quad L = 10d_{e0}, \quad N_x = 8.$$

同一输入把电子热动量宽度写成 $u_{th} = \sqrt{T_e q_e / m_e} / c$ 。沿用本节的非相对论热速率约定，便有

$$\frac{\lambda_{De}}{d_{e0}} = \frac{v_{te}}{c} = \sqrt{\frac{T_e}{m_e c^2}} = \sqrt{\frac{100}{510998.95}} \approx 1.399 \times 10^{-2}.$$

而网格与一步的无量纲尺度分别为

$$\frac{\Delta x}{d_{e0}} = \frac{10}{8} = 1.25, \quad \frac{\lambda_{De}}{\Delta x} \approx 1.119 \times 10^{-2}, \quad \omega_{pe} \Delta t = 0.2, \quad \frac{v_{te} \Delta t}{\Delta x} \approx 2.24 \times 10^{-3}.$$

先读这四个数各自回答的问题。最后一个数小，只说明一个电子热速率在一步内移动的距离远小于一个 cell；它不能把 $\lambda_{De} / \Delta x \ll 1$ 变成空间分辨。这里的 1D/2D producer 因而可以为其固定 gather、shape、边界和输出 cadence 检查能量账本，却不由该 consumer 证明 Debye 屏蔽、Langmuir 色散、Landau damping 或热涨落谱已经按目标物理分辨。EP/EF 的 **0.003** 断言也没有读取 λ_D 、谱峰或温度矩。

这不是对该 regression 的否定。它的可支持结论本来就是“在这份输入和已记录时刻，指定的粒子动能加场能变化不超过阈值”；用一个本来不测空间分辨率的 consumer 去给它附会分辨率结论，才是错误。相反，宏粒子数也要单独解释：1D 的每 cell 4 个、2D 的每 cell 2×2 个宏粒子是采样设置，不是三维真实 Debye 球内物理粒子数 N_D 。

若研究目标真的需要 Debye 尺度，建立新的分辨率合同。首先从目标物理决定需要比较的 $\lambda_D / \Delta x$ 与时间窗，而不是机械复用本例的 8 cells；随后在固定物理长度、边界、温度、密度与可观察量定义下进行网格和宏粒子数的独立扫描。consumer 至少应包含与问题匹配的 density/field spectrum、温度或速度分布矩、解析色散/阻尼率、收敛趋势或实验 reference 中的一项；同时继续保留 $E_f + E_p$ 作为另一条数值账本。只有“尺度分辨”和“能量漂移”分别有证据，才可以讨论计算是否既稳定又回答了原来的热等离子体问题。

这个计算卡只重新计算官方输入已声明的无量纲量，并不运行 WarpX，也不宣布某个通用的 $\lambda_D / \Delta x$ 通过线。不同几何、模型、可观察量和误差预算需要各自的 reference 与收敛计划。

1.11 从连续模型到 PIC 离散变量

前面的方程还没有直接变成程序里的数组。PIC 的第一步不是把分布函数存成一个高维网格，而是用带权粒子样本代表它，再用网格上的有限差分或谱变量承载场。可以把这条映射写成下面的最小对应关系：

连续对象	PIC 离散载体	典型时间层/位置	在 WarpX 代码中应如何理解
$f_s(x, p, t)$	物种粒子的位置、动量、权重集合	$x_p^n, p_p^{n-1/2}, w_p$	ParticleContainer 中的粒子样本，不是一个逐点存储的分布函数
$\rho(x, t)$	shape-weighted charge density	ρ^n 或 $\rho^{n+1/2}$	由粒子沉积得到；rho_fp 与 rho_buf 还可能分别属于 fine 与 coarse-buffer 路径

连续对象	PIC 离散载体	典型时间层/位置	在 WarpX 代码中应如何理解
$J(x, t)$	trajectory-based current density	通常跨越 $n- > n+1$	Esirkepov/Villasenor 等 current deposition 需要 old/new 轨迹或等价的 crossing 信息
E, B	staggered/collocated grid fields	由 solver 规定	Efield_*, Bfield_* 的后缀表示时间层、网格位置或辅助副本，不能只按变量名猜物理时刻
连续性方程	离散 source synchronization	每个粒子推进步或 solver stage	SyncCurrentAndRho(), guard exchange、AMR average-down 等共同完成可供场求解器使用的 source

最容易被忽略的是 ρ 和 J 的时间语义不同。单时间层的 charge deposition 可以直接对粒子位置取样：

$$\rho_i^n = (1/\Delta V_i) \sum_p q_p w_p S_i(x_p^n),$$

而守恒的 current deposition 必须表达粒子从 x_p^n 到 x_p^{n+1} 的输运：

$$\Delta t (\nabla_h \cdot J)_i = \rho_i^n - \rho_i^{n+1}.$$

这里的第二式不是说所有 current deposition 都在程序中显式先算出左右两边，而是说明它们必须满足同一条离散连续性关系。也因此，

- DepositCharge() 负责单时间层的 ρ 采样及其时间层、几何和 AMR 桥接；
- DepositCurrent() 及其 Esirkepov/Villasenor 等路径负责轨迹输运产生的 J ；
- SyncCurrentAndRho() 负责把不同 level、边界和 source buffer 中的结果整理成 solver 可消费的源项。

这三层不能合并成“粒子把电荷写到网格”一句话。后续第 5 章会从 kernel 角度展开，第 6 章则会继续说明不同 field solver 如何消费这些 source。附录 A 给出 rho_fp、rho_buf、current_fp、current_buf 和 lev 等 WarpX 实现变量的速查定义。

1.12 这一章对后面源码章节的真正约束

到这里，后续读 WarpX 代码时至少要带着下面这些硬问题，而不是只盯函数名：

1. 粒子推进器是否在离散时间层上合理近似了 Liouville 流？
2. 沉积算法是否把连续性方程离散闭合到了 rho/J？

3. field solver 处理的是 Maxwell 还是 Poisson, 约束方程怎样传播?
4. shape factor 和 finite-size particles 是如何改写噪声、aliasing 和 self-force 的?
5. diagnostics 到底在测真实物理量, 还是只看离散噪声底的一个投影?

如果没有这几层边界, 后面源码里的:

- OneStep_nosub
- PushParticlesandDeposit
- SyncCurrentAndRho
- PushPSATD
- ElectrostaticSolver
- ImplicitSolver

都会被读成“工程控制流”, 而不是“连续模型的离散化实现”。

1.12.1 跨章交接卡: 从连续对象进入离散循环

第 1 章回答“哪些物理对象必须被表示”; 第 2 章回答“这些对象在一个离散时间步的什么时刻交换”。因此, 进入第 2 章前, 不应只带着一张函数名清单, 而应先写出下列状态账本:

连续问题	必须保留的离散状态	进入第 2 章后首先核对	不能直接推出的结论
分布函数 f_s 怎样被近似	宏粒子的 x_p 、 p_p 、 w_p 与 shape	2.2 中粒子样本如何 gather、deposit	宏粒子数较多不自动保证相空间的每个细节都被分辨
ρ 与 $\mathbf{J}$ 怎样成为场的 source	单时间层 ρ^n 与跨轨迹的 $\mathbf{J}^{n+1/2}$	2.2–2.3 中连续性方程和 leapfrog 时间层	一次 charge deposition 不能替代守恒的 current deposition 或 source synchronization
$\mathbf{E}, \mathbf{B}$ 怎样作用于粒子又被 source 改写	网格位置、时间层和场表示	2.3–2.5 中交错 FDTD 或谱空间推进	“有 Maxwell 方程”不说明某个 stencil、边界或 solver 组合具有可接受色散
哪些尺度必须由网格和步长承担	ω_p 、 λ_D 、特征速度、 Δt 、 Δx	2.3 的时间层和可分辨尺度讨论	CFL 可通过不表示 $\omega_p \Delta t$ 、 $\lambda_D / \Delta x$ 或统计误差已经满足物理目标

这张表给出一个严格的交接顺序: 先说明粒子和场分别代表什么, 再标出它们所在的时间层, 最后才追踪程序调用。若不先区分“连续对象”“离散状态”和“可比较 observable”, 就会把稳定性、守恒性与物理分辨率误当成同一类判断。第 2 章开头的接读卡将把这四类状态排成一次 $n \rightarrow n+1$ 的实际交换顺序。

1.13 证据范围与继续阅读

本章下列论述可直接回到两部已提供逐段讲解的基础来源：

- Birdsall 1985: sheet model 的 randomization / correlation / thermalization 时间尺度，以及 finite-grid / aliasing / fluctuation / heating 主线。
- Dawson 1983: numerical experiment 视角、superparticle / weighted particles 的 kinetic 边界，以及 finite-size particles、网格和 FFT-Poisson 的 electrostatic contract。

下面两项适合作为后续补充阅读，但本章不依赖其未取得全文的细节：

- Hockney–Eastwood: 加权粒子、heating estimates 和 optimum path 的经典表述。
- Yee 1966: staggered FDTD 与离散约束传播的原始出处。

继续阅读时，应优先回到原始论文、教材或官方文档，并为每一条主张标明它属于连续模型、离散算法还是特定代码实现；不要用其中任一层代替另外两层。第 2 章会把这里的 leapfrog、CFL、Debye 长度和数值色散接到同一条离散主循环上。

1.14 练习与源码定位

1. 变量桥接题：根据 1.11 的映射表，说明为什么 ρ_{fp}/ρ_{buf} 不能直接当作两个不同物理量，并指出它们分别在哪个 AMR/source-synchronization 场景出现。
2. 尺度判断题：给定 $\lambda_D/\Delta x = 0.5$ 和 $v_t\Delta t/\Delta x = 1.2$ ，列出至少两个可能的数值风险，并说明它们分别属于空间分辨率、粒子跨单元输运还是时间推进约束。
3. 源码定位题：从 `Source/Evolve/WarpXEvolve.cpp` 的 `OneStep_nosub()` 开始，而不是从全局搜索结果中猜入口。完成下列三步，并交付一个三行表格（函数、连续对象、调用后可认为“已准备好”的数据）：
 - 记录 `PushParticlesandDeposit()` 与 `SyncCurrentAndRho()` 的相对次序；前者对应粒子输运和源项沉积，后者应使场求解前的 J/ρ 经滤波、guard-cell/MR 同步和边界处理而可被消费。
 - 打开 `WarpX::SyncCurrentAndRho()` 的定义，列出其中至少两项 source 数据准备动作，并说明它们为什么不能由连续方程本身自动保证。
 - 任选一个场更新分支：PSATD 路线继续定位 `PushPSATD()`（定义在 `Source/FieldSolver/WarpXPushFieldsEM.cpp`）；FDTD 路线则定位 `EvolveB()/EvolveE()` 的调用。说明该分支把哪一种离散 Maxwell 闭合推进到下一时间层。

第二章 PIC 总循环：从 Vlasov-Maxwell 到离散时间推进

本章先不急进入某一个 WarpX 函数。生产级 PIC 代码的困难不在于“有粒子、有网格、有 Maxwell 方程”这几个名词，而在于这些对象必须在离散时间层、离散空间布局、并行 guard cells、边界条件和守恒约束之间保持一致。后续逐行读 WarpX 时，本章给出判断代码是否“物理上在做正确事情”的基准。

本章给出的函数名和调用关系是读代码的定位方法，而不是某一份源码树的版本标签。下文的 Source/... 与 Examples/... 均相对于 WarpX 源码根目录；使用不同 WarpX 版本时，应先从 Evolve()、OneStep()、PushParticlesandDeposit() 和 SyncCurrentAndRho() 等职责明确的入口重新建立调用关系，而不要把行号或本机工作区布局当作稳定 API。

Yee 1966 在本书中只承担一个窄的历史定位：有限差分 Maxwell 方程通过合适的场点布置处理导体边界。它不能替代本章的 stencil、时间层或色散推导；这些内容由连续方程、离散推导和现代实现三层分别说明。CartesianYeeAlgorithm.H、FiniteDifferenceSolver.cpp、EvolveB.cpp 与 EvolveE.cpp 的交叉定位可供读者复查，但现代代码不构成对历史论文逐式等价的证明。

2.0.1 从第 1 章进入本章：先写状态，再读调用顺序

承接第 1 章的交接卡，本章把“连续对象的离散表示”排成一次真正的时间交换。读者第一次阅读时，先把下面四个问题写在同一页上：

1. 当前粒子状态是 $x^n, p^{n-1/2}, w$ ，还是某个初始化、碰撞或隐式迭代中的试探状态？
2. 当前 ρ 是对哪个位置时间层的取样， $\mathbf{J}$ 是否已经表示了 $x^n \rightarrow x^{n+1}$ 的粒子输运？
3. 场 $\mathbf{E}, \mathbf{B}$ 位于何种网格和时间层，它们在 gather、沉积和 field solve 前后分别被谁消费？
4. $\omega_p \Delta t$ 、 $\lambda_D / \Delta x$ 、CFL 和统计粒子数分别约束哪一种误差，而不是把它们缩成一个“稳定”标签？

可按如下顺序进入正文：先用 2.1–2.2 把 f_s 、粒子、shape、 ρ 与 $\mathbf{J}$ 对齐；再用 2.3 写清整数步、半整数步和可分辨尺度；随后用 2.4–2.5 区分场更新表示；最后用 2.6 将这些状态接入真实 PIC step。这样，PushParticlesandDeposit()、SyncCurrentAndRho() 与场推进函数才是状态转换的实现，而不是可以脱离物理和时间层独立记忆的调用顺序。

2.1 连续模型：Vlasov-Maxwell 系统

对物种 s ，相空间分布函数 $f_s(\mathbf{x}, \mathbf{p}, t)$ 满足 Vlasov 方程：

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{p}} f_s = 0.$$

相对论动量与速度满足

$$\mathbf{p} = \gamma m_s \mathbf{v}, \quad \gamma = \sqrt{1 + \frac{|\mathbf{p}|^2}{m_s^2 c^2}}, \quad \mathbf{v} = \frac{\mathbf{p}}{\gamma m_s}.$$

电磁场满足 Maxwell 方程：

$$\frac{\partial \mathbf{B}}{\partial t} = -\nabla \times \mathbf{E},$$

$$\frac{\partial \mathbf{E}}{\partial t} = c^2 \nabla \times \mathbf{B} - \frac{\mathbf{J}}{\epsilon_0},$$

以及约束方程

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0}, \quad \nabla \cdot \mathbf{B} = 0.$$

源项来自分布函数的矩：

$$\rho(\mathbf{x}, t) = \sum_s q_s \int f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p},$$

$$\mathbf{J}(\mathbf{x}, t) = \sum_s q_s \int \mathbf{v}(\mathbf{p}) f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p}.$$

PIC 的核心任务就是把这套连续耦合系统离散成两个相互交换信息的对象：宏粒子和网格场。

2.2 宏粒子表示与形函数

宏粒子近似把 f_s 写成有限个带权粒子的和：

$$f_s(\mathbf{x}, \mathbf{p}, t) \approx \sum_{p \in s} w_p S_x(\mathbf{x} - \mathbf{x}_p(t)) S_p(\mathbf{p} - \mathbf{p}_p(t)).$$

这里 w_p 是宏粒子权重， S_x 是空间形函数。把粒子源项沉积到网格单元或网格点 i ，常见电荷密度形式是

$$\rho_i^n = \frac{1}{\Delta V_i} \sum_p q_p w_p S_i(\mathbf{x}_p^n).$$

场 gather 则使用同一类形函数从网格插值回粒子位置：

$$\mathbf{E}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{E}_i^n, \quad \mathbf{B}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{B}_i^n.$$

如果只看这两个公式，很容易以为 PIC 的网格-粒子耦合只是“双向插值”。这个理解不够。电流 $\mathbf{J}$ 的沉积必须表达粒子在一个时间步内的轨迹，否则离散电荷守恒会被破坏。

连续系统满足连续性方程：

$$\frac{\partial \rho}{\partial t} + \nabla \cdot \mathbf{J} = 0.$$

在网格上，电流沉积应尽量满足对应的离散形式：

$$\frac{\rho_i^{n+1} - \rho_i^n}{\Delta t} + (\nabla_h \cdot \mathbf{J}^{n+1/2})_i = 0.$$

这解释了为什么 WarpX 这样的代码会提供 Esirkepov、Villasenor-Buneman、Vay 等沉积分支。它们的差异不是表面上的“把电流放到哪里”，而是如何在离散网格上把粒子轨迹、电荷守恒、网格布局和并行边界结合起来。

2.3 leapfrog 时间层

显式电磁 PIC 的标准时间层是 leapfrog：

- 粒子位置在整数步： $\mathbf{x}^n, \mathbf{x}^{n+1}$ 。
- 粒子动量在半整数步： $\mathbf{p}^{n-1/2}, \mathbf{p}^{n+1/2}$ 。
- 电流自然在半整数步： $\mathbf{J}^{n+1/2}$ 。
- 电磁场在 Yee/FDTD 路径中按半步磁场和整步电场交错推进。

粒子位置推进可写成

$$\frac{\mathbf{x}^{n+1} - \mathbf{x}^n}{\Delta t} = \mathbf{v}^{n+1/2}.$$

动量推进写成抽象形式：

$$\frac{\mathbf{p}^{n+1/2} - \mathbf{p}^{n-1/2}}{\Delta t} = q (\mathbf{E}_p^n + \bar{\mathbf{v}} \times \mathbf{B}_p^n).$$

其中 $\bar{\mathbf{v}}$ 的具体定义取决于 pusher。Boris、Vay、Higuera-Cary 等 pusher 的差别留到粒子推进章节逐行讲解。本章只强调主循环必须给 pusher 提供正确时间层的 $\mathbf{x}$ 、 $\mathbf{p}$ 、 $\mathbf{E}$ 、 $\mathbf{B}$ 。

WarpX 的显式无 subcycling 路径在 `Source/Evolve/WarpXEvolve.cpp` 的 `WarpX::OneStep_nosub()` 开头直接把这个时间层写进注释：

```
Push particle from  $\mathbf{x}^{\{n\}}$  to  $\mathbf{x}^{\{n+1\}}$ 
                    from  $\mathbf{p}^{\{n-1/2\}}$  to  $\mathbf{p}^{\{n+1/2\}}$ 
Deposit current  $\mathbf{j}^{\{n+1/2\}}$ 
Deposit charge density  $\rho^{\{n\}}$ 
```

这四行是读 WarpX 主循环的锚点。任何 field gather、collision、ionization、deposition、sync、field solve 的位置都应围绕这些时间层理解。

2.3.1 ω_p 不是背景常数，而是时间离散必须尊重的最快等离子体尺度

对电子等离子体，最基本的本征时间尺度是 plasma frequency：

$$\omega_p = \sqrt{\frac{n_e e^2}{m_e \epsilon_0}}.$$

它决定了最简单的 Langmuir 振荡周期

$$T_p = \frac{2\pi}{\omega_p}.$$

对显式 leapfrog PIC，稳定和分辨不是同一件事。只要时间层排列正确、场更新满足 CFL，代码也许不会立刻炸掉；但如果

$$\omega_p \Delta t$$

已经接近或超过 1 的量级，那么单步内粒子和场已经跨过了 plasma oscillation 的核心相位结构，后面看到的高噪声、相位误差、非物理 heating，往往不是“分析脚本太苛刻”，而是主循环本身没有分辨这个最快内禀尺度。

这也是为什么 Birdsall 1985 后面会把

$$\omega_p \Delta t, \quad v_t \Delta t / \Delta x$$

都写成数值健康度的第一层控制量。对 WarpX 来说，这条边界不会自动由 `ComputeDt()` 替你保证。`ComputeDt()` 只根据 `solver/CFL`、`const_dt`、`max_dt`、`maxParticleVelocity()` 和 AMR refinement 给出一个可运行步长；它并不知道你要不要精确分辨 Langmuir 振荡、electrostatic shielding 或弱不稳定增长率。

所以本章这里先压实一个最重要的判断：

- `ComputeDt()` 保证的是一层离散稳定性和时间步组织约束；
- $\omega_p \Delta t$ 是否足够小，仍然是物理建模和分辨率设计问题。

2.3.2 λ_D 不只是一条长度定义，它直接约束 Δx

和 ω_p 对偶的空间尺度是 Debye length。对非相对论热电子，

$$\lambda_D = \sqrt{\frac{\epsilon_0 k_B T_e}{n_e e^2}} = \frac{v_{th,e}}{\omega_p}.$$

这条式子把热速度、plasma frequency 和 shielding length 绑在了一起。对 PIC 而言，能否把 plasma 当作 collective medium 与 网格是否真的分辨了 shielding 不是分开的两个问题。

如果

$$\Delta x \gg \lambda_D,$$

那么 cell 内已经把最基本的 shielding 结构粗化掉了。接下来即使宏观波形看起来还能跑，field fluctuation、aliasing、self-force 和 nonphysical collisionality 也会被系统性放大。这就是为

什么第 1 章已经把 λ_D 、 N_D 和统计时间尺度单独拎出来；在第 2 章里，它进一步变成主循环的硬分辨率边界：

- Δt 决定是否分辨 ω_p ；
- Δx 决定是否分辨 λ_D ；
- 两者一起决定 leapfrog + grid PIC 到底是在近似同一个 plasma，还是已经换成了另一个更噪、更热、更强 alias 的离散模型。

2.4 FDTD 场更新的数学骨架

忽略 PML、divergence cleaning、宏观介质和边界时，Yee/FDTD 的主更新可以写成三段：

$$\mathbf{B}^{n+1/2} = \mathbf{B}^n - \frac{\Delta t}{2} \nabla_h \times \mathbf{E}^n,$$

$$\mathbf{E}^{n+1} = \mathbf{E}^n + c^2 \Delta t \nabla_h \times \mathbf{B}^{n+1/2} - \frac{\Delta t}{\epsilon_0} \mathbf{J}^{n+1/2},$$

$$\mathbf{B}^{n+1} = \mathbf{B}^{n+1/2} - \frac{\Delta t}{2} \nabla_h \times \mathbf{E}^{n+1}.$$

这说明一个电磁 PIC step 的顺序不能随意交换。电场更新需要本步沉积出来的 $\mathbf{J}^{n+1/2}$ ，所以粒子推进与电流沉积必须在 `EvolveE(dt)` 之前完成。WarpX 的 FDTD 路径正是这样组织的：

对应的核心源码节选来自 `Source/Evolve/WarpXEvolve.cpp` 的 `WarpX::OneStep_nosub()`，这里保留源项同步和 FDTD 场推进部分；PSATD 分支和 PML 后处理在第 3 章继续展开：

```
// Synchronize J and rho:
// filter (if used), exchange guard cells, interpolate across MR levels
// and apply boundary conditions
SyncCurrentAndRho();

// For extended PML: copy J from regular grid to PML, and damp J in PML
if (do_pml && pml_has_particles) { CopyJPML(); }
if (do_pml && do_pml_j_damping) { DampJPML(); }

ExecutePythonCallback("beforeEsolve");

EvolveF(0.5_rt * dt[0], /*rho_comp=*/0);
EvolveG(0.5_rt * dt[0]);
FillBoundaryF(guard_cells.ng_FieldSolverF);
FillBoundaryG(guard_cells.ng_FieldSolverG);

EvolveB(0.5_rt * dt[0], SubcyclingHalf::FirstHalf, a_cur_time); // We
↪ now have B^{n+1/2}
FillBoundaryB(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

if (m_em_solver_medium == MediumForEM::Vacuum) {
    // vacuum medium
    EvolveE(dt[0], a_cur_time); // We now have E^{n+1}
} else if (m_em_solver_medium == MediumForEM::Macroscopic) {
```

```

    // macroscopic medium
    MacroscopicEvolveE(dt[0], a_cur_time); // We now have E^{n+1}
} else {
    WARPX_ABORT_WITH_MESSAGE("Medium for EM is unknown");
}
FillBoundaryE(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

EvolveF(0.5_rt * dt[0], /*rho_comp=*/1);
EvolveG(0.5_rt * dt[0]);
EvolveB(0.5_rt * dt[0], SubcyclingHalf::SecondHalf, a_cur_time + 0.5_rt
↪ * dt[0]); // We now have B^{n+1}

```

数学动作	WarpX 源码位置
粒子从 $\mathbf{x}^n, \mathbf{p}^{n-1/2}$ 推到 $\mathbf{x}^{n+1}, \mathbf{p}^{n+1/2}$, 并沉积源项	Source/Evolve/WarpXEvolve.cpp: WarpX::OneStep_nosub() 对 PushParticlesandDeposit() 的调用
同步 $\rho, \mathbf{J}$: 滤波、guard cells、AMR、边界 F, G 半步、 $\mathbf{B}$ 半步	同一函数中的 SyncCurrentAndRho() 同一函数 FDTD 分支中的 EvolveF()、EvolveG()、EvolveB(...FirstHalf...)
$\mathbf{E}$ 整步	同一分支中的 EvolveE() 或 MacroscopicEvolveE()
F, G 半步、 $\mathbf{B}$ 半步	同一分支中的第二个 EvolveF()、EvolveG()、EvolveB(...SecondHalf...)
PML 与 guard cell 处理	同一分支中的 DampPML() 与 FillBoundary*()

这里的 F, G 是 divergence cleaning 相关辅助场，不是最小 Maxwell 更新必需项。WarpX 把它们插在场推进两侧，是为了在实际模拟中控制 $\nabla \cdot \mathbf{E}$ 或 $\nabla \cdot \mathbf{B}$ 误差及 PML 相关处理。

2.4.1 CFL 不是经验参数，而是离散 Maxwell 更新的因果上界

WarpX 的 ComputeDt() 不会把 FDTD 时间步写死成 $\min(\Delta x_i)/c$ ，而是把这件事委托给具体差分算法。Source/Evolve/WarpXComputeDt.cpp 的 WarpX::ComputeDt() 直接在主类层完成 solver 分派：

```

} else if (electromagnetic_solver_id ==
↪ ElectromagneticSolverAlgo::PSATD) {
    deltat = cfl * minDim(dx) / PhysConst::c;
} else {
    #if defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER)
        if (electromagnetic_solver_id == ElectromagneticSolverAlgo::Yee) {
            deltat = cfl * CylindricalYeeAlgorithm::ComputeMaxDt(dx,
↪ n_rz_azimuthal_modes);
        }
    #elif defined(WARPX_DIM_RSPHERE)
        if (electromagnetic_solver_id == ElectromagneticSolverAlgo::Yee) {
            deltat = cfl * SphericalYeeAlgorithm::ComputeMaxDt(dx);
        }
    #else
        if (grid_type == GridType::Collocated) {
            deltat = cfl * CartesianNodalAlgorithm::ComputeMaxDt(dx);
        } else if (electromagnetic_solver_id ==
↪ ElectromagneticSolverAlgo::Yee

```

```

        || electromagnetic_solver_id ==
        ↪ ElectromagneticSolverAlgo::ECT) {
            deltat = cfl * CartesianYeeAlgorithm::ComputeMaxDt(dx);
    } else if (electromagnetic_solver_id ==
    ↪ ElectromagneticSolverAlgo::CKC) {
            deltat = cfl * CartesianCKCAlgorithm::ComputeMaxDt(dx);

```

对标准 Cartesian Yee，真正的 CFL 上界由 Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceAlgorithms/CartesianYeeAlgorithm.H 中的 CartesianYeeAlgorithm::ComputeMaxDt() 给出：

```

static amrex::Real ComputeMaxDt ( amrex::Real const * const dx ) {
    using namespace amrex::literals;
    amrex::Real const delta_t = 1._rt / ( std::sqrt( AMREX_D_TERM(
        1._rt / (dx[0]*dx[0]),
        + 1._rt / (dx[1]*dx[1]),
        + 1._rt / (dx[2]*dx[2])
    ) ) * PhysConst::c );

    return delta_t;
}

```

也就是

$$\Delta t_{\max}^{\text{Yee}} = \frac{1}{c \sqrt{\Delta x^{-2} + \Delta y^{-2} + \Delta z^{-2}}}.$$

它表达的是离散光锥约束：单步内，Yee curl stencil 不能让信息传播得比离散网格所允许的因果速度更快。warpx.cfl 只是把这个严格上界再乘一个安全系数，不是“拍脑袋调参”。

2.4.2 数值色散从这里开始：连续光锥被离散 stencil 改写

一旦用有限差分近似 curl，连续真空色散关系

$$\omega^2 = c^2 |\mathbf{k}|^2$$

就不再原样保留。以 Yee 为例，差分导数对应的是

$$k_d \rightarrow \frac{2}{\Delta d} \sin \frac{k_d \Delta d}{2},$$

于是离散色散关系变成近似的

$$\sin^2 \frac{\omega \Delta t}{2} = c^2 \Delta t^2 \sum_d \frac{\sin^2(k_d \Delta d / 2)}{\Delta d^2}.$$

这意味着 phase velocity 和 group velocity 都会偏离连续值，且偏离大小依赖传播方向、波数和网格各向异性。数值色散不是后处理图里才会出现的现象，它在 UpwardDx() / DownwardDx() 这种最基本的差分定义处就已经写进去了。

Yee 的 x 向导数由同一文件中的 CartesianYeeAlgorithm::UpwardDx() 与 DownwardDx() 实现：

```

static amrex::Real UpwardDx (
    amrex::Array4<amrex::Real const> const& F,
    amrex::Real const * const coefs_x, int const /*n_coefs_x*/,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    amrex::Real const inv_dx = coefs_x[0];
    return inv_dx*( F(i+1,j,k,ncomp) - F(i,j,k,ncomp) );
}

template< typename T_Field>
static amrex::Real DownwardDx (
    T_Field const& F,
    amrex::Real const * const coefs_x, int const /*n_coefs_x*/,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    amrex::Real const inv_dx = coefs_x[0];
    return inv_dx*( F(i,j,k,ncomp) - F(i-1,j,k,ncomp) );
}

```

也就是 staggered grid 上的前/后向一阶差分:

$$D_x^+ F_i = \frac{F_{i+1} - F_i}{\Delta x}, \quad D_x^- F_i = \frac{F_i - F_{i-1}}{\Delta x}.$$

这里的 Upward/Downward 不只是“正向/反向”, 而是在 nodal 与 cell-centered 位置之间搬运离散导数。正是这种 staggered 几何, 让 Yee 在保持二阶精度的同时把 E/B 交错布置起来。

2.4.3 Yee / Nodal / CKC 的差别本质上是离散色散关系不同

对 collocated/nodal solver, WarpX 的 CartesianNodalAlgorithm 不再用 staggered 前后差分, 而是直接用中心差分。可从 Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceAlgorithms/CartesianNodalAlgorithm.H 的 UpwardDx() 与 DownwardDx() 回查:

```

static amrex::Real UpwardDx (
    amrex::Array4<amrex::Real const> const& F,
    amrex::Real const * const coefs_x, int const /*n_coefs_x*/,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    Real const inv_dx = coefs_x[0];
    return 0.5_rt*inv_dx*( F(i+1,j,k,ncomp) - F(i-1,j,k,ncomp) );
}

static amrex::Real DownwardDx (
    amrex::Array4<amrex::Real const> const& F,
    amrex::Real const * const coefs_x, int const n_coefs_x,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    return UpwardDx( F, coefs_x, n_coefs_x, i, j, k, ncomp);
}

```

所以 nodal grid 上 UpwardDx 和 DownwardDx 等价, 说明这里已经没有 Yee 那种 staggered 位置语义, 而是一个 collocated 中心差分系统。它的 CFL 上界虽然在当前实现里和 Yee 一样都是

$$\Delta t_{\max} \sim \frac{1}{c \sqrt{\sum_d \Delta d^{-2}}},$$

但数值色散与奇偶模耦合特征已经不同。

CKC 则更进一步，不再满足“一个方向只看一对最近邻”的局部导数定义。可从 `Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceAlgorithms/CartesianCKCAlgorithm.H` 的 `ComputeMaxDt()` 与 `UpwardDx()` 回查：

```
static amrex::Real ComputeMaxDt ( amrex::Real const * const dx ) {
#ifdef WARPX_DIM_1D_Z
    amrex::Real const delta_t = dx[0]/PhysConst::c;
#elif defined WARPX_DIM_XZ
    amrex::Real const delta_t = std::min( dx[0], dx[1] )/PhysConst::c;
#else
    amrex::Real const delta_t = std::min( dx[0], std::min( dx[1],
        ↪ dx[2] ) ) / PhysConst::c;
#endif
    return delta_t;
}

...
return alphax * (F(i+1,j ,k ,ncomp) - F(i, j, k ,ncomp))
    + betaxy * (F(i+1,j+1,k ,ncomp) - F(i ,j+1,k ,ncomp))
    + F(i+1,j-1,k ,ncomp) - F(i ,j-1,k ,ncomp))
    + betaxz * (F(i+1,j ,k+1,ncomp) - F(i ,j ,k+1,ncomp))
    + F(i+1,j ,k-1,ncomp) - F(i ,j ,k-1,ncomp))
```

这说明 CKC 的真实目标不是“换一套写法”，而是通过更宽的横向耦合 stencil 改写离散色散关系，从而改善高方向性传播下的 phase error。代价则是：

- stencil 更宽；
- 算法/geometry 适用范围更窄；
- guard-cell 和边界处理的组合空间更复杂。

2.5 PSATD 与 FDTD 的主循环差异

FDTD 在实空间用局部 stencil 近似 curl。PSATD 则在谱空间解析积分线性 Maxwell 方程的一部分。物理方程相同，但离散算法不同：

- FDTD 的优势是局部、显式、边界和 AMR 工程路径相对直接。
- PSATD 能显著降低数值色散，适合相对论束流、激光等问题，但对 FFT、并行 domain decomposition、边界、current correction 和时间平均场有更复杂要求。

这条分界线和前面几节正好连起来：

- leapfrog 规定了粒子、场和源项的时间层关系；
- ω_p 与 λ_D 规定了 plasma 自身是否被给定的 $\Delta t/\Delta x$ 分辨；
- CFL 规定了 Maxwell 更新是否还能保持离散因果；

- Yee/Nodal/CKC/PSATD 则进一步决定同一组 $\Delta t, \Delta x$ 会把波动相速度、群速度和 aliasing 改写成什么样。

所以“主循环能跑”不等于“主循环近似的是对的物理系统”。真正的 PIC loop 要同时满足：

time-layer consistency + charge/source consistency
+ physical scale resolution
+ solver-dependent stability/dispersion control.

WarpX 在 `OneStep_nosub()` 内部把两者清楚分开：

- PSATD 分支在 `Source/Evolve/WarpXEvolve.cpp` 的 `WarpX::OneStep_nosub()` 内，核心是 `PushPSATD(a_cur_time)`。
- FDTD 分支在同一函数内，核心是 `EvolveB/EvolveE/EvolveB`。

这意味着本书后续讲 field solver 时不能把“Maxwell solver”写成单一算法。`algo.maxwell_solver` 的选择会改变主循环内的场推进、同步、边界和可用功能。

2.6 一个完整 PIC step 的离散组织

把物理动作映射到程序实现，一个时间步至少包含这些层次：

1. 用户 callback、信号、诊断、负载均衡和步长更新。
2. 场 gather 前的 guard cell 与 auxiliary field 准备。
3. 电离、QED、粒子注入等改变粒子集合的多物理模块。
4. 粒子推进、碰撞、沉积电流和电荷。
5. 源项同步：滤波、guard cells、AMR fine/coarse 交换、边界条件。
6. 场推进：FDTD、PSATD、implicit、electrostatic 或 hybrid。
7. PML、moving window、粒子边界、重分布、排序。
8. 诊断写出和终止条件检查。

WarpX 在 `Source/Evolve/WarpXEvolve.cpp` 的 `WarpX::Evolve()` 中正是围绕这些层次组织外层循环。真正的主循环不是教科书五行伪代码，而是把守恒离散化、时间层一致性和大规模并行工程组合起来的控制流。

读者的时间层判断卡

遇到一个包含 AMR、JRhom 或 implicit 的输入时，先不要按函数出现次数数“执行了多少步”。先写出外层物理时间区间 $t^n \rightarrow t^{n+1}$ ，再给每次内部重复贴上唯一标签：细层的真实推进、source 的时间积分，或非线性求解的试探。最后才检查该路径在下次 gather 前交出了哪一个 ρ 、 $\mathbf{J}$ 、 $\mathbf{E}$ 、 $\mathbf{B}$ 时间层。这个顺序能防止把 subcycling、JRhom 与 implicit 都误读成同一种“多调用几次”的算法。

2.6.1 AMR subcycling: 两个时间步不是同一个时间步的重复调用

无 subcycling 时, 第 0 层和更细层使用同一个外层时间步, `OneStep_nosub()` 可以把粒子推进、source synchronization 和场推进看成一条统一的 $n \rightarrow n+1$ 链。打开 subcycling 后, 这个图像不再成立。`OneStep_sub1()` 的这一路径明确限定: 只支持两级 mesh refinement, 且每个方向的 refinement ratio 必须为 2。

令粗层时间步为 Δt_c , 细层时间步为

$$\Delta t_f = \frac{\Delta t_c}{2}.$$

一个粗层周期内的基本推进结构是:

阶段	细层	粗层/母网格	源项职责
第一个半周期	推进一次粒子和场, 步长 Δt_f	暂不完成整步	将 <code>current_fp</code> 、 <code>rho_fp</code> 限制到 coarse patch
中间同步	细层已到 $t + \Delta t_f$	粗层推进到相应中间时间	合并细层、粗层和 buffer 中的源项
第二个半周期	再推进一次粒子和场, 步长 Δt_f	继续完成粗层剩余半步	第二段细层源项限制/合并后参与粗层场更新
粗层周期末	到 $t + \Delta t_c$	到 $t + \Delta t_c$	粗细层的场、源项和 guard cells 重新同步

因此, subcycling 不是简单地把 `OneStep_nosub()` 调两次。粗层粒子只推进一次, 而细层粒子推进两次; 粗层场的更新还要消费两个细层时间片上累积并平均/合成后的电流。源码中的调用顺序可以压缩成:

```
fine push/deposit at  $t \rightarrow$  restrict source  $\rightarrow$  fine field half/full/half,
coarse push/deposit at  $t \rightarrow$  combine coarse+fine source  $\rightarrow$  coarse field advance,
fine push/deposit at  $t + \Delta t_f \rightarrow$  restrict source  $\rightarrow$  fine field half/full/half,
combine second fine source  $\rightarrow$  coarse field completion.
```

这里的 `restrict` 和 `add` 不能与普通 guard-cell exchange 混为一谈: 前者改变的是 coarse/fine source 的层级表示, 后者只是同一层相邻 patch 间的数据可见性。对电荷来说, `rho_buf` 还可能来自 transition-zone 粒子在 coarse 几何上的直接沉积; 因此 subcycling 中的 source 合成既不是“把 fine rho 全部平均下来”这么简单, 也不是由场求解器自动补齐。

`OneStep_sub1()` 开头的断言拒绝 electrostatic solver 与 subcycling 的组合。原因不是 electrostatic 不能使用 AMR, 而是这条 subcycling 例程的时间组织只为显式 electromagnetic field advance 编写, 不能把 Poisson/electrostatic 路径的源项和场解时序悄悄套进来。

所以读 AMR PIC loop 时要同时检查四个不变量:

1. 细层是否确实使用 $\Delta t_c/2$, 且在一个粗层周期内推进两次;
2. 粗层粒子是否只推进一次, 避免重复计数粒子输运;
3. 两个细层时间片的 current/rho 是否分别完成 restrict、边界合并和 coarse source add;

4. 粗细层字段与 auxiliary/guard data 是否在下一 gather 前重新可见。

这四项共同构成 AMR subcycling 的时间一致性条件。缺少其中任何一项，都可能得到“每个 patch 都成功更新”但跨层电荷守恒、场相位或粒子 gather 已经不一致的结果。

2.6.2 JRhom 与 implicit: 同一个外层 step 内部也可能有不同的时间合同

标准 OneStep_nosub()、PSATD-JRhom 和 implicit solver 都可能被外层 WarpX::OneStep() 视为一次迭代，但它们内部对“源项在什么时候被求值”的定义不同。从 WarpX::OneStep() 的分派和各路径的函数职责看，应按下面三条路径分别阅读：

1. 标准显式 **electromagnetic PIC**。入口是 OneStep_nosub()；一次粒子推进产生 J 与 rho，随后统一进入 SyncCurrentAndRho()。场端是 FDTD 的 B-E-B 或一次 PSATD 推进；普通显式 collision placement 可以按各自时间层与它组合。
2. **PSATD-JRhom**。入口是 OneStep_JRhom()；它先推进粒子但跳过普通沉积，再按 rho/J 的时间依赖在 Δt 内做多次相对时间沉积。每个 deposit interval 都执行一次谱空间场推进，并可选跨 $2\Delta t$ 时间平均；该路径只支持 PSATD，且不支持 current_correction 与 split momentum collision push。
3. **implicit electromagnetic PIC**。入口是 ImplicitSolver::OneStep()；以 $E^{n+\theta}$ 或中间场为猜测，在非线性/线性 RHS 评估中反复推进粒子并构造 $J^{n+1/2}$ 。nonlinear solver 先求自洽中间电场，再完成粒子和场的后半步；一次 RHS 评估不是一次物理时间步，mass-matrix/JFNK 也会改变 J 的构造路径。

JRhom 的关键不是“PSATD 多调用几次”，而是把时间依赖的源项显式建模成一组谱空间可消费的历史量。OneStep_JRhom() 的顺序可以压缩为：

1. 以 skip_deposition=true 把粒子从 $x^n, p^{n-1/2}$ 推到 $x^{n+1}, p^{n+1/2}$ ；
2. 把 E/B 以及可选的 divergence-cleaning 场变换到谱空间；
3. 按 time_dependency_rho 和 time_dependency_J 在相对时间上沉积 rho_new 与 J，每次执行 filter、guard/AMR 同步和 Fourier transform；
4. 对每个子区间执行 PSATDPushSpectralFields()，最后把场变回实空间并处理 PML、边界和 guard cells。

其中 m_JRhom_subintervals 把一个外层步拆成 $\text{sub_dt} = \Delta t / n_{\text{deposit}}$ 的多个沉积区间；打开 time averaging 时，内部循环还会覆盖两个完整时间步的源/场积分。因而 JRhom 中的 rho 和 J 不是标准显式路径里“本步各沉积一次”的同义词，不能直接用 OneStep_nosub() 的单点时间图解释它。

implicit 路径的差异更加根本。以 SemiImplicitEM::OneStep() 为例，程序先保存粒子和旧场，将磁场推进到半步，然后由 nonlinear solver 反复调用 ComputeRHS()；ImplicitSolver::PreRHSOp() 在每次 RHS 构造里：

- 用当前猜测的中间电场推进粒子；
- 形成半步电流；

- 在需要时把 `current_fp_non_suborbit`、mass matrices 或 JFNK 线性阶段的贡献合并进 J；
- 最后调用 `SyncCurrentAndRho()`，再把同步后的源项交给隐式残差/线性系统。

所以 implicit 中必须区分三个层次：

1. 物理时间步 $t^n \rightarrow t^{n+1}$ ；
2. 非线性迭代中的中间场猜测 $E^{n+\theta}$ ；
3. 每次 RHS 或 Jacobian 评估中的粒子/source 重算。

如果把第 3 层误写成“程序又推进了一个物理时间步”，就会错误理解粒子数、能量账本和 `SyncCurrentAndRho()` 的调用次数。相反，如果把 JRhom 的多个 deposit interval 当成 nonlinear iteration，也会把时间积分和求解器迭代混为一谈。

本章余下章节的阅读规则因此固定为：先识别外层物理时间步，再识别内部的 source subinterval 或 nonlinear iteration，最后才判断某次 `PushParticlesandDeposit()` 是物理推进、试探性 RHS 构造，还是历史源项重建。

读者可以用下表快速定位一个输入卡采用的时间组织：

优先判断	进入路径	这一外层步内部发生什么
m_implicit_solver 已创建	<code>ImplicitSolver::OneStep()</code>	多次 nonlinear iteration/RHS source rebuild，最后才提交一个物理时间步
psatd.JRhom 开启	<code>OneStep_JRhom()</code>	多个相对时间的 rho/J 沉积区间和对应的 PSATD 谱推进
AMR subcycling 开启	<code>OneStep_sub1()</code>	细层做两次 <code>dt/2</code> 推进并限制/合并源项，粗层做一次 <code>dt</code> 推进
以上均否	<code>OneStep_nosub()</code>	push/deposit、 <code>SyncCurrentAndRho()</code> ，再进行 FDTD 或 PSATD 场推进

图中三种“内部多次执行”含义不同：implicit 的重复是求解器试探，JRhom 的重复是同一物理时间步内的源项时间积分，subcycling 的重复则是真实的细层物理推进。只有最后完成的外层调用才代表一次可提交的物理时间步。

2.7 本章后的源码阅读入口

读者现在可以从三个源码入口继续：

目标	入口
看外层时间步如何组织	<code>Source/Evolve/WarpXEvolve.cpp: WarpX::Evolve()</code>

目标	入口
看显式电磁无 subcycling 的标准 step	Source/Evolve/WarpXEvolve.cpp: WarpX::OneStep_nosub()
看主循环如何进入粒子容器	Source/Evolve/WarpXEvolve.cpp: WarpX::PushParticlesandDepos it()
看两级 AMR subcycling 的细/粗层时间组织	Source/Evolve/WarpXEvolve.cpp: WarpX::OneStep_sub1()
看 JRhom 的多次相对时间沉积与谱推进	Source/Evolve/WarpXEvolve.cpp: WarpX::OneStep_JRhom()
看 implicit RHS 中的粒子推进与 source syn- chronization	Source/FieldSolver/ImplicitSol vers/ImplicitSolver.cpp: Implic itSolver::PreRHSOp()

2.8 参数示例与最小运行案例

如果把本章压回一个最小、可运行、可验证的输入骨架，可从下面的官方案例开始：

- Examples/Tests/langmuir/inputs_test_1d_langmuir_multi

它把本章真正讨论的五类量都放在同一个最小问题上：

- geometry.dims = 1
- algo.current_deposition = esirkepov
- algo.field_gathering = energy-conserving
- warpx.cfl = 0.8
- max_step = 80
- 周期场边界

也就是说，本章的抽象讨论并不是悬空的。这里的：

- leapfrog 时间层
- ω_p
- λ_D
- FDTD curl 更新
- rho/J 连续性关系

都能在这条最小 Langmuir 主线上落到真实输入。

Examples/Tests/langmuir/CMakeLists.txt 将这张输入卡注册为 test_1d_langmuir_multi：它使用两个 MPI rank，并把末态 plotfile diags/diag1000080 交给 analysis_1d.py。这给出了一条读者可回查的闭环，而不是一串脱离上下文的历史数字：

1. 输入卡设置 `max_step = 80`、周期边界、`esirkepov` 沉积和第 40/80 步的 `full diagnostics`;
2. `analysis_1d.py` 从传入的末态 `plotfile` 读取 `Ez`，按脚本中的解析 `Langmuir` 解重建同一时刻的场，并要求 `error_rel < 0.05`;
3. 脚本随后调用 `analysis_utils.py` 的 `check_charge_conservation(data)`。对这张 `esirkepov`、非 `PSATD`、非 `RZ` 的输入，`helper` 实际检查

$$\frac{\max |\operatorname{div} E - \rho/\epsilon_0|}{\max |\rho/\epsilon_0|} < 10^{-11}.$$

这里的 `divE` 是诊断输出中的离散散度字段；这项 `gate` 检查的是该离散表示下的 `Gauss-law/source` 一致性，不是“任意几何、边界、粒子形函数和求解器组合都守恒”的证明。单次运行得到的具体误差会随构建、并行布局和案例修改而变化，因此读者应以分析脚本的定义和阈值为准，而不是把某次残差数值当成通用常数。

因此，完成这个案例的最低交付应包括：输入参数、所消费的末态诊断、解析场误差和离散 `Gauss-law` 误差分别说明什么，以及它们不能外推到哪些算法组合。这样才真正把连续模型、离散方程和可运行的验证量连起来。

2.8.1 第一次运行的读者路线：构建、CTest 与手动分析各自回答什么

上一节给出了一个可回查的案例，但“有输入文件”不等于“已按正确维度编译、以预期 `rank` 数运行并消费输出”。首次操作时，把下面三层分开，能避免绝大多数把构建错误、调度错误和物理误读混为一谈的问题。

第一层是构建能力。官方 `CMake` 文档说明维度是编译期选项，而当前源码允许 `1`、`2`、`3`、`RZ`、`RCYLINDER` 和 `RSPHERE`。本节的输入是 `1D`，因此建立一个明确的 `1D build directory`：

```
export WARPX_ROOT=/path/to/warpx
export WARPX_BUILD="$WARPX_ROOT/build-1d"

cmake -S "$WARPX_ROOT" -B "$WARPX_BUILD" -DWarpX_DIMS=1
cmake --build "$WARPX_BUILD" -j 4
```

不要把这个命令理解为“任意输入都能运行”。`WarpX` 的可执行文件名会编码编译选项；当前构建逻辑同时为 `1D application` 建立 `warpx.1d` 链接。因此，在继续前应确认实际二进制与输入的 `geometry.dims` 相容，而不是拿一个 `3D`、`RZ` 或旧 `build directory` 的可执行文件碰运气。

第二层是 `CTest` 参考执行。在默认 `WarpX_APP` 与 `BUILD_TESTING` 路径中，`Examples/` 会登记测试；`test_1d_langmuir_multi` 的 `CMake` 条目固定了 `1D`、`2-rank`、输入、分析脚本和 `checksum`。注意：这个名字是 `test set`，不是 `CTest` 的单个条目。

`WarpX` 的 `add_warpx_test()` 实际创建 `.run`、`.analysis` 和 `.checksum` 三项。因此先用 `-N` 只列出将匹配的项：

```
ctest --test-dir "$WARPX_BUILD" \
-N -R '^test_1d_langmuir_multi\.*'
```

正确配置时，这一步应列出：

```
test_1d_langmuir_multi.run
test_1d_langmuir_multi.analysis
test_1d_langmuir_multi.checksum
```

若总数为零，应先检查 build 是否启用了测试、是否包含 1D/MPI 能力，以及正则是否保留了字面量点号和后缀；零项不是一次通过的 **Langmuir** 验证。确认列表后，再执行同一组：

```
ctest --test-dir "$WARPX_BUILD" \
  -R '^test_1d_langmuir_multi\..*' \
  --output-on-failure
```

三项的职责和顺序也应分开读：`.run` 在 test working directory 以内按登记的两个 MPI rank 生成 producer 输出；`.analysis` 依赖 `.run`，调用 `analysis_1d.py diags/diag1000080` 检查解析场和离散 Gauss-law；`.checksum` 又依赖 `.analysis`，将指定输出交给默认回归 consumer。于是 `.run` 单独成功只能说明程序按该启动配置完成，不能代替解析场比较；反过来，三项共同通过也只支持这一条被登记的输入/consumer 合同，不等于已经对任意参数覆盖、任意硬件后端或完整 Langmuir 理论完成验证。

第三层是手动运行与复核。只有需要改变输入、保留自己的输出或观察中间 diagnostics 时，才绕开 CTest。应在新的运行目录中复制输入和分析器，而不是把 `diags/` 写进源码案例目录：

```
export RUN_DIR=$HOME/warpx-work/langmuir-1d
mkdir -p "$RUN_DIR"
cp "$WARPX_ROOT/Examples/Tests/langmuir/inputs_test_1d_langmuir_multi"
  ↪ "$RUN_DIR/inputs"
cp "$WARPX_ROOT/Examples/Tests/langmuir/analysis_1d.py" "$RUN_DIR/"
cp "$WARPX_ROOT/Examples/Tests/langmuir/analysis_utils.py" "$RUN_DIR/"

cd "$RUN_DIR"
mpirun -np 2 "$WARPX_BUILD/bin/warpx.1d" inputs
python analysis_1d.py diags/diag1000080
```

最后一行需要可用的 `yt`、`numpy`、`scipy` 和 `matplotlib` 分析环境；这与 C++ 可执行文件能否启动是两项不同前提。直接运行会生成 `warpx_used_inputs`，其中包含显式和隐式使用的参数；诊断默认写到 `diags/`。两者应与终端输出一一起保存，才能说明此次运行实际使用了什么输入。若改动过 `max_step`、`diagnostics` 名称或输出周期，`diags/diag1000080` 未必仍存在，分析脚本和 consumer 路径也必须随之重新核对。

这三层形成一条首运行合同：先让编译维度匹配输入，再用 CTest 确认官方注册的 **producer/consumer** 路径，最后才在独立目录修改输入并重新定义 **analysis**。任何一层成功都不能替代另外两层；尤其是“程序退出为零”既不自动产生解析误差 gate，也不自动证明手动修改后的输入仍满足原测试的结论范围。

2.8.2 受控修改路线：一个命令行覆盖会改变哪一份证据

第一次改输入时，最常见的错误不是参数语法，而是继续把原案例的 `analysis` 当成新运行的结论。WarpX 允许在命令行覆盖输入项，并把显式和隐式生效的参数写进 `warpx_used_inputs`；这使覆盖操作可追溯，却不让原来的 consumer 自动适配。

以本节 Langmuir 输入为例，`max_step = 80` 与 `diag1.intervals = 40` 共同产生第 40 与第 80 步的 Full diagnostics；CTest 登记的 `analysis_1d.py diags/diag1000080` 明确消费后者。下面的短运行只适合检查“命令行覆盖是否真正进入本次 producer”：


```
export PROBE_DIR=$HOME/warpx-work/langmuir-max-step-10
mkdir -p "$PROBE_DIR"
cp "$WARPX_ROOT/Examples/Tests/langmuir/inputs_test_1d_langmuir_multi"
  ↪ "$PROBE_DIR/inputs"

cd "$PROBE_DIR"
mpirun -np 2 "$WARPX_BUILD/bin/warpx.1d" inputs max_step=10 \
  > stdout.log 2>&1
rg '^max_step' warpx_used_inputs
```

这里 `warpx_used_inputs` 中的 `max_step = 10` 才是“覆盖已被消费”的证据；`stdout.log` 只能帮助定位启动、MPI 或参数错误。因为步数 10 没有达到这个输入的 40-step 输出间隔，原来注册的 `diags/diag1000080` 不会由此产生。故而下面这条命令在该 probe 上没有意义：

```
python analysis_1d.py diags/diag1000080
```

它失败或找不到文件不能说明 Langmuir 物理失败，只说明 consumer 仍指向原 producer 的末态。反过来，若手工预先留下同名旧目录，脚本甚至可能读取旧数据，造成更危险的假阳性。因此每个参数试验应有独立目录，且开始前确认 `diags/` 的时间步与本次 `warpx_used_inputs` 相符。

要重新获得一个有物理含义的比较，读者有两条不同的路线：

1. 保持官方合同。不改 `max_step`、`diag1.intervals`、场字段或解析模型参数，运行到 80 步，再把新生成的 `diags/diag1000080` 交给原 `analysis_1d.py`。这能重新检查指定 1D、2-rank 输入的解析场与离散 Gauss-law gate。
2. 建立新合同。若改动步数、diagnostic 周期、网格、沉积、边界或理论参数，先写明新的 producer 文件、consumer 读取字段、reference、容差与不可外推范围。`analysis_1d.py` 通过命令行参数取得文件名，并以文件内时间重建理论场；但这不自动证明原来的 0.05 阈值和 `check_charge_conservation` 的适用条件在修改后仍合理。

因此，一个最小的修改记录至少回答四个问题：哪一项输入被覆盖、`warpx_used_inputs` 如何证实它、这次实际写出了什么 `diagnostics`、哪个 consumer 因何仍然或不再适用。这比保存一串终端输出更接近可重复的数值实验，也把“配置变了”与“物理结论变了”分开。

2.9 基础文献与证据范围

本章直接依托的基础来源是：

- Birdsall 1985
 - leapfrog 最小教学骨架
 - $\omega_p \Delta t$
 - $v_t \Delta t / \Delta x$
 - finite-grid / aliasing / heating 主线
- Dawson 1983

- electrostatic / full EM 的数值模型边界
- full EM 时间步与 light mode / CFL 的关系
- Darwin 作为 radiation-free low-frequency route

下列来源只作为继续阅读线索，不在本章中承担未核实的公式或数值结论：

- Yee 1966
- Hockney-Eastwood

继续扩展基础文献时，优先选择能够给出完整推导、适用假设和数值实验条件的原始教材或论文。书目线索本身只能帮助定位来源，不能代替公式、算法或案例结论的核查。

2.10 进一步阅读与练习

进一步阅读：

1. 第 3 章：WarpX 演化：把本章的 PIC loop 抽象结构接到 `main.cpp` $\rightarrow$ `WarpX::Evolve()` 的真实调用链。
2. Birdsall 1985：继续核对 $\omega_p \Delta t$ 、 $\lambda_D / \Delta x$ 、finite-grid aliasing 和 numerical heating 的条件与量纲。
3. Dawson 1983：继续比较 full EM、Darwin、quiet start 和 statistical measurements 如何改变“PIC 总循环”的解释方式。

练习题：

1. 解释为什么 `ComputeDt()` 给出的可运行时间步，不自动保证 $\omega_p \Delta t \ll 1$ 。
2. 用本章的 λ_D 讨论说明：为什么 $\Delta x \gg \lambda_D$ 时，即使主循环稳定，也可能已经不是同一个物理 plasma。
3. 结合本章的 Langmuir 案例，指出 `analysis_1d.py` 的两条核心断言分别对应本章哪两类理论边界。
4. 案例闭环题：依次读取 `inputs_test_1d_langmuir_multi`、`langmuir/CMakeLists.txt` 和 `analysis_1d.py`，交付一张四行表（输入设置、诊断 surface、分析断言、不可外推范围）。表中必须写清 `test_1d_langmuir_multi` 的两个 rank、`analysis_1d.py` `diags/diag1000080` 的绑定，以及为什么不能只从 `max_step = 80` 猜测分析器实际消费的输出路径和检查量。

2.11 本章结论

PIC 总循环的关键不在于依次调用“推粒子、沉积、推场”三个动作，而在于每个动作所使用的时间层和空间表示相容。读者应按以下顺序判断一条 PIC 路线：

1. 先确定连续问题与可分辨尺度。等离子体频率、Debye 长度、光波 CFL 和预期物理时间窗决定 `dt`、网格和粒子采样是否有机会描述目标问题。

2. 再确定粒子与网格的交换方式。gather、形函数和 ρ/J deposition 必须共同满足连续性和离散布局要求；仅有稳定的场更新不能补偿不相容的源项。
3. 区分外层时间步和内部重复。implicit nonlinear iteration、JRhom 的 source 时间积分与 AMR subcycling 都可能在一个外层步内多次执行，但它们分别代表试探、积分和真实细层推进，不能混用同一类验证量解释。
4. 最后用与问题匹配的 **observable** 验证。Langmuir 的解析场与 $\text{div}E - \rho/\epsilon_0$ 能检验指定输入下的波动和 Gauss-law 链；程序退出或 checksum 本身不能证明所有几何、边界和算法组合正确。

这条顺序把连续 Vlasov–Maxwell 模型连接到真实的离散时间推进，也为第 3 章的生命周期调用图、第 4、5 章的粒子/沉积链和第 6 章的场求解器选择提供共同的时间层坐标。

第三章 WarpX 主演化路径：生命周期、初始化与 Evolve()

本章开始进入 WarpX 源码。目标不是概括“WarpX 有一个 Evolve 函数”，而是建立一个可复查的调用图：程序从 `main.cpp` 进入，如何构造 WarpX 对象，如何读取参数和初始化数据，如何计算步长，最后如何在 `WarpXEvolve.cpp` 中把一个个 PIC step 推进下去。

本章的任务是让读者能从一个输入出发，追踪它何时变成网格、场、粒子和诊断，再进入一个物理时间步。使用任何 WarpX 源树时，都应优先按 `main.cpp`、`InitData()`、`ComputeDt()`、`Evolve()` 和 `OneStep()` 的职责与调用关系检索，而不要把固定行号或某个分支名称当作算法语义。

`main.cpp` 负责生命周期，WarpX 类建立模拟状态，`WarpXEvolve.cpp` 组织时间推进，`WarpXInitData.cpp` 则准备首个时间步之前的状态。`OneStep_sub1()`、PSATD-JRhom 和 implicit solver 的入口会在本章中定位；场算法的离散公式、粒子的 nonlinear solve 参数和 mass-matrix kernel 分别在后续相关章节展开，避免在调用图中打断物理主线。

3.0.1 源码定位约定

WarpX 的实现会继续演进，固定行号只能说明某一次快照的位置，不能说明算法语义。本章因此把文件路径和函数/类符号作为可迁移的定位锚点：例如，读者在 `Source/Evolve/WarpXEvolve.cpp` 中搜索 `WarpX::OneStep_nosub`，而不是依赖某个行号。引用小段源码时，重点是观察其输入、输出和先后关系；阅读自己的 WarpX 版本时，应先重新搜索同名符号，再比较实现是否改变。

3.1 顶层入口：main.cpp

WarpX 的可执行入口在 `Source/main.cpp`。主函数的控制流非常短：

```
initialize_external_libraries(argc, argv)
warpx = WarpX::GetInstance()
warpx.InitData()
warpx.Evolve()
is_warpx_verbose = warpx.Verbose()
WarpX::Finalize()
print timer if verbose
finalize_external_libraries()
```

核心调用的职责如下：

源码锚点	代码含义
Source/main.cpp: initialize_external_libraries	初始化 AMReX/MPI/GPU runtime 等 WarpX 依赖的底层运行环境。
Source/main.cpp: WarpX::GetInstance()	取得全局 WarpX 单例。
Source/main.cpp: warpx.InitData()	把参数、网格、场、粒子、诊断和边界准备好。
Source/main.cpp: warpx.Evolve()	进入时间推进主循环。
Source/main.cpp: WarpX::Finalize()	释放 WarpX 单例。
Source/main.cpp: finalize_external_libraries	结束外部库。

主函数中的核心段落如下：

```

warpx::initialization::initialize_external_libraries(argc, argv);

auto timer = ablastr::utils::timer::Timer{};
timer.record_start_time();

auto& warpx = WarpX::GetInstance();
warpx.InitData();
warpx.Evolve();
const auto is_warpx_verbose = warpx.Verbose();
WarpX::Finalize();

timer.record_stop_time();
if (is_warpx_verbose) {
    amrex::Print() << "Total Time          : "
                    << timer.get_global_duration() << '\n';
}

warpx::initialization::finalize_external_libraries();

```

逐块看，这段代码只有一个物理模拟对象 `warpx`。`InitData()` 之前还没有进入时间推进；`Evolve()` 返回之后模拟已经结束，后续只做计时、profiling 和库资源释放。这里没有任何粒子或场循环，因为 WarpX 把模拟状态封装在 WarpX 单例内部。

这个入口说明：WarpX 的主程序不直接管理粒子数组或场数组；它只管理生命周期。真正的模拟状态集中在 WarpX 对象及其持有的 `MultiParticleContainer`、field register、diagnostics、solver、PML 等成员中。

3.2 WarpX 类与单例构造

WarpX 类定义在 `Source/WarpX.H`。关键声明包括：

源码锚点	声明	作用
Source/WarpX.H	<code>class WarpX : public amrex::AmrCore</code>	WarpX 以 AMReX 的 AMR 基类为基础。

源码锚点	声明	作用
WarpX::GetInstance()	static WarpX& GetInstance();	全局单例入口。
WarpX::Finalize()	static void Finalize();	删除单例。
WarpX::InitData() WarpX::Evolve()	void InitData(); void Evolve(int numsteps=-1);	初始化模拟数据。 外层时间推进。
WarpX::OneStep*()	OneStep、OneStep_no_sub、OneStep_sub1、OneStep_JRhom	主循环内部的单步推进路径。

单例实现位于 Source/WarpX.cpp:

- WarpX::GetInstance() 检查 m_instance，为空则调用 MakeWarpX()。
- WarpX::Finalize() 调 ResetInstance() 删除对象。
- WarpX::WarpX() 设置 m_instance=this，初始化 warning manager，调用 ReadParameters()，做向后兼容处理，初始化 EB，建立 istep/nsubsteps/t_new/t_old/dt 数组，并创建 MultiParticleContainer。

构造函数中最重要的调用是 ReadParameters()。这意味着 solver 类型、边界、步长策略、滤波、静电/电磁模式等会在 InitData() 前决定。

3.3 ReadParameters(): 主循环分支的来源

WarpX::ReadParameters() 定义在 Source/WarpX.cpp。完整参数系统很大，本章只列出直接影响主循环的部分。

参数或逻辑	读取位置	对主循环的影响
max_step、stop_time	WarpX::ReadParameters()	Evolve() 用它们限制循环终点。
algo.maxwell_solver	WarpX::ReadParameters()	选择 PSATD、Yee、CKC、ECT、HybridPIC、None 等路径。
PSATD 不支持 PEC/PMC 的断言	WarpX::ReadParameters()	solver 选择会反过来限制边界条件。
algo.evolve_scheme	WarpX::ReadParameters()	决定 explicit、theta implicit 等演化框架。
warpx.cfl、verbose、regrid_int、do_subcycling	WarpX::ReadParameters()	控制步长、输出、重网格和 AMR 子循环。
warpx.do_electrostatic	WarpX::ReadParameters()	静电 solver 非空时把 electromagnetic solver 设为 None。
const_dt、max_dt、dt_update_interval	WarpX::ReadParameters()	控制固定步长和运行中步长更新。

参数或逻辑	读取位置	对主循环的影响
filter 默认开关	<code>WarpX::ReadParameters()</code>	显式 scheme 默认滤波，隐式 scheme 默认关闭滤波。

这里有一个阅读原则：输入文件里的参数名只是表层。要理解参数的真实含义，必须追到 `ReadParameters()` 中它如何被读入、被断言约束、被改写，并进一步影响 `Evolve()`、`ComputeDt()` 或 solver 对象。

3.4 构造函数只建“跨 level 外壳”，不直接建完整网格数据

仅从 `ReadParameters()` 还看不出一个常见实现边界：`WarpX::WarpX()` 构造函数里虽然已经决定了 solver 路线，但此时还没有有效的 `BoxArray` 和 `DistributionMapping`。构造函数中的注释明确说明：

```
// Geometry on all levels has been defined already.
// No valid BoxArray and DistributionMapping have been defined.
// But the arrays for them have been resized.
```

所以构造函数能做的是：

- 创建不依赖具体网格盒划分的跨 level 对象：
 - `MultiParticleContainer`
 - `m_electrostatic_solver`
 - `m_hybrid_pic_model`
 - solver 指针数组外壳
- 按 `maxLevel()+1` 先 `resize` 各种 level 容器：
 - `istep`
 - `nsubsteps`
 - `t_old/t_new`
 - `dt`

但它不能在这里直接分配真正的 `Efield_fp/Bfield_fp/current_fp/rho_fp`。这些字段必须等到 `MakeNewLevelFromScratch()` → `AllocLevelData()` → `AllocLevelMFs()`，拿到真实的 `ba/dm`、`index type` 和 `guard cell` 之后才创建。

这也是为什么：

- `effective potential` 在构造期只创建 `EffectivePotentialES` 对象；
- `hybrid PIC` 在构造期只创建 `HybridPICModel` 对象；
- `implicit solver` 即使已经存在，也还没分配 `mass matrices`。

真正的 level 级附加字段要到后面才各自落地。

3.5 AllocLevelData() / AllocLevelMFs() 里，四条 solver 分支的落点不同

AllocLevelData() 定义在 Source/WarpX.cpp。它先调用 guard_cells.Init(...), 再进入 AllocLevelMFs(...)。这一层真正决定的是：哪些 MultiFab 需要随着 level 一起出生。

先看隐式分支。AllocLevelMFs() 并不会一次性把全部隐式字段都分好，它只先放两类最基础的 level 数据：

```
if (m_implicit_solver) {
    m_fields.alloc_init(FieldType::current_fp_non_suborbit,
        ↪ Direction{0}, lev,
        amrex::convert(ba, jx_nodal_flag), dm, ncomps,
        ↪ ngJ, 0.0_rt);
    ...
    m_fields.alloc_init(FieldType::E_old, Direction{2}, lev,
        amrex::convert(ba, Ez_nodal_flag), dm, ncomps,
        ↪ ngEB, 0.0_rt);
}
```

也就是说，隐式路线在 level 分配期先保存：

- 非 suborbit 粒子的独立电流容器 current_fp_non_suborbit
- 旧电场时间层 E_old

而更重的 MassMatrices_X/Y/Z、MassMatrices_PC 不是在这里分配，而是在后续 ImplicitSolver::InitializeMassMatrices() 里，等标准 current_fp/Efield_fp 的 index type、guard cells 和 deposition 算法都稳定后再决定组件数。

再看 hybrid PIC。它在构造期只有一个模型对象，真正的 level 字段由 HybridPICModel::AllocateLevelMFs(...) 填入共享的 m_fields register：

```
if (WarpX::electromagnetic_solver_id ==
    ↪ ElectromagneticSolverAlgo::HybridPIC)
{
    m_hybrid_pic_model->AllocateLevelMFs(
        m_fields,
        lev, ba, dm, ncomps, ngJ, ngRho, ngEB, jx_nodal_flag,
        ↪ jy_nodal_flag,
        jz_nodal_flag, rho_nodal_flag, Ex_nodal_flag, Ey_nodal_flag,
        ↪ Ez_nodal_flag,
        Bx_nodal_flag, By_nodal_flag, Bz_nodal_flag
    );
}
```

这一调用会分配：

- hybrid_electron_pressure_fp
- hybrid_rho_fp_temp
- hybrid_current_fp_temp

- hybrid_current_fp_plasma
- 可选的 hybrid_current_fp_external

若启用 `add_external_fields`，还会进一步分配 `external vector potential` 相关字段。也就是说，`hybrid` 不是在 `WarpX` 根层自己持有一套独立网格，而是把专用状态嵌进统一的 `field register`。

再看 `effective potential electrostatic solver`。这条线在构造期创建了 `EffectivePotentialES` 对象，但 `level` 分配期没有额外的 `effective_potential_*` 专用字段。它复用的是静电共享合同：

```
if( (electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrame) ||
    (electrostatic_solver_id ==
      ↪ ElectrostaticSolverAlgo::LabFrameElectroMagnetostatic) ||
    (electrostatic_solver_id ==
      ↪ ElectrostaticSolverAlgo::LabFrameEffectivePotential) ||
    (electromagnetic_solver_id == ElectromagneticSolverAlgo::HybridPIC)
    ↪ ) {
    rho_ncomps = ncomps;
}

if (electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrame ||
    electrostatic_solver_id ==
      ↪ ElectrostaticSolverAlgo::LabFrameElectroMagnetostatic ||
    electrostatic_solver_id ==
      ↪ ElectrostaticSolverAlgo::LabFrameEffectivePotential )
{
    m_fields.alloc_init(FieldType::phi_fp, lev, amrex::convert(ba,
      ↪ phi_nodal_flag), dm,
                        ncomps, ngPhi, 0.0_rt );
}
```

所以 `effective potential` 在根层最关键的差异不是“多了什么字段”，而是后续 solver 如何解释和消费 `rho_fp/phi_fp`。

最后是 PML。它同样不是在 `AllocLevelMFs()` 里出生。真正创建 PML 对象是在 `InitFromScratch()` 的最后一步：

```
AmrCore::InitFromScratch(time); // This will call
  ↪ MakeNewLevelFromScratch
...
mypc->AllocData();
mypc->InitData();

InitPML();
```

这条顺序说明：

- 先有普通 `level` 的 `fields / particles`
- 后有作为边界子系统的 PML

因此 PML 是初始化末段附加上的 `patch` 级边界对象，而不是和 `Efield_fp/Bfield_fp` 同时由 `AllocLevelMFs()` 常规分配的主网格字段。

把对象落点按路径分开读，会比把构造、分配和初始化末段塞进同一张宽表更清楚：

- **标准场**: 构造期只准备容器外壳; `AllocLevelMFs()` 分配 `Efield_fp`、`Bfield_fp`、`current_fp`、`rho_fp` 和 `phi_fp`; `InitLevelData()` 再填入物理初值。
- **effective potential**: 构造期创建 `EffectivePotentialES`; level 分配期不另建专用字段, 而是复用 `rho_fp/phi_fp`; 静电求解器随后按 `effective-potential` 语义消费这些字段。
- **hybrid PIC**: 构造期创建 `HybridPICModel`; level 分配期把 `hybrid_*` 字段放入共享 `m_fields`; `HybridPICModel::InitData()` 再编译 parser 并准备外加电流和矢势。
- **implicit**: solver 对象在构造期已存在; level 分配期先分配 `current_fp_non_suborbit` 与 `E_old`; 随后 `Define()`、`CreateParticleAttributes()` 和 `InitializeMassMatrices()` 逐步补齐隐式响应数据。
- **PML**: 构造期只知道参数和开关, `AllocLevelMFs()` 也不创建 PML patch; `InitPML()` 在获得真实 `boxArray`、`DistributionMapping`、`dt` 和 `m_fields` 后再创建边界对象。

3.6 InitData(): 把状态准备到可推进

`WarpX::InitData()` 定义在 `Source/Initialization/WarpXInitData.cpp`。它不是简单分配内存, 而是把一个模拟从“参数已读”变成“可以推进第一步”。

核心顺序如下:

源码动作	解释
进入 <code>InitData()</code> , 检查 MPI thread level 创建 <code>MultiDiagnostics</code> 和 <code>MultiReducedDiags</code>	并行运行前的运行环境检查。 诊断系统在初始化早期建立。
非 restart: <code>ComputeDt()</code> 、打印步长网格、 <code>InitFromScratch()</code> 、 <code>InitDiagnostics()</code>	从头运行时先确定步长, 再建立网格/粒子/诊断。
restart: <code>InitFromCheckpoint()</code> 、打印步长网格、 <code>PostRestart()</code> <code>ComputeMaxStep()</code> 、PML factors、NCI corrector、buffer masks	checkpoint 恢复不走完全相同的初始化路径。 准备停止步数、吸收边界和数值不稳定修正。
宏观介质、静电 solver、HybridPIC 初始化 网格摘要、guard cell 检查、打印 PIC 参数、 写 used inputs	solver 相关对象拿到场布局 and 参数。 把运行状态和输入记录下来。
初始 div cleaning、自洽静电/磁静场、外场叠加	从头运行时在第一个 step 前建立初始场。
初始 full/reduced diagnostics 性能提示和 solver issue 检查	允许输出第 0 步或 restart 后诊断。 给出已知风险提示。

`InitFromScratch()` 调用 `AmrCore::InitFromScratch(time)` 建立 AMR level, 然后让 `mypc->AllocData()` 和 `mypc->InitData()` 初始化粒子, 最后初始化 PML。

3.7 ComputeDt(): 步长不是一个固定常数

WarpX::ComputeDt() 定义在 Source/Evolve/WarpXComputeDt.cpp。

逻辑可以分成四类：

1. HybridPIC 必须显式给出 warpx.const_dt。
2. 纯静电或无 Maxwell solver 时，必须给出 const_dt 或激活 dt_update_interval。
3. 若用户给了 const_dt，直接使用。
4. 否则按 solver 计算 CFL 限制：静电/PSATD 用最小 cell size 与 c ，FDTD 调用具体几何和算法的 ComputeMaxDt()。

最终 dt 被 resize 到 max_level+1。若启用 subcycling，粗层步长由细层步长乘 refinement ratio 得到。

把它写成决策顺序会比宽表更容易在阅读和排版中保持清楚：

1. 用户固定步长优先。设置 warpx.const_dt 后，ComputeDt() 直接采用它，所有 CFL 估计都不再决定步长；稳定性责任也随之转给用户。
2. 特殊模型先检查强制条件。HybridPIC 必须给出 const_dt。静电路径若未使用固定步长，则以 max_dt 为初值；若也未给出，则退回到 CFL $\Delta x_{\min}/c$ 。
3. PSATD 使用最小网格尺度的光速尺度。在没有固定步长时，初值为 CFL $\Delta x_{\min}/c$ 。这只是本函数的初始选择，不能替代该配置的物理分辨率判断。
4. FDTD 按几何和网格布置选择稳定上限。Cartesian collocated grid 调用 CartesianNodalAlgorithm::ComputeMaxDt；Yee/ECT 调用 CartesianYeeAlgorithm::ComputeMaxDt；CKC 调用 CartesianCKCAlgorithm::ComputeMaxDt。RZ/RCYLINDER 的 Yee 路径还使用 azimuthal mode 数，RSPHERE 则走 SphericallyYeeAlgorithm::ComputeMaxDt。这些函数名比展开成长公式更适合作为源码阅读入口。

因此，ComputeDt() 不只是“取最小网格长度除以光速”，而是在参数层先决定有没有用户强制时间步，再按 solver 家族和几何去选真正的稳定上限公式。

运行中自适应步长由 WarpX::ApplyDtLimiters() 处理。它首先经 ParticleGrid SpeedMax() 从 mypc->maxParticleVelocity() 计算最大粒子跨网格速度，再与可选的等离子体频率、回旋频率和 max_dt 限制一起取最严格的步长。仅考虑粒子跨网格限制时，公式是

$$\Delta t_{\text{new}} = \text{CFL} \frac{\Delta x_{\min}}{v_{\max}}$$

ApplyDtLimiters() 更新 finest level 的 dt，再向粗层回推。因此它并不等同于一个只看粒子速度的旧接口：开启 max_omegap_dt、max_omegac_dt 或 max_dt 时，任何一个限制都可以成为最终瓶颈。

这里还要补一条参数层边界：warpx.const_dt 与 warpx.dt_update_interval 在 ReadParameters() 中就是互斥的。也就是说，运行时 adaptive timestep 不是“在固定步长上再做微调”，而是一条和 const_dt 完全不同的时间组织路线。Evolve() 里只有当 m_dt_update_interval.contains(step+1) 为真，或首步需要应用 max_dt 时，才会在步首调用 ApplyDtLimiters()。

3.8 Evolve() 外层时间步

WarpX::Evolve() 定义在 Source/Evolve/WarpXEvolve.cpp。它不是单纯调用 OneStep(), 而是在每个 step 前后管理大量状态。

外层结构是:

```
cur_time = t_new[0]
numsteps_max = max_step or istep[0] + numsteps
for step in range while cur_time < stop_time:
    check signals
    multi_diags->NewIteration()
    run beforestep callback
    CheckLoadBalance(step)
    maybe update dt from particle speeds
    ExplicitFillBoundaryEBUpdateAux()
    hybrid initialization if first step
    field ionization / QED / particle injection
    OneStep(cur_time, dt[0], step)
    resampling / mirrors
    increment istep and time
    diagnostics prepack, moving window, particle boundary handling
    electrostatic or hybrid field solve if selected
    optional velocity synchronization for diagnostics
    afterstep callback, diagnostics, warnings, signals, stop check
```

关键动作:

源码动作	读法
初始化 cur_time 和循环边界	numsteps=-1 时使用全局 max_step。
信号检查、诊断新迭代	支持运行中断、checkpoint 和诊断状态更新。
callback、负载均衡、可选步长更新	更新步长前需要同步粒子速度时间层。
ExplicitFillBoundaryEBUpdateAux()	显式 scheme 为后续 field gather 准备场。
field ionization、QED、particle injection	多物理事件在 OneStep() 前改变粒子集合。
OneStep(cur_time, dt[0], step)	进入单步推进分派。
更新 istep 和 t_old/t_new	单步推进后更新时间状态。
诊断预处理、moving window、粒子边界	OneStep() 后的工程处理同样影响物理结果。
静电或 HybridPIC 的场解	非标准电磁路径的场更新位置不同。
诊断需要时同步粒子速度	为输出把 p 与 x 放到同步时间层。
reduced/full diagnostics 和 callback	诊断写出发生在本步状态更新之后。
未使用输入检查、计时、信号、停止	第一步后检查输入 typo, 最后判断是否退出。

注意 Evolve() 中多物理和诊断并不都在 OneStep() 内部。比如 field ionization、QED 和 particle injection 在 OneStep() 之前, resampling、moving window、粒子边界和某些 electrostatic/hybrid 场解在 OneStep() 之后。

3.8.1 步末 moving window: 连续坐标与整数网格平移

要理解 moving window, 先把它放回 Evolve() 主循环中: MoveWindow(step+1, move_j) 发生在 OneStep() 完成、cur_time 和 t_new 更新之后, 粒子边界处理之前。

对应逻辑位于 WarpX::Evolve():

```
cur_time += dt[0];

ShiftGalileanBoundary();

// sync up time
for (int i = 0; i <= max_level; ++i) {
    t_old[i] = t_new[i];
    t_new[i] = cur_time;
}
multi_diags->FilterComputePackFlush( step, false, true );

const bool move_j = m_is_synchronized;
// If m_is_synchronized we need to shift j too so that next step we can
    ↪ evolve E by dt/2.
// We might need to move j because we are going to make a plotfile.
const int num_moved = MoveWindow(step+1, move_j);
```

MoveWindow() 的第一层逻辑是维护一个连续窗口位置 moving_window_x, 但只有当它相对当前几何左边界跨过整数个 cell 时才真正平移网格数据。实现位于 Source/Utils/WarpXMovingWindow.cpp:

```
if (!moving_window_active(step)) { return 0; }

// Update the continuous position of the moving window,
// and of the plasma injection
moving_window_x += (moving_window_v - WarpX::beta_boost * PhysConst::c)
    / (1 - moving_window_v * WarpX::beta_boost /
    ↪ PhysConst::c) * dt[0];
const int dir = moving_window_dir;

// Update current injection position for all containers
::UpdateInjectionPosition(*myipc, gamma_boost, beta_boost,
    ↪ boost_direction, moving_window_dir, dt[0]);

// Update antenna position for all lasers
// TODO Make this specific to lasers only
myipc->UpdateAntennaPosition(dt[0]);

// compute the number of cells to shift on the base level
amrex::Real new_lo[AMREX_SPACEDIM];
amrex::Real new_hi[AMREX_SPACEDIM];
const amrex::Real* current_lo = geom[0].ProbLo();
const amrex::Real* current_hi = geom[0].ProbHi();
const amrex::Real* cdx = geom[0].CellSize();
const int num_shift_base = static_cast<int>((moving_window_x -
    ↪ current_lo[dir]) / cdx[dir]);

if (num_shift_base == 0) { return 0; }
```

这段代码中的速度变换是相对论速度合成公式:

$$v'_w = \frac{v_w - \beta_b c}{1 - v_w \beta_b / c}.$$

因此 boosted-frame 模拟中窗口速度不是简单使用输入的 `moving_window_v`。输入参数在 `read_moving_window_parameters()` 中先由以 c 为单位的无量纲数转成 SI 速度；运行时再按 boost 速度变换到模拟坐标系。

`active` 判定本身也不是模糊的“某个阶段窗口有效”，而是源码里一个明确的闭开区间：

$$\text{start_moving_window_step} \leq n < \text{end_moving_window_step},$$

若 `end_moving_window_step < 0` 则表示没有终止步。这也是为什么 `Evolve()` 调的是 `MoveWindow(step+1, ...)`：窗口平移是本步结束后、下一步开始前的状态更新。

当 `num_shift_base != 0` 时，`MoveWindow()` 调用 `ResetProbDomain()` 更新几何域，随后由 `shiftMF()` 平移场、source、PML、 F/G 、 ρ 与 fluid 的 `MultiFab`。核心赋值为：

```
amrex::Box dstBox = mf[mfi].box();
if (num_shift > 0) {
    dstBox.growHi(dir, -num_shift);
} else {
    dstBox.growLo(dir, num_shift);
}
AMREX_PARALLEL_FOR_4D ( dstBox, nc, i, j, k, n,
{
    dstfab(i,j,k,n) = srcfab(i+shift.x,j+shift.y,k+shift.z,n);
})
```

即

$$F_{\text{new}}(\mathbf{i}) = F_{\text{old}}(\mathbf{i} + N_{\text{shift}} \hat{e}_{\text{dir}}).$$

新露出的边界层不是随便置零。对外部场，`shiftMF()` 可以用常量外场或 `parser` 外场重新初始化；对背景粒子，`MoveWindow()` 构造整数 cell 宽度的 `particleBox` 并调用 `pc.ContinuousInjection(particleBox)`。这解释了 `moving window` 的数值设计：连续窗口位置负责物理速度，整数 cell 平移负责保持网格离散结构和宏粒子 `spacing`。

这里还要和步末的 `ContinuousFluxInjection(cur_time, dt[0])` 分开。二者不是同一件事：

- `moving-window continuous injection`：在新露出的体网格区域里补背景体分布；
- `continuous flux injection`：从定义好的注入面持续打入粒子通量。

前者是“窗口移动后补齐新计算域”，后者是“边界源项继续往域内送粒子”。

3.9 OneStep(): 求解器分派器

`WarpX::OneStep()` 定义在 `Source/Evolve/WarpXEvolve.cpp`。它按 `solver` 和 `AMR` 情况分派：

```
if m_implicit_solver:
    m_implicit_solver->OneStep(...)
else:
    if electromagnetic_solver_id is None or HybridPIC:
```



```

        push particles, optionally split collisions, skip deposition
    else:
        if finest_level == 0:
            OneStep_nosub or OneStep_JRhom
        else:
            OneStep_nosub or OneStep_sub1

```

这段代码体现了 WarpX 的设计：OneStep() 不直接写某一种 PIC 算法的全部细节，而是先把路径分开。

读者的生命周期检查卡。从一个输入追到可比较的输出时，不要把函数名称当成时间层。先确认输入参数如何决定求解器、几何和 AMR 分支；再确认 InitData() 已创建可被第一步消费的离散状态；随后把 Evolve() 视为提交一个外层区间 $t^n \rightarrow t^{n+1}$ 的边界；最后才由 OneStep() 判断这个区间实际采用哪一条时间合同。每次读到一次循环、一次沉积或一次 RHS 评估，都要回问：它是在推进物理时间，还是在为同一外层步重建 source? 输出文件出现后，还必须接回独立 reference 和 observable，不能把控制流命中当作物理验证。

可将这条检查卡压成五个连续问题：

1. 输入参数先决定求解器、几何和 AMR 分支；
2. 初始化负责创建可被第一步消费的离散状态；
3. 外层步只定义 $t^n \rightarrow t^{n+1}$ 的提交边界；
4. 单步分派才决定实际进入哪条时间合同；
5. 可观察输出必须与独立 reference 一起判断。

分支	源码锚点	含义
implicit solver	m_implicit_solver->OneStep(...)	交给隐式 solver 自己推进一整步。
electrostatic / HybridPIC	electromagnetic_solver == None/HybridPIC	粒子推进但跳过标准电磁沉积路径，场解在外层后处理。
标准电磁无 MR	finest_level == 0	进入 OneStep_nosub() 或 PSATD-JRhom。
有 MR 无 subcycling	!m_do_subcycling	仍进入 OneStep_nosub()，所有 level 使用同一步长推进。
有 MR 且 subcycling	m_do_subcycling	进入 OneStep_sub1()；该函数拒绝超过两个 level，且要求 2:1 refinement ratio。

几个断言值得后续单独讲：

- JRhom 与 split momentum collision 不能组合。
- subcycling 要求 finest_level == 1。
- subcycling 与 split momentum collision 也不能组合。

这些不是文档层面的“建议”，而是源码级功能边界。

还应补一条输入层边界：psatd.JRhom 不是布尔开关，而是一个编码了源项时间模型的字符串。ReadParameters() 里它会同时决定：

- J 的时间依赖是 constant / linear / quadratic；
- rho 的时间依赖是 constant / linear / quadratic；
- 一个 PIC step 里切成多少个 JRhom subinterval。

因此 JRhom 开启后，后续变的不是“另一个小优化开关”，而是 OneStep() 内部的时间组织、rho_fp 的组件语义和谱空间源项更新公式。

3.10 OneStep_nosub(): 显式电磁标准路径

WarpX::OneStep_nosub() 定义在 Source/Evolve/WarpXEvolve.cpp。这是本书第一个需要逐行读懂的核心函数。

它的结构分为四段。

第一段：粒子推进、碰撞与沉积。

源码原文：

```
// Push particle from x^{n} to x^{n+1}
//                               from p^{n-1/2} to p^{n+1/2}
// Deposit current j^{n+1/2}
// Deposit charge density rho^{n}

ExecutePythonCallback("beforedeposition");

// with collisions placed in the middle of the momentum push
if (m_collisions_split_momentum_push) {
    // push particles (half momentum)
    PushParticlesandDeposit(
        a_cur_time,
        /*skip_deposition=*/true,
        PositionPushType::None,
        MomentumPushType::FirstHalf
    );
    // perform particle collisions
    ExecutePythonCallback("beforecollisions");
    mypc->doCollisions(a_step, a_cur_time, a_dt);
    ExecutePythonCallback("aftercollisions");

    // push particles (full position and half momentum)
    PushParticlesandDeposit(
        a_cur_time,
        /*skip_deposition=*/false,
        PositionPushType::Full,
        MomentumPushType::SecondHalf
    );
}
else {
    ExecutePythonCallback("beforecollisions");
    mypc->doCollisions(a_step, a_cur_time, a_dt);
    ExecutePythonCallback("aftercollisions");
}
```

```

    PushParticlesandDeposit(
        a_cur_time,
        /*skip_deposition=*/false,
        PositionPushType::Full,
        MomentumPushType::Full
    );
}

ExecutePythonCallback("afterdeposition");

```

- 注释明确时间层: $\mathbf{x}^n \rightarrow \mathbf{x}^{n+1}$, $\mathbf{p}^{n-1/2} \rightarrow \mathbf{p}^{n+1/2}$, 沉积 $\mathbf{J}^{n+1/2}$ 和 ρ^n 。
- 如果 `m_collisions_split_momentum_push` 为真, 先做半个动量 push, 再碰撞, 再做位置 push 和后半动量 push。
- 否则先做碰撞, 再调用 `PushParticlesandDeposit()` 完整推进粒子并沉积。

第二段: 源项同步。

源码原文:

```

// Synchronize J and rho:
// filter (if used), exchange guard cells, interpolate across MR levels
// and apply boundary conditions
SyncCurrentAndRho();

// At this point, J is up-to-date inside the domain, and E and B are
// up-to-date including enough guard cells for first step of the field
// solve.

// For extended PML: copy J from regular grid to PML, and damp J in PML
if (do_pml && pml_has_particles) { CopyJPML(); }
if (do_pml && do_pml_j_damping) { DampJPML(); }

```

- `SyncCurrentAndRho()` 会处理滤波、guard cells、AMR 跨层插值/加和和边界。
- PML 若含粒子或需要电流阻尼, 会复制和阻尼 PML 中的电流。

第三段: PSATD 或 FDTD 场推进。

FDTD 分支的核心源码如下:

```

} else {
    EvolveF(0.5_rt * dt[0], /*rho_comp=*/0);
    EvolveG(0.5_rt * dt[0]);
    FillBoundaryF(guard_cells.ng_FieldSolverF);
    FillBoundaryG(guard_cells.ng_FieldSolverG);

    EvolveB(0.5_rt * dt[0], SubcyclingHalf::FirstHalf, a_cur_time); //
    ↪ We now have B^{n+1/2}
    FillBoundaryB(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

    if (m_em_solver_medium == MediumForEM::Vacuum) {
        EvolveE(dt[0], a_cur_time); // We now have E^{n+1}
    } else if (m_em_solver_medium == MediumForEM::Macroscopic) {
        MacroscopicEvolveE(dt[0], a_cur_time); // We now have E^{n+1}
    } else {

```

```

        WARPX_ABORT_WITH_MESSAGE("Medium for EM is unknown");
    }
    FillBoundaryE(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

    EvolveF(0.5_rt * dt[0], /*rho_comp=*/1);
    EvolveG(0.5_rt * dt[0]);
    EvolveB(0.5_rt * dt[0], SubcyclingHalf::SecondHalf,
            a_cur_time + 0.5_rt * dt[0]); // We now have B^{n+1}

```

- PSATD 走 PushPSATD(a_cur_time), 并处理 hybrid QED、PML、平均场、 F/G guard cells。
- FDTD 走 EvolveF/G 半步、EvolveB(dt/2)、EvolveE(dt)、EvolveF/G 半步、EvolveB(dt/2)。

第四段：回调。

- afterEvolve callback 在场更新后执行。

从物理角度看，OneStep_nosub() 做的事情是：用旧场 gather 推粒子，得到新位置和半步电流；把源项修整到网格和边界一致；再用这些源项推进电磁场。

3.11 SyncCurrentAndRho(): 源项不是沉积完就可用

SyncCurrentAndRho() 定义在 Source/Evolve/WarpXEvolve.cpp。

它的分支很重要：

- PSATD 且 periodic single box 时，会立即同步 J 和 ρ 。
- PSATD 非 periodic single box 时，若没有 current correction 且不是 Vay deposition，才在这里同步。
- Vay deposition 在特定情况下先只做 filter。
- FDTD 路径总是 SyncCurrent("current_fp") 和 SyncRho()。
- 最后对 ρ 和 J 施加 PEC 等边界处理。

这说明“沉积”与“可用于场解”之间有一段不可忽略的工程层：滤波、guard cell、AMR 和边界会改变源项数组的可用状态。

3.12 PushParticlesandDeposit(): 进入粒子容器

PushParticlesandDeposit() 的两个重载定义在 Source/Evolve/WarpXEvolve.cpp。

第一层重载遍历所有 AMR level。第二层重载做三件事：

1. 根据 do_current_centering 和 current_deposition_algo == Vay 选择当前沉积字段名。

2. 调用 `mypc->Evolve(...)`, 把 field register、level、字段名、时间、`dt[lev]`、subcycling half、是否跳过沉积、位置/动量 push 类型传入粒子容器。
3. 对 RZ/柱/球几何做逆体积缩放, 并在有流体物种时调用流体容器演化。

因此, 下一阶段逐行阅读必须从 `mypc->Evolve()` 继续进入 `Source/Particles.PushParticlesandDeposit()` 是主循环到粒子模块的接口, 不是粒子 pusher 本身。

3.13 OneStep_sub1() 与 JRhom 的位置

理解这两个特殊分支时, 先把它们放回主循环时间层。

`OneStep_sub1()` 定义在 `Source/Evolve/WarpXEvolve.cpp`。这条 subcycling 路径要求两个 level 和 2:1 refinement ratio: fine patch 用小步长推两次, coarse patch 和 mother grid 推一次, coarse 场使用两次 fine current 的平均效果。入口断言与函数注释共同限定了这个适用范围:

```
* This version of subcycling only works for 2 levels and with a
  ↳ refinement
* ratio of 2.
* The particles and fields of the fine patch are pushed twice
* (with dt[coarse]/2) in this routine.
* The particles of the coarse patch and mother grid are pushed only once
* (with dt[coarse]). The fields on the coarse patch and mother grid
* are pushed in a way which is equivalent to pushing once only, with
* a current which is the average of the coarse + fine current at the 2
* steps of the fine grid.
```

第一段 fine step 的核心源码如下:

```
PushParticlesandDeposit(fine_lev, cur_time, SubcyclingHalf::FirstHalf);
RestrictCurrentFromFineToCoarsePatch(
    m_fields.get_mr_levels_alldirs(FieldType::current_fp, finest_level),
    m_fields.get_mr_levels_alldirs(
        FieldType::current_cp, finest_level, skip_lev0_coarse_patch),
    fine_lev);
RestrictRhoFromFineToCoarsePatch(fine_lev);

EvolveB(fine_lev, PatchType::fine, 0.5_rt*dt[fine_lev],
  ↳ SubcyclingHalf::FirstHalf, cur_time);
EvolveF(fine_lev, PatchType::fine, 0.5_rt*dt[fine_lev], /*rho_comp=*/0);
EvolveE(fine_lev, PatchType::fine, dt[fine_lev], cur_time);
EvolveB(fine_lev, PatchType::fine, 0.5_rt*dt[fine_lev],
    SubcyclingHalf::SecondHalf,
    cur_time + 0.5_rt * dt[fine_lev]);
EvolveF(fine_lev, PatchType::fine, 0.5_rt*dt[fine_lev], /*rho_comp=*/1);
```

因此 subcycling 的物理时间层是

$$\Delta t_0 = 2\Delta t_1.$$

fine level 在一个 coarse step 内走两个完整 leapfrog 小步。随后对 current 和 ρ 做 restriction 到 coarse patch; `StoreCurrent()/RestoreCurrent()` 保证 coarse 粒子自身 current 能在两个 half coarse step 中分别叠加对应的 fine contribution。

这里 `StoreCurrent()/RestoreCurrent()` 的角色需要说得更硬一点：subcycling 不是简单把 fine current 直接覆写 coarse current，而是要先保留 coarse 粒子本身在大步时间层上的电流，再把两次 fine-step 的 restriction 结果分别叠回 coarse half-step。否则 coarse mother grid 看到的就不是“一个 coarse 大步上等效的平均源项”，而会把 coarse 自身电流和 fine 补偿混在一起。

`OneStep_JRhom()` 定义在 `Source/Evolve/WarpXEvolve.cpp`。它是 PSATD-JRhom 专用路径，会多次沉积 J 和 ρ ，在谱空间推进字段，并支持时间平均场。入口先断言 solver 必须是 PSATD，并且粒子 push 时跳过标准沉积：

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    WarpX::electromagnetic_solver_id ==
        ↪ ElectromagneticSolverAlgo::PSATD,
    "JRhom algorithm not implemented with the FDTD solver"
);

// Push particle from x^{n} to x^{n+1}
//           from p^{n-1/2} to p^{n+1/2}
const bool skip_deposition = true;
PushParticlesandDeposit(cur_time, skip_deposition);

// Initialize PSATD-JRhom loop:

// 1) Prepare E,B,F,G fields in spectral space
PSATDForwardTransformEB();
if (WarpX::do_dive_cleaning) { PSATDForwardTransformF(); }
if (WarpX::do_divb_cleaning) { PSATDForwardTransformG(); }
```

随后 JRhom 以

$$\delta t = \frac{\Delta t}{m}$$

为子区间，多次在相对时间 `t_deposit_current` 和 `t_deposit_charge` 沉积源项：

```
const int n_deposit = WarpX::m_JRhom_subintervals;
const amrex::Real sub_dt = dt[0] / static_cast<amrex::Real>(n_deposit);
const int n_loop = (WarpX::fft_do_time_averaging) ? 2*n_deposit :
    ↪ n_deposit;

for (int i_deposit = 0; i_deposit < n_loop; i_deposit++)
{
    if (time_dependency_J != TimeDependencyJ::Constant) {
        ↪ PSATDMoveJNewToJOld(); }

    const amrex::Real t_deposit_current = (time_dependency_J ==
        ↪ TimeDependencyJ::Linear) ?
        (i_deposit-n_deposit+1)*sub_dt :
        ↪ (i_deposit-n_deposit+0.5_rt)*sub_dt;

    const amrex::Real t_deposit_charge = (time_dependency_rho ==
        ↪ TimeDependencyRho::Linear) ?
        (i_deposit-n_deposit+1)*sub_dt :
        ↪ (i_deposit-n_deposit+0.5_rt)*sub_dt;

    mypc->DepositCurrent(
        m_fields.get_mr_levels_alldirs(current_string, finest_level),
```

```
dt[0], t_deposit_current);
SyncCurrent("current_fp");
PSATDForwardTransformJ("current_fp", "current_cp");
```

所以 JRhom 的核心不是多次 gather，也不是多次粒子 push，而是用同一次粒子轨道在多个相对时间沉积源项，让 PSATD 在一个 step 内看到更高阶的 $\tilde{\mathbf{J}}(t)$ 和 $\tilde{\rho}(t)$ 。

这条线路还有两条必须写清的功能限制：

1. JRhom 不支持 FDTD，只能走 PSATD。
2. JRhom 和 current_correction、Vay deposition 都不兼容；源码层会把 current_correction 关掉，并显式禁止 Vay deposition 和 JRhom 组合。

所以这一支的真实定位是：它不是“PSATD 上再附加一个任意可叠加的小修正”，而是 PSATD 自身的一种替代性时间积分组织方式。

3.13.1 implicit 分支：一次物理步包含多次试探性 source 重建

在 WarpX::OneStep() 中，只要 m_implicit_solver 非空，程序就不会进入 OneStep_nosub()、OneStep_sub1() 或 OneStep_JRhom()，而是把整步交给 m_implicit_solver->OneStep(...)。

以 SemiImplicitEM::OneStep() 为代表，隐式电磁步的控制流是：

顺序	源码动作	时间层/物理含义
1	SaveParticlesAtImplicitStepStart()	保存 x^n, p^n ，供非线性迭代和最终提交使用
2	初始化 $E^{n+\theta}$ 猜测、保存 E_old	构造 solver 的中间场未知量，而不是直接写最终 E^{n+1}
3	EvolveB($\Delta t/2$)	先把 WarpX 所有的磁场推进到半步
4	m_nlsolver->Solve(..)	反复调用 ComputeRHS()，求粒子和中间电场自洽的离散方程
5	SetElectricFieldAndApplyBCs()、FinishImplicitParticleUpdate()	将收敛的中间场写回，并把粒子从半步状态完成到 t^{n+1}
6	第二个 EvolveB($\Delta t/2$)	完成磁场后半步，物理时间步才真正结束

因此 m_nlsolver->Solve() 不是一个普通的函数调用包装，而是这条路径的核心时间组织。SemiImplicitEM::ComputeRHS() 会先用当前猜测的 $E^{n+1/2}$ 更新 WarpX 持有的电场，然后调用 PreRHSOp()；后者定义在 Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp，完成：

1. 以当前中间场推进粒子位置和速度；
2. 形成 $J^{n+1/2}$ ；

3. 按需要合并 `current_fp_non_suborbit`、`mass-matrix` 或 JFNK 线性阶段贡献；
4. 做 RZ/柱/球几何的逆体积缩放；
5. 调用 `SyncCurrentAndRho()`，把源项整理后交给隐式 RHS 或 Jacobian。

这条链中的 `PushParticlesandDeposit()` 可能在多个 nonlinear iteration、Jacobian evaluation 和 particle Picard iteration 中重复出现，但这些重复并不代表多个物理时间步。它们是在同一个 $t^n \rightarrow t^{n+1}$ 问题上，对不同中间场猜测重新计算离散残差。

`implicit` 还有两个容易误读的实现边界：

- `CumulateJ()` 必须在 `SyncCurrentAndRho()` 之前，把 `mass-matrix` 路径之外的 J 贡献合入 `current_fp`；否则同步的是不完整源项。
- `m_use_mass_matrices_jacobian` 和 `m_particle_suborbits` 会让 Jacobian 阶段只推进 suborbit 粒子或直接用 `ComputeJfromMassMatrices()` 构造电流，因而不能假定每次 RHS 都走同一个粒子 kernel。

这也解释了为什么 `implicit` 的验证不能只复制显式 Langmuir 的“单步场误差”判据。至少要分别检查：非线性求解是否收敛、粒子最终状态是否只提交一次、RHS 期间的 source synchronization 是否完整，以及最终 E/B 时间层是否与 `FinishImplicitParticleUpdate()` 一致。

3.13.2 nonlinear solver、JFNK 与 mass-matrix: J 的三种构造层

`ImplicitSolver::parseNonlinearSolverParams()` 定义在 `Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp`。它首先读取 `nonlinear_solver`，再决定后续 J 是通过完整粒子响应、Newton/JFNK 近似，还是 PETSc SNES 的线性阶段构造：

配置	solver 对象	粒子/电流路径	关键边界
picard	PicardSolver	每次 RHS 直接用当前场推进粒子并沉积 J	<code>parseNonlinearSolverParams()</code> 将 <code>max_particle_iterations=1</code> 、 <code>particle_tolerance=0</code> 设为最小 Picard 路径
newton	NewtonSolver	非线性外层配合 JFNK；可选 <code>particle_suborbits</code> 与 <code>mass-matrix Jacobian</code>	<code>use_mass_matrices_jacobian</code> 和 <code>use_mass_matrices_pc</code> 只能在该类 solver 中启用
petsc_snes	PETScSNES	由 PETSc 管理 nonlinear/linear solve，仍复用 <code>PreRHSOp()</code> 构造源项	必须以 <code>AMREX_USE_PETSC</code> 编译，否则源码直接 abort

在普通 nonlinear RHS 阶段，`PreRHSOp()` 的电流来源可以概括为完整粒子响应：

$$J = J_{\text{particle}} + J_{\text{non-suborbit}}$$

但在使用 mass matrices 的 Jacobian 阶段，源码采用的是线性化响应：

$$J(E_0 + \delta E) = J_{\text{suborbit}} + J_0 + M \delta E,$$

其中 E_0 是 Newton step 开始时由 `SaveE()` 保存的电场， J_0 是以 E_0 推进的非 mass-matrix 粒子电流， M 是 `MassMatrices_X/Y/Z` 表示的离散响应算子。这个式子正是 `CumulateJ()` 和 `ComputeJfromMassMatrices()` 之间的职责分界：

- `CumulateJ()` 把 `current_fp_non_suborbit` 加回当前 `current_fp`，补上不在 mass matrices 中的粒子贡献；
- `ComputeJfromMassMatrices()` 根据当前 $E-E_0$ 、 J_0 和各方向交叉响应，把 $M \delta E$ 写入 `current_fp`；
- `SyncCurrentAndRho()` 只负责之后的滤波、边界、guard/level 通信，不负责判断 J 应该由完整粒子还是线性响应产生。

`ComputeJfromMassMatrices()` 还必须处理 Yee/nodal staggering。源码先根据 $J_x/J_y/J_z$ 的 `ixType()` 计算 `offset_xx ... offset_zz`，再用 `Sxx/Sxy/.../Szz` 的多分量 stencil 访问邻近电场。因此 mass matrix 不是一个可以在任意 centering 上直接相乘的标量系数；它同时编码了方向耦合、空间 support 和网格位置偏移。把它简写成 $M = \partial J / \partial E$ 只足以说明物理意图，不足以替代对 index type 和 component offset 的源码核对。

配置层也有明确的几何限制：参数检查拒绝在 3D 启用 `use_mass_matrices_jacobian`，拒绝在 RSPHERE 使用 mass matrices；`mass_matrices_pc_width` 只在非 3D 情况下读取。因而这条路径不能被描述成所有 implicit geometry 的通用加速开关。

最后，`particle_suborbits` 改变的是粒子响应如何被拆分，而不是外层物理时间步。在线性 Jacobian 阶段，若启用 suborbit，`PreRHSOp()` 可以只推进 suborbit 粒子并用 `ComputeJfromMassMatrices(J_from_MM_only)` 补齐响应；若未启用，则由 mass matrices 直接构造线性阶段的 J 。这正是 implicit 验证必须同时记录 solver 类型、particle suborbit、mass-matrix 开关和最终 source gate 的原因。

3.13.3 AMR subcycling 修改后的验证卡：先分清时间层、source 和回归

AMR subcycling 与碰撞的 `ndt_subcycle`、场求解器的内部循环不是同一个概念。这里讨论的是 `warpx.do_subcycling = 1` 进入 `OneStep_sub1()` 的两级网格时间合同：一个 coarse 物理步内，fine patch 和其粒子走两次小步，coarse patch 与 mother grid 只提交一次大步。官方开发文档也把它列为 `OneStep_nosub()` 之外的 PIC-loop 核心入口。

第一层：先确认这真的是受支持的两级分支。`Examples/Tests/subcycling/test_2d_subcycling_mr` 注册为 2D、2-rank case；输入设定 `amr.max_level = 1`、`warpx.do_subcycling = 1`、moving window 和一次 step 250 的 Full diagnostics。这里的 `max_level = 1` 表示 level 0 与 level 1 两层，不是“只有一层”；`OneStep_sub1()` 还会在运行时拒绝电静力路径、超过两层的层级，或任何方向 refinement ratio 不为 2 的配置。因而改 AMR 结构、几何或时间步之前，应先把这些分派条件与实际输入逐项核对；仅出现细网格或一个 `diag1000250` 目录，都不能证明进入了这条时间合同。

第二层：把一个 coarse 步按 source 生命周期阅读。第一 fine 小步后，代码先 restriction fine `current_fp` 和 `rho`，再完成过滤、边界和该层场推进；随后 coarse 粒子沉积的电流被

StoreCurrent() 保存, fine contribution 经 AddCurrentFromFineLevelandSumBoundary() 叠入, coarse/fine 场各推进半个 coarse 步。中点的 UpdateAuxiliaryData() 为下一次 fine gather 准备跨层辅助场。第二 fine 小步重复 restriction, 随后 RestoreCurrent() 恢复 coarse 自身的另一半 source, 再叠加 fine contribution 并完成第二个 coarse 半步。

这条顺序说明 current_fp、current_cp、current_buf 和 rho 在不同阶段不是可互换的同义词; StoreCurrent()/RestoreCurrent() 也不是可删除的缓存优化。若修改 restriction、buffer、过滤、auxiliary gather 或 half-step 次序, 应直接记录每个阶段的 source 与时间标签, 才可能判断 coarse source 是否仍对应两个 fine 时间片。单个末态 plotfile 不能替代这份 route-level ledger。

第三层: 正确解读官方测试的 **consumer**。该 CTest 的 analysis 槽位是 OFF, 只调用 analysis_default_regression.py --path diags/diag1000250。后者把输出交给 checksum API; 因此 PASS 只能说明在这个 2D、2-rank、CKC、PML、moving-window 和指定粒子配置下, 最终 Full output 没有偏离已登记的回归基线。它是必要的输出回归, 不是解析场、连续性、两级 source 账本或 transition-zone route-count 的物理证明。

第四层: 按改动对象补上缺失的比较。改分派、level 数或 refinement ratio 时, 先用第一层确认入口和拒绝条件; 改 current/rho restriction、buffer 或同步时, 第二层要求 pre/post restriction、coarsen 与边界后的同一字段比较; 改场更新、PML 或 moving window 时, 另选与物理问题匹配的解析解、守恒量或独立 reference; 改 diagnostics/writer 时, 再按第 8 章的 producer/consumer 卡检查采样时间。若没有对应 comparison, 结论只能是“分支或回归尚可检查”, 不能写成“AMR subcycling 的物理正确性已经验证”。

3.14 参数示例与最小运行闭环

如果把本章压成一个最小、可执行、可回查的演化入口, 可从下面的官方回归案例开始:

- Examples/Tests/langmuir/inputs_test_1d_langmuir_multi

输入文件直接消费了本章讲到的一组顶层参数:

- max_step = 80
- algo.current_deposition = esirkepov
- algo.field_gathering = energy-conserving
- warpx.cfl = 0.8
- 周期场边界

这里需要避免一个常见误读: 这个输入 没有显式设置 algo.maxwell_solver。因此它不能用于证明某个 solver 参数由该输入给出; 实际采用的默认 solver 必须回到所用版本的 ReadParameters() 与官方文档确认。

它成为可执行案例, 是因为 Examples/Tests/langmuir/CMakeLists.txt 把它注册为 test_1d_langmuir_multi: 一维、2 个 MPI rank, 分析命令为 analysis_1d.py diags/diag1000080。也就是说, max_step = 80 只规定步数; 最终 plotfile 名称和分析入口来自 CMake 注册, 不能单凭步数猜出。

对本章来说, 这个案例最重要的意义不是“Langmuir 物理本身”, 而是它把主循环连接到明确的消费端。对于这个显式电磁、无 AMR 的输入, 主路径是:

```

main.cpp
-> WarpX::GetInstance()
-> InitData()
-> ComputeDt()
-> Evolve()
-> OneStep()
-> PushParticlesandDeposit()
-> SyncCurrentAndRho()
-> EvolveB/EvolveE/EvolveB
-> diagnostics

```

分析脚本 `analysis_1d.py` 读取最终 plotfile，按解析的 Langmuir 波重建 E_z ，并要求最大相对误差满足

$$\max |E_{z,\text{sim}} - E_{z,\text{theory}}| / \max |E_{z,\text{theory}}| < 0.05.$$

随后它还调用 `check_charge_conservation(data)`。对这个 Esirkepov、非 PSATD、非 RZ 的案例，辅助分析会检查离散 Gauss-law 残差；这补充了场形状的比较，但不等同于证明所有几何、所有 solver 或 AMR 分支的守恒性。

这样，`inputs_test_1d_langmuir_multi` 把本章的“主循环入口”压成一个可复现的闭环：

- 有参数入口：输入文件
- 有测试编排：`test_1d_langmuir_multi` 的 CMake 注册与 2-rank 规模
- 有明确消费端：`analysis_1d.py diags/diag1000080`
- 有两类检查： E_z 解析误差阈值和离散 Gauss-law 残差

这条路线把 WarpX 主类生命周期和 `Evolve()` 主链从静态调用图连接到输入、输出和可检查的物理量。它只覆盖这个 Langmuir 配置，不能据此推出所有 solver、几何或 AMR 分支都已验证。

3.14.1 生命周期追踪卡：一项输入何时成为可解释的证据

仅仅在输入文件里看到 `max_step = 80`，还不能说明它已经控制了实际运行，更不能说明末态分析已经成立。对本节的 Langmuir 案例，可以把这一项参数沿源码分成四个不可互换的观察点：

1. 参数被读取。Source/main.cpp 先取得 WarpX 单例；`WarpX::WarpX()` 随即调用 `ReadParameters()`。在 Source/WarpX.cpp 中，`ReadParameters()` 用无前缀的 `ParmParse` 查询 `max_step` 并写入 `WarpX::max_step`。这说明输入键已经进入主演化对象，但此时网格、粒子、diagnostics 和任何输出都还没有建立。
2. 初始化已经越过记录点。fresh run 的 `WarpX::InitData()` 依次调用 `ComputeDt()`、`InitFromScratch()` 和 `InitDiagnostics()`；在完成 guard-cell 检查和参数打印后，才调用 `WriteUsedInputsFile()` 写出 `warp_x_used_inputs`。因此，`warp_x_used_inputs` 出现能证明这次进程已经走到 `InitData()` 的这一后段并记录了生效输入；它不能证明 `Evolve()` 已执行一步，也不能证明任何解析误差或守恒 gate 已通过。

3. 参数实际限制外层循环。Source/Evolve/WarpXEvolve.cpp 中，默认 Evolve() 把 max_step 作为 numsteps_max。同一文件在 istep[0] == max_step 时识别最终时间步并请求 diagnostics flush。可是“达到第 80 步”仍不等于“产生 diag1000080”：本输入还必须有 diag1.intervals = 40、diag1.diag_type = Full，而具体目录名和分析器仍由测试编排决定。
4. **consumer** 给出可支持的结论。Examples/Tests/langmuir/CMakeLists.txt 把这个 input 注册为 1D、2-rank 的 test_1d_langmuir_multi，并指定 analysis_1d.py diags/diag1000080。脚本再从文件时间重建理论场、使用 0.05 场误差阈值并调用离散 Gauss-law 检查。只有到这一层，才有指定输入、指定并行布局 and 指定末态下的物理比较；它仍不覆盖其他 solver、几何或修改后的输入。

读者可用下面四条检索把这条链从自己的 WarpX 源树中重新定位。它们是源码导航，不会启动模拟：

```
rg -n 'queryWithParser\(pp, "max_step"' "$WARPX_ROOT/Source/WarpX.cpp"
rg -n 'WriteUsedInputsFile'
  ↪ "$WARPX_ROOT/Source/Initialization/WarpXInitData.cpp"
rg -n 'numsteps_max|final_time_step'
  ↪ "$WARPX_ROOT/Source/Evolve/WarpXEvolve.cpp"
rg -n -A 8 'test_1d_langmuir_multi' \
  "$WARPX_ROOT/Examples/Tests/langmuir/CMakeLists.txt"
```

这也给出一个更可靠的故障分类。若没有 warpx_used_inputs，应优先检查启动、参数读取或初始化前/中的错误；若它存在但缺少预期 diags/diag1000080，应核对实际停止步、diagnostic 周期、诊断类型和运行是否在写出前终止；若 plotfile 已存在但 analysis_1d.py 失败，应回到 consumer 读取的字段、文件时间、reference 和阈值。三种情形分别对应生命周期的不同层，不能统一归因成“参数无效”或“Langmuir 物理失败”。

因此，参数到结论的最小追踪链是：

```
inputs 中的 max_step
-> ReadParameters() 读取到 WarpX 状态
-> InitData() 写出 warpx_used_inputs
-> Evolve() 用 max_step 限制外层步
-> Full diagnostics 按 intervals 写出末态
-> CTest 注册的 analysis consumer 比较 reference
```

第 2 章的命令行受控修改路线正是这张卡的后续：覆盖 max_step 可以改变前四段，却不会替第五、六段重新定义诊断周期、consumer 或物理阈值。修改后必须重新核对完整链，而不是只凭一份 warpx_used_inputs 或一个退出码下结论。

3.15 进一步阅读与练习

进一步阅读：

1. 第 4 章：粒子推进器：继续从 PushParticlesandDeposit() 进入 mypc->Evolve() 和粒子推进器。
2. 第 5 章：沉积与形函数：继续展开 SyncCurrentAndRho()、沉积、guard/source synchronization。

3. 对 lifecycle、subcycling/JRhom 与 moving window 三条分支，继续分别追问：状态在哪个时间层更新、哪些网格层参与、以及结果应通过哪个 observable 判断。

练习题：

1. 说明为什么 `WarpX::WarpX()` 里只能创建跨-level 外壳，而不能直接分配完整 `MultiFab` 主字段。
2. 用本章的 `StoreCurrent()/RestoreCurrent()` 解释：为什么 subcycling 不能简单拿 fine current 覆盖 coarse current。
3. 结合本章的 Langmuir 案例，指出 `inputs_test_1d_langmuir_multi` 中哪些参数分别进入 `ReadParameters()`、`ComputeDt()` 和 `OneStep_nosub()` 的不同层次。
4. 案例闭环题。阅读该输入、`Examples/Tests/langmuir/CMakeLists.txt` 和 `analysis_1d.py`，写出四行表格：输入参数、测试规模、分析命令、通过条件。解释为什么 `max_step = 80` 不能单独推出输出目录或通过阈值，并说明两类检查各自能与不能支持什么结论。

3.16 本章小结

本章建立的主演化路线可压缩为五个读者检查点。它们按一个输入从启动到一次时间步的顺序排列，而不是按源码文件的出现顺序排列：

1. 启动与构造：从 `main.cpp` 进入 `GetInstance()`、`WarpX::WarpX()` 和 `ReadParameters()`。先问哪些参数与 solver 分支在建立网格前已经确定。
2. 初始化：依次检查 `InitData()`、`ComputeDt()`、`InitFromScratch()` 或 `InitFromCheckpoint()`。确认初始 level、粒子、场、PML 和 diagnostics 在第一步前已就绪。
3. 外层推进：在 `Evolve()` 中核对 callback、负载均衡、注入、边界与诊断分别发生在单步前还是单步后。
4. 单步分派：在 `OneStep()` 中判断该输入实际走 implicit、electrostatic/HybridPIC、`OneStep_nosub()`、subcycling 还是 JRhom。
5. 显式主链：从 `PushParticlesandDeposit()` 到 `SyncCurrentAndRho()`，再到 `PushPSATD()` 或 `EvolveB/EvolveE/EvolveB`，确认粒子输运、源项同步和场推进的时间层相容。

3.16.1 跨章交接卡：从调用图保留到可验证的状态

第 3 章的调用图只说明控制流进入了哪些阶段；它不能替代后续章节对变量、时间层和 observable 的定义。读者从本章进入初始化、推进、沉积和诊断时，至少应保留下面四项信息：

交接问题	本章已经定位的入口	下一章必须继续确认的内容	不能直接推出的结论
输入动量在什么单位下被解释?	ReadParameters() 、species/injector 配置	输入中的 $\gamma m b$ eta、容器中的 $\gamma m m a * v$ 与 diagnostics metadata 的差异，见 附录 A	同一个 ux/uy/uz 数值在输入、粒子数 组和输出中可直接互 换
第一个物理时间步前 有哪些状态?	InitData()、In itFromScratch() /InitFromCheckp oint()	第 3A 章中的 level、 field、PML、exter- nal field、species 与 diagnostics 初始化顺 序	一次对象分配或 writer 创建就证明初 态的物理约束已满足
显式粒子轨迹怎样变 成 solver source?	PushParticle sandDeposit()、 mypc->Evolve()、 SyncCurrentAnd Rho()	第 4、5 章中的 $\mathbf{x}^n$ 、 $\mathbf{u}^{n-1/2}$ 、 $\mathbf{J}^{n+1/2}$ 、 old/new ρ 以及 AMR/边界同步	任意 rho 或 curr ent_* 数组都已经 是场求解器消费的最 终源项
怎样判断这条路径可 信?	Evolve()、OneS tep() 与 diagnostics 调度	第 6–8 章中的 solver 前提、边界条件、 producer/consumer、 reference 和 observ- able	程序完成、文件写出 或 checksum 一致就 证明物理结论

因此，跨章阅读时可把一条输入压缩成下列核查链：

输入量纲与配置
-> InitData 创建的离散初态
-> 一个时间步内的粒子/源项/场时间层
-> 同步后由 solver 消费的状态
-> 有独立 reference 的 observable

这张交接卡的用途是防止两类常见跳步：把输入参数的语义直接当成内部数组语义，或把控制流命中和输出文件存在直接当成物理验证。后续章节必须分别补全这条链上的离散公式、实现分派和可检验的证据。

第四章 3A. WarpX 初始化链：从 `InitData()` 到初始粒子和外部场

本章把 `WarpX::InitData()` 展开成一条完整的初始化链。它补足第 3 章中“初始化”只作为主循环前置步骤的不足：这里开始逐块解释 fresh run / restart、AMR level 初始化、外部场、species 注入器、粒子创建 kernel、Gaussian beam、openPMD 文件注入和 projection divergence cleaning。

阅读初始化代码时，应先围绕四个主入口建立因果关系：`InitData()` 决定 fresh/restart 分叉，`InitFromScratch()` 建立初始 level，`InitDiagnostics()` 准备可观察输出，`AddExternalFields()` 把外部场加入初态。使用不同 WarpX 版本时，优先按这些函数的职责和调用关系检索，不要把行号或分支名称当作初始化语义。

读者问题	首先追踪的对象
程序启动后哪些参数在构造对象前已被锁定？	参数读取、 <code>WarpX::WarpX()</code> 与 <code>ReadParameters()</code>
参数、level 字段和 <code>InitData()</code> 的顺序如何落到对象上？	<code>InitFromScratch()</code> 、level allocation 与 field data
外场、species、粒子创建与散度修正分别如何进入初态？	<code>AddExternalFields()</code> 、 <code>PlasmaInjector</code> 、粒子创建与 projection cleaner
哪些案例能区分不同初始化路径？	initial distribution、space charge、external field 与 restart 的观察量

4.0.1 阅读路线：先构造初态，再追踪分支

初始化代码同时涉及参数、AMR、粒子、外部场和 diagnostics。第一次阅读应先建立“第一步推进之前哪些状态必须已经存在”的主线，再进入特定 injector 或 I/O 分支：

1. 先读 **3A.1–3A.3**。区分构造期、bootstrap、fresh run 与 restart，写清哪些参数会在对象构造或 `InitData()` 分叉前锁定；这一步回答初态从哪一个全局配置开始。
2. 再读 **3A.4–3A.5**。沿 level allocation、field data、PML、diagnostics 与外部场建立网格侧初态，特别区分叠加到网格的 external field 和在 gather 时供粒子消费的 external field；这一步回答第一步能读到哪些场对象。
3. 按输入类型读 **3A.6–3A.12**。`PlasmaInjector`、体注入、Gaussian beam、openPMD 和 projection cleaner 是不同的初态构造路径。选择其中一条时，先记录它创建的粒子/场对象、时间位置和限制，而不是把它们统称为“加载初始条件”。
4. 最后读 **3A.13–3A.16**。用匹配的 regression、历史最小骨架和练习检验初态是否真的进入 `Evolve()`；结论必须区分输入/接口覆盖、writer 输出和物理 observable。

这条路线的停止条件是：读者能够从一个输入参数追踪到它创建的初态对象，并说明第一个时间步的哪个阶段会消费该对象。

4.1 3A.1 初始化链为什么值得单独成章

理解初始化时，先把以下三层边界分开：

1. `WarpX::WarpX()` 构造期只完成参数读取和跨 level 外壳创建。
2. `InitData()` 才在 fresh run / restart 之间分叉，并把 AMR level、粒子、诊断、PML、外场和初始静电/磁静场组织到第一步之前。
3. species 注入、外部场、projection cleaning、openPMD/Gaussian beam 分别有自己的参数、时间层和验证边界，不能笼统地称为“初始化完成”。

PIC 程序的时间推进方程通常写成：

$$f^n, \mathbf{E}^n, \mathbf{B}^n \longrightarrow f^{n+1}, \mathbf{E}^{n+1}, \mathbf{B}^{n+1}.$$

但第一个时间步之前必须先构造一个满足几何、边界、粒子权重、外部场、离散约束和并行布局的初态。WarpX 的初始化不是“读入参数然后开始跑”，而是完成以下任务：

1. 选择 fresh run 或 checkpoint restart。
2. 建立 AMReX level、field MultiFab、PML、EB 和 solver 数据结构。
3. 初始化 diagnostics 和 reduced diagnostics。
4. 创建 species、解析密度/动量/位置分布，并生成宏粒子。
5. 读入或解析外部场，并把外部场叠加到 self field 或 particle external field。
6. 对外部 A/B 场做 projection divergence cleaning。
7. 输出第 0 步 diagnostics，并检查 guard cell、solver 配置和 known issue。

本章先建立从程序启动到第一个时间步的主干；需要实现级细节时，再沿源码文件、函数和调用关系逐项核对。

4.2 3A.2 启动层先于 `InitData()`：MPI、AMReX、FFT、PETSc 与启动前提

许多初始化说明从 `WarpX::InitData()` 往后讲，但 WarpX 真正的初始化链更早就开始了。`main.cpp` 最外层先做的是：

```
int
main (int argc, char* argv[]) {
    warpx::initialization::initialize_external_libraries(argc, argv);
    {
        auto& warpx = WarpX::GetInstance();
        warpx.InitData();
    }
}
```

```

        warpx.Evolve();
        WarpX::Finalize();
    }
    warpx::initialization::finalize_external_libraries();
}

```

这说明 `InitData()` 之前还有一层独立的 bootstrap 逻辑，负责：

1. 初始化 MPI、AMReX、FFT，以及可选 PETSc。
2. 覆盖一批 AMReX 默认 parser 行为。
3. 解析 geometry、periodicity 和整数数组表达式。
4. 锁定 `geometry.dims`、moving window 和 warning policy 这类全局运行前提。

这一层主要分布在：

- `Initialization/WarpXAMReXInit.*`
- `Initialization/WarpXInit.*`
- `WarpX::MakeWarpX()`

4.2.1 3A.2.1 参数系统先分命名空间，再分读取语义

WarpX 的参数不是“一个大表一次性读完”。它先按 `ParmParse` 前缀拆成局部命名空间，然后再决定是：

- 立即求值成数值；
- 保留 parser 字符串，稍后编译；
- 解释成 interval 切片；
- 还是先解析成枚举型算法选择。

最上游的运行控制参数就是无前缀直接读取：

```

const ParmParse pp; // Traditionally, max_step and stop_time do not have
    ↪ prefix.
utils::parser::queryWithParser(pp, "max_step", max_step);
utils::parser::queryWithParser(pp, "stop_time", stop_time);

```

而大部分初始化参数则先按前缀取视图：

```

const ParmParse pp_amr("amr");
const ParmParse pp_algo("algo");
ParmParse const pp_warpx("warpx");
const ParmParse pp_geometry("geometry");
const ParmParse pp_boundary("boundary");
const ParmParse pp_psatd("psatd");

```

接下来又会分成几种读取壳：

1. `queryWithParser/queryArrWithParser`

- 适合 `warpx.cfl`、`const_dt`、`amr.n_cell` 这类“读入就该变成数值”的参数；

2. `Store_parserString + makeParser`

- 适合 `ref_patch_function(x,y,z)`、外场 `parser`、`diagnostics` 过滤函数这类“先保留表达式，再在别处编译执行”的参数；

3. `IntervalsParser`

- 适合 `sort_intervals`、`dt_update_interval`、`diagnostics intervals` 这类切片语法，不应误看成普通整数；

4. `query_enum_sloppy + AMREX_ENUM`

- 适合 `algo.evolve_scheme`、`warpx.do_electrostatic`、`algo.current_deposition` 这类算法枚举入口。

这层结构解释了为什么同样都出现在 `parameters.rst` 里，`max_step`、`ref_patch_function(x,y,z)` 和 `algo.evolve_scheme` 在源码中会走三种完全不同的消费链。后续参数索引如果想真正有用，就必须至少区分：

- 参数前缀；
- 第一次被哪个 `ParmParse` 命名空间读取；
- 它属于数值参数、`parser` 字符串、`interval` 还是枚举型算法选择。

例如 `WarpXAMReXInit.cpp` 并不是简单调用 `amrex::Initialize()`，而是把一个缺省值覆盖回调一起传进去：

```
amrex::Initialize(
    argc,
    argv,
    build_parm_parse,
    MPI_COMM_WORLD,
    ::overwrite_amrex_parser_defaults
);
```

这个回调会统一注入常量、改写 `amrex.abort_on_out_of_gpu_memory`、`amrex.the_arena_is_managed`、`amrex.omp_threads`、粒子 `tiling` 缺省值，以及由 `boundary.field*/boundary.particle*` 反推 `geometry.is_periodic`。换句话说，WarpX 从程序一启动就已经不是“AMReX 原生默认设置”。

同一个文件还会在 AMReX 初始化后立即把 `geometry.prob_lo/prob_hi`、`amr.n_cell`、`max_grid_size*`、`blocking_factor*` 这类可能带表达式的输入预解析并写回 `parser`，保证后面的 `Geometry`、`warpx_job_info` 和 `yt` 读到的是数值结果，而不是未展开的表达式字符串。

4.2.2 3A.2.2 文档 alias、AMReX-owned 参数与 WarpX 自有 parser 不是一回事

参数索引继续往下清理后，会发现还有一类参数不能简单用“有没有 grep 到同名字符串”来理解。典型例子是：

- geometry.prob_lo/hi
- boundary.field_lo/hi
- boundary.potential_lo/hi_x/y/z
- psatd.nox/noy/noz
- qed_schwinger.xmin/ymin/zmin/xmax/ymax/zmax

这些名字在文档里是 grouped alias，但源码里真实读取的仍然是拆开的 key。以 geometry.prob_lo/hi 为例，WarpX 真正做的是：

```
utils::parser::getArrWithParser(
    pp_geometry, "prob_lo", prob_lo, 0, AMREX_SPACEDIM);
utils::parser::getArrWithParser(
    pp_geometry, "prob_hi", prob_hi, 0, AMREX_SPACEDIM);

pp_geometry.addarr("prob_lo", prob_lo);
pp_geometry.addarr("prob_hi", prob_hi);
```

所以 geometry.prob_lo/hi 只是文档层“成对参数”的写法，真正 parser key 仍然是 prob_lo 和 prob_hi。boundary.field_lo/hi、boundary.particle_lo/hi、boundary.potential_lo/hi_x/y/z、psatd.nox/noy/noz 和 Schwinger 区域边界框也都属于这一类。

还要再区分另一类：参数确实出现在 WarpX 手册里，但 WarpX 并不直接读取，而是由 AMReX 自己消费，WarpX 只消费结果。例如 amr.ref_ratio/amr.ref_ratio_vect 和 amrex.async_out/amrex.async_out_nfiles。本书引用的 WarpX 源树没有独立的：

```
ParmParse("amr").query("ref_ratio", ...)
ParmParse("amrex").query("async_out", ...)
```

但后续代码会直接消费已经构造好的 ref_ratios，或者在 plotfile I/O 邻近层继续设置 WarpX 自己的 field_io_nfiles/particle_io_nfiles。因此，参数索引如果要稳定，至少要区分三类：

1. WarpX 自身直接 parse；
2. WarpX 子对象 parse；
3. AMReX-owned 输入，WarpX 只消费 materialized 结果。

另一条关键链在 WarpX::MakeWarpX()：

```
warpX::initialization::check_dims();
warpX::initialization::read_moving_window_parameters(...);
ConvertLabParamsToBoost();
parse_field_boundaries();
parse_particle_boundaries(...);
CheckGriddingForRZSpectral();
m_instance = new WarpX();
```

因此单例构造前就已经锁定了四类全局事实：

- 当前可执行文件与 `geometry.dims` 是否匹配；
- moving window 是否开启、方向和速度是什么；
- field / particle boundary 类型是什么；
- RZ spectral 的 gridding 约束是否满足。

例如 `check_dims()` 直接把编译维度和 `geometry.dims` 做强一致断言，而 `read_moving_window_parameters()` 会把输入文件里的 `moving_window_v` 从“以光速为单位的无量纲参数”转换成真正的物理速度 $v = (\cdots)c$ ，再写进 WarpX 的全局静态状态。

这里还要补一层工程来源。`check_dims()` 能做这种强断言，前提不是“WarpX 在运行时随意切换几何”，而是构建系统已经先把几何变体锁死了。当前主构建链在顶层 `CMakeLists.txt` 中通过 `WarpX_DIMS` 生成一组按维度后缀分裂的 `lib_${SD} / pyWarpX_${SD}` target；旧 GNUmake 链则通过 `DIM`、`USE_RZ` 以及 `-DWARPX_DIM_3D`、`-DWARPX_DIM_XZ`、`-DWARPX_DIM_RZ` 这组宏编码几何变体。因此初始化阶段读到的 `geometry.dims` 实际是在和“当前可执行文件已经按哪一种几何编译出来”做一致性检查，而不是在决定后面要不要临时切换到另一种维度。

再往下，WarpX 构造函数开头还会调用：

```
warpX::initialization::initialize_warning_manager();
```

也就是在任何 `ReadParameters()` 和 `InitData()` 之前，先读入：

- `warpX.always_warn_immediately`
- `warpX.abort_on_warning_threshold`

这说明 warning manager 在 WarpX 里也是初始化启动层的一部分，而不是后面运行时再临时决定的日志选项。

这一层再往下细分，还要注意 `MakeWarpX()` 和构造期 `ReadParameters()` 之间并不是简单前后顺序，而是“前者已经开始改写后者将要读取的参数”。例如 `ConvertLabParamsToBoost()` 不只是保存 `gamma_boost`，而是会先把：

- `geometry.prob_lo/prob_hi`
- `warpX.fine_tag_lo/fine_tag_hi`
- `slice.dom_lo/dom_hi`

按 boosted-frame 规则直接回写进 parser。若同时启用 moving window，它计算的转换系数还会显式依赖 moving_window_v/c。因此 boosted-frame 与 moving-window 的第一次耦合，其实发生在 ReadParameters() 之前，而不是发生在后面的 Evolve()。

同样，CheckGriddingForRZSpectral() 也不是单纯做断言。对 WARPX_DIM_RZ + algo.maxwell_solver = psatd 这条组合，它会在构造 WarpX 对象之前直接改写：

- amr.blocking_factor_x/y
- amr.max_grid_size_x/y

并要求 longitudinal 方向至少满足“每个 MPI rank 至少有一个 block，且每个 rank 至少有 8 个 z cells”的约束。也就是说，构造函数里的 ReadParameters() 读到的并不总是用户输入文件里的原始 AMR 分块值，而往往已经是启动层重写过的 spectral-compatible 版本。

把这一点看清之后，初始化启动层就能更准确地拆成两段：

1. 预初始化与 parser 改写段：

- initialize_external_libraries()
- amrex_init()
- parse_geometry_input()
- read_moving_window_parameters()
- ConvertLabParamsToBoost()
- CheckGriddingForRZSpectral()

2. 构造与运行态落地段：

- initialize_warning_manager()
- ReadParameters()
- BackwardCompatibility()
- InitEB()
- MultiParticleContainer / ParticleBoundaryBuffer / solver objects‘ 的真正创建

这个分层很有用，因为后面再读 boosted-frame 例子、RZ spectral gridding、moving window 连续注入或 warning manager，就不会再把“参数在哪一步被锁定”与“对象在哪一步消费这些参数”混在一起。

再往前走一步，ReadParameters() 还不只是“把输入存进成员”。它已经在替后面的对象图做分叉。例如：

- do_fluid_species 先通过 fluids.species_names 决定，随后直接控制是否创建 MultiFluidContainer；
- electrostatic_solver_id 与 electromagnetic_solver_id 先在 ReadParameters() 中互相覆盖和约束，随后决定静电 solver 具体实例化成 LabFrameExplic itES、EffectivePotentialES 还是 RelativisticExplicitES；
- electromagnetic_solver_id == HybridPIC 时，构造函数后半段才真正创建 HybridPICModel；

- `grid_type`、`do_current_centering`、`n_rz_azimuthal_modes` 等参数先在 `ReadParameters()` 中完成默认值推导和合法组合检查，后面再影响 `nodal flags`、`centering coefficients`、`fluid` 兼容性与容器初始化。

同样，`ReadParameters()` 里还有一类“源码先推导默认，再由用户输入覆盖”的派生状态。最典型的是：

```
if (!do_divb_cleaning
    && m_p_ext_field_params->B_ext_grid_type !=
        ExternalFieldType::default_zero
    && m_p_ext_field_params->B_ext_grid_type !=
        ExternalFieldType::constant
    ...
    && (electromagnetic_solver_id == Yee
        || electromagnetic_solver_id == HybridPIC
        || ...))
{
    m_do_initial_div_cleaning = true;
}
pp_warpX.query("do_initial_div_cleaning", m_do_initial_div_cleaning);
```

这里 `do_initial_div_cleaning` 不是单纯从输入文件机械读取，而是先根据外部场类型、`grid type` 和 `solver` 组合推导一个默认值，再允许用户显式覆盖。也就是说，后面 `ProjectionDivCleaner` 是否参与初态构造，不是孤立地由一个布尔开关决定，而是由初始化启动层和 `ReadParameters()` 主体共同塑造的。

到这一层为止，初始化阶段已经可以更清楚地压成一条完整链：

1. 启动层先改 `parser`、锁定全局静态状态。
2. `ReadParameters()` 再把这些值落到 `WarpX` 成员，并决定对象图分叉。
3. 构造函数后半段据此创建 `MultiParticleContainer`、`ParticleBoundaryBuffer`、`MultiFluidContainer` 和 `solver objects`。
4. 最后 `InitData()` 才真正开始分配 `level data`、生成粒子和构造初态。

但这里还差最后一层经常被误读的东西：`ReadParameters()` 里那些看上去分散的算法检查，其实会继续决定后面到底分配哪类 `MultiFab`、是否创建 `implicit solver` 额外状态，以及 `ProjectionDivCleaner` 是否在初态构造中插队。

最紧的一组约束是：

- `grid_type=hybrid` 会把 `do_current_centering` 推成默认真值，并要求 `algo.field_gathering=momentum-conserving`；
- 这又会让 `AllocLevelData()` 中的 `aux_is_nodal=true`，从而把 `Efield_aux/Bfield_aux`、后续 `coarse-aux cax` 和 `gather buffer` 路径切到 `nodal` 版本；
- 如果同时显式要求 `do_current_centering=1`，源码只允许 `grid_type=hybrid`，并且在 `level` 分配时额外分配 `current_fp_nodal`。

换句话说，`grid_type`、`field_gathering_algo`、`do_current_centering` 在这里并不是三个平行开关，而是一组会继续改写场 `index type` 和附加存储布局的上游选择器。

同样，`current_deposition_algo` 也不是只影响某个沉积 `kernel`。源码先把 `PSATD`、`HybridPIC` 和 `electrostatic` 的默认 `deposition` 拉回 `Direct`，再对 `Vay` 加上三重限制：

- 只能配 PSATD
- 不能配 mesh refinement
- 不能配 do_current_centering

这条限制后面会直接落成额外字段分配：如果真的选了 Vay, AllocLevelMFs() 会专门分配 current_fp_vay。所以它已经是初始化对象图的一部分，而不只是运行时算法分派。

implicit 系列更明显。ReadParameters() 一旦看到

- semi_implicit_em
- theta_implicit_em
- strang_implicit_spectral_em

就会立即构造 m_implicit_solver，并同时要求：

- current deposition 只能是 Esirkepov / Villasenor / Direct
- EM solver 只能是 Yee / CKC / PSATD
- particle pusher 只能是 Boris / HigueraCary
- field gather 不能是 momentum-conserving

然后这条链会继续穿到 InitFromScratch() 和 AllocLevelMFs()：

- InitFromScratch() 在 mypc->InitData() 之前调用 m_implicit_solver->Define() 和 CreateParticleAttributes();
- AllocLevelMFs() 在 fine patch 上额外分配 current_fp_non_suborbit 与 E_old。

因此 implicit 不是“场求解器章节内部的一个局部话题”，而是在初始化阶段就已经改变粒子属性表和字段分配合同。

particle_shape、filter 和 projection div cleaning 也属于同一种“前置组合约束”。只要存在 particles 或 lasers, algo.particle_shape 就变成强制参数，并直接设定 nox=noy=noz；这反过来继续影响 guard-cell 配额、沉积 stencil 和排序默认值。use_filter 也不是后面可有可无的一步卷积，因为 AllocLevelData() 会先 InitFilter()，再把 filter stencil 长度交给 guard_cells.Init(...)，于是它会继续改写 guard-cell 和 buffer 需求。

最后，m_do_initial_div_cleaning 也不是孤立的布尔开关。源码先根据外部 B 场类型、EM solver 组合和是否已经启用 do_divb_cleaning 推导默认值，再允许用户覆盖；而它的下游消费点就在初始化主链里，直接决定 ProjectionDivCleaner 是否参与初始场构造。所以到这一步为止，ReadParameters() 的真实地位可以概括成一句话：

它不是单纯读参数，而是在 AllocLevelData() 和 InitFromScratch() 之前，先把“允许哪种物理-算法-存储组合存在”这件事裁成一个可执行的对象图。

接下来再往下走一步，就会看到 AllocLevelMFs() 真正把这张对象图摊成一组具体字段。这里最容易误判的是 rho、aux、外场、HybridPIC/fluid/macrosopic/EB 这些分支。

先看 rho_fp。它并不是总存在的基础字段，而是只在几类路径下显式分配：

- electrostatic / electromagnetostatic / effective-potential
- HybridPIC
- do_dive_cleaning
- PSATD 且启用了 update_with_rho 或 current_correction

而且它的分量数也不是固定值：普通 electrostatic 或 HybridPIC 只需要 ncomps, do_dive_cleaning 一般把它升到 2*ncomps, PSATD 又会根据 JRhom 是否开启在 ncomps 和 2*ncomps 之间切换。也就是说, WarpX 初始化阶段持不持有一份“可演化的 rho 状态”, 本身就是 solver contract 的一部分。

与此平行的还有三类不同的辅助标量/约束场:

- phi_fp: 只属于 electrostatic 系列;
- F_fp: 只属于 div(E) cleaning;
- G_fp: 只属于 div(B) cleaning。

不要把它们混成“又分配了几个标量场”。它们分别服务于 Poisson 势解、电场散度清理和磁场散度清理, 而且 G 的 coarse-patch index type 还会继续受 grid_type 控制。

再看外场分支。源码实际上维护了两套完全不同的合同:

1. grid external fields

它们分配成 Efield_fp_external/Bfield_fp_external, index type 必须匹配 fp, 后面由 AddExternalFields() 加到主网格场上。

1. particle external fields

它们分配成 E_external_particle_field/B_external_particle_field, index type 必须匹配 aux, 而且分量数直接来自外部粒子场元数据。

所以 particle external field 不是 grid external field 的别名; 它跟粒子 gather 所看的 aux 路径绑定得更紧。这也解释了为什么只要 mypc->m_E_ext_particle_s 或 m_B_ext_particle_s 是 read_from_file, 最常见的 aux -> fp alias 优化就会被打断, Efield_aux/Bfield_aux 必须改成单独分配。

剩下几条看似“模块化”的分支, 其实也都在 AllocLevelMFs() 里继续扩展 field registry:

- electromagnetic_solver_id == HybridPIC 时, m_hybrid_pic_model->AllocateLevelMFs(...) 会追加 hybrid 自己的 level 字段;
- do_fluid_species 时, myfl->AllocateLevelMFs(...) 之后立刻 InitData(...), 说明 fluid 已经进入初始化主链, 而不是仅仅挂了个容器;
- m_em_solver_medium == Macroscopic 时, m_macroscopic_properties->AllocateLevelMFs(...) 只允许 lev==0, 直接把 mesh refinement 排除在外;
- EB::enabled() 时, 所有 level 至少都会有 distance_to_eb 与 m_eb_reduce_particle_shape, 而 finest level 上又会继续长出 m_eb_update_E/B; 如果 solver 还是 ECT, 则再额外长出 edge_lengths、face_areas、area_mod、Venl、ECTRhofield 和 FaceInfoBox 借用关系。

因此 `AllocLevelMFs()` 的真实角色不是“机械把 `MultiFab` 开出来”，而是把前面 `ReadParameters()` 裁出来的对象图真正展开成可执行状态。

更关键的是，这些特例字段不会等到 `Evolve()` 才第一次使用。在 `InitData()` 后半段，源码会立刻：

- `BuildBufferMasks()`
- `m_macroscopic_properties->InitData(...)`
- `m_electrostatic_solver->InitData()`
- `m_hybrid_pic_model->InitData(m_fields)`
- `ProjectionCleanDivB()`

这就说明初始化链的最后三步其实是：

1. `ReadParameters()` 决定哪些组合允许存在；
2. `AllocLevelMFs()` 把这些组合摊成真实字段和对象；
3. `InitData()` 后半段立刻消费这些字段，把它们变成一份可跑的初态。

再往后一步，`InitData()` 后半真正把这条合同闭合掉。它的顺序不是“初始化完就直接写 diagnostics”，而是先做一轮收尾和首次消费：

- `ComputeMaxStep()`
- `ComputePMLFactors()`
- 可选 `InitNCICorrector()`
- `BuildBufferMasks()`
- `m_macroscopic_properties->InitData(...)`
- `m_electrostatic_solver->InitData()`
- `m_hybrid_pic_model->InitData(m_fields)`
- `CheckGuardCells()`
- `ProjectionCleanDivB()`

这一步说明前面 `AllocLevelMFs()` 分配出来的对象并不是先放着，很多都会在这里立刻进入第一次初始化消费。

尤其要区分两步经常被混在一起的电场初始化动作：

1. `m_electrostatic_solver->InitData()`

这是 solver 对象自身的初始化，fresh run 和 restart 都会走。

1. `ComputeSpaceChargeField(reset_fields=false)`

这是 fresh-run 下的初始 self-field / electrostatic solve，只在需要时触发，而且还明确保留网格上已有的用户指定值，不会先把字段清空。

它的触发条件也不是“只有 electrostatic solver 才会跑”，而是三选一：

- 开启 electrostatic solver
- 任意 species 打开 `initialize_self_fields`
- 指定了 boundary potential

但如果当前走的是 HybridPIC，源码又会显式跳过这条初始 field-solve 路径。

在这之前，若 `m_do_initial_div_cleaning` 成立，还会先执行 `ProjectionCleanDivB()`。因此初始 B 场的散度修正顺序其实非常明确：先完成外场读入与对象初始化，再做 `div B` cleaning，随后才进入初始 self-field / electrostatic solve。

Python callback 也不是一个统一“初始化结束后调用”的事件，而是被拆成了三类窗口：

- `beforeInitEsolve`: 只在 fresh run，发生在初始场求解前；
- `afterInitEsolve`: 只在 fresh run，发生在初始场求解后、外加场提交前；
- `afterInitatRestart`: 只在 restart，表示恢复态后处理，而不是 fresh-run 初始求解窗口的一部分。

外加场这里也有两步，不能混读：

1. `LoadExternalFields()`

先把 external grid fields 装到 `Efield_fp_external/Bfield_fp_external`，把 particle-only external fields 装到 `E_external_particle_field/B_external_particle_field`，并在 finest level 给 Python `loadExternalFields` callback 一个写这些 buffer 的机会。

1. `AddExternalFields()`

再把 grid external fields 统一加回 `Efield_fp/Bfield_fp` 主场。

所以第 0 步最终主场不是“纯 self-field 结果”，而是“初始 self-field / electrostatic solve 结果，再叠加 grid external fields”。

这之后才进入第 0 步 diagnostics：

- `multi_diags->FilterComputePackFlush(istep[0]-1)`
- `reduced_diags->ComputeDiags(...)`
- `reduced_diags->WriteToFile(...)`

因此第 0 步输出的并不是“原始输入快照”，而是“初始化主链已经全部完成后的第一份可运行状态快照”。对于 restart，这一步则取决于 `write_diagnostics_on_restart`，表示是否要把恢复态也立刻写成一份 diagnostics。

4.3 3A.3 顶层入口: fresh run 与 restart 分叉

源码文件: Source/Initialization/WarpXInitData.cpp

函数: WarpX::InitData()

第 3 章已经给过总表, 这里看决定数据来源的核心分支。

```
if (!restart_chkfile.empty())
{
    InitFromCheckpoint(restart_chkfile);
    PrintDtDxDyDz();
    PostRestart();
}
else
{
    ComputeDt();
    PrintDtDxDyDz();
    InitFromScratch();
    InitDiagnostics();
}
```

这段分支决定初始化数据来源:

- restart: 从 checkpoint 恢复 mesh、field、particles 和时间层, 再做 restart 后处理;
- fresh run: 先由 CFL、网格和 solver 计算 dt, 再从零建立 AMR level、field、particles 和 diagnostics。

物理上, restart 应该恢复一个已经离散一致的状态; fresh run 则必须从输入参数构造这种一致状态。后面讲的外部场、初始粒子、Gaussian beam、openPMD 注入和 projection cleaning, 主要属于 fresh run 路径。

可以把 fresh run 与 restart 的责任边界压缩成下面的初始化路线表:

输入状态	主要调用顺序	进入 Evolve() 前必须具备的状态
checkpoint restart	InitFromCheckpoint() -> PostRestart()	从 checkpoint 恢复 AMR level、fields、particles 和时间层, 再完成 restart 后处理
fresh run	ComputeDt() -> InitFromScratch() -> MakeNewLevelFromScratch() -> mypc->AllocData()/InitData() -> PML -> InitDiagnostics()	根据输入重新建立 AMR、solver、粒子、外部场和初始 diagnostics

这张图的重点不是列出每一个初始化 helper, 而是固定数据来源的不可互换性: restart 路径恢复已经离散化的状态, fresh run 路径才负责从参数重新物化 AMR、solver、粒子、外部场和初始 diagnostics。后面的 PlasmaInjector、Gaussian beam、openPMD 文件注入和 projection cleaning 都必须放在 fresh-run 分支内理解, 不能被误写成 restart 的重复初始化。

4.4 3A.4 InitFromScratch(): AMReX level 与粒子初始化

源码文件: Source/Initialization/WarpXInitData.cpp

函数: WarpX::InitFromScratch()

AMReX 回调文件: Source/WarpX.cpp 回调函数: WarpX::MakeNewLevelFromScratch()

```
void
WarpX::InitFromScratch ()
{
    const Real time = 0.0;
    AmrCore::InitFromScratch(time); // calls MakeNewLevelFromScratch

    if (m_implicit_solver) {
        m_implicit_solver->Define(this, /*from_restart=*/false);
        m_implicit_solver->CreateParticleAttributes();
    }

    mypc->AllocData();
    mypc->InitData();

    InitPML();
    ExecutePythonCallback("allocdata");
}
```

这里的顺序很重要:

1. AmrCore::InitFromScratch(time) 驱动 AMReX 创建 AMR level; 它会回调 WarpX::MakeNewLevelFromScratch(), 后者对每个 level 依次调用 AllocLevelData(lev, ...) 和 InitLevelData(lev, time)。因此 AllocLevelData 是这个回调链的一部分, 不是 InitFromScratch() 顶层直接调用的无参步骤。
2. 若启用了 implicit solver, Define() 与 CreateParticleAttributes() 在粒子初始化前建立求解器状态和所需粒子属性。
3. mypc->AllocData() 为粒子容器准备数据结构, mypc->InitData() 创建初始粒子。
4. InitPML() 初始化吸收边界数据结构; 随后 allocdata Python callback 才获得已经分配完的初始化对象。

因此, species 初始化发生在 field/level 数据结构已经存在之后, 但在正式时间推进之前。

粒子容器入口:

源码文件: Source/Particles/PhysicalParticleContainer.cpp

函数: PhysicalParticleContainer::InitData()

```
void PhysicalParticleContainer::InitData ()
{
    AddParticles(0); // Note - add on level 0
    Redistribute(); // We then redistribute
}
```

这两行是后续所有粒子创建逻辑的入口。AddParticles(0) 负责生成初始粒子, Redistribute() 负责把粒子分配到正确 tile, 并清理 invalid 粒子。

4.5 3A.5 外部场初始化: grid field 与 particle external field

WarpX 支持两类外部场:

- grid external field: 外部场先放在 Efield_fp_external/Bfield_fp_external 网格 MultiFab 上, 随后由 AddExternalFields() 叠加到初始 E/B 网格场并施加 field boundary;
- particle external field: 外部场保存在 E/B_external_particle_field 中, 在 particle gather 时参与粒子受力, 并不经 AddExternalFields() 写入主 E/B 场。

外部场初始化的关键是: 外部场可以是常量、parser 函数、openPMD 文件或 Python 回调。读者应避免把“外部场”理解成单一数组。

源码文件: Source/Initialization/WarpXInitData.cpp

函数: WarpX::LoadExternalFields(int lev)

```
if (grid_B_from_file || grid_E_from_file ||
    particle_B_from_file || particle_E_from_file) {
    WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
        WarpX::do_moving_window == 0,
        "External fields from file are not compatible with the moving
        ↪ window.");
}
```

这里的布尔名是为阅读压缩后的语义名: 源码中展开检查 B_ext_grid_type/E_ext_grid_type 与 m_B_ext_particle_s/m_E_ext_particle_s。这个断言覆盖 **grid** 和 **particle** 两类 file-driven external field: moving window 移动后, 文件场的 MultiFab 没有自动重读/平移机制, 故当前实现直接拒绝这个组合。

断言之后, LoadExternalFields() 分三步处理数据: parser 或文件填充 grid external field; 在 finest level 调用 loadExternalFields Python callback; 再按 metadata 的每张 map 填充 particle external field。不要把这三步压成一个无参的“读文件函数”: 实际的 ReadExternalFieldFromFile(path, MultiFab*, field, component, map_index) 总是明确写出目标场与分量。

外部场的物理约束是: 如果读入的是 B 或矢量 A, 数值上还需要检查离散散度误差; 这就是后面 projection divergence cleaner 的用途。

4.6 3A.6 species 初始化: PlasmaInjector 是参数总容器

PlasmaInjector 的职责不是推进粒子, 而是把输入文件中一个 species 的初始化规则收集成一组可供 kernel 调用的对象。

源码文件: Source/Initialization/PlasmaInjector.cpp

函数: PlasmaInjector::PlasmaInjector()

```
std::string injection_style = "none";
utils::parser::query(pp_species, source_name, "injection_style",
    ↪ injection_style);
std::transform(injection_style.begin(),
               injection_style.end(),
```

```

        injection_style.begin(),
        ::tolower);

num_particles_per_cell_each_dim.assign(3, 0);

if (injection_style == "singleparticle") {
    setupSingleParticle(pp_species);
    return;
} else if (injection_style == "multipleparticles") {
    setupMultipleParticles(pp_species);
    return;
} else if (injection_style == "gaussian_beam") {
    setupGaussianBeam(pp_species);
} else if (injection_style == "nrandompercell") {
    setupNRandomPerCell(pp_species);
} else if (injection_style == "nfluxpercell") {
    setupNFluxPerCell(pp_species);
} else if (injection_style == "nuniformpercell") {
    setupNUniformPerCell(pp_species);
} else if (injection_style == "external_file") {
    setupExternalFile(pp_species);
} else if (injection_style != "none") {
    SpeciesUtils::StringParseAbortMessage("Injection style",
    ↪ injection_style);
}

```

这个分支定义了初始粒子创建的第一层分类：

injection_style	含义	后续创建路径
singleparticle	单个手工粒子	AddNParticles()
multipleparticles	手工粒子列表	AddNParticles()
gaussian_beam	空间高斯束流	AddGaussianBeam()
external_file	openPMD 粒子文件	AddPlasmaFromFile()
nrandompercell/nuni formpercell	体密度注入	AddPlasma()
nfluxpercell	面通量注入	AddPlasmaFlux()

在这一步之后，PlasmaInjector 还会把 host 侧 functor 拷贝到 device 侧，保证后续 GPU kernel 可以调用。

4.7 3A.7 密度和动量分布：从文本参数到 functor

密度解析由 SpeciesUtils::parseDensity() 完成。

源码文件：Source/Utils/SpeciesUtils.cpp

```

if (rho_prof_s == "constant") {
    amrex::Real density = 0;
    utils::parser::getWithParser(pp_species, source_name, "density",
    ↪ density);
    h_inj_rho.reset(new InjectorDensity(
        InjectorDensityConstant*)nullptr, density));
} else if (rho_prof_s == "parse_density_function") {

```

```

std::string str_density_function;
utils::parser::Store_parserString(
    pp_species, source_name, "density_function(x,y,z)",
    ↪ str_density_function);
density_parser = std::make_unique<amrex::Parser>(
    utils::parser::makeParser(str_density_function,{"x","y","z"}));
h_inj_rho.reset(new InjectorDensity(
    (InjectorDensityParser*)nullptr, density_parser->compile<3>()));
} else if (rho_prof_s == "read_from_file") {
    std::string density_file;
    std::string field_name = "density";
    bool distributed = true;
    utils::parser::get(pp_species, source_name,
        "read_density_from_path", density_file);
    utils::parser::query(pp_species, source_name,
        "density_mesh_name", field_name);
    pp_species.query("read_density_distributed", distributed);
    h_inj_rho.reset(new InjectorDensity(
        (InjectorDensityFromFile*)nullptr,
        density_file, field_name, geom, distributed));
}

```

因此 file profile 不只是一个路径：density_mesh_name 选择文件中的 mesh record（默认 density），geom 提供目标几何，read_density_distributed 决定读取布局。复现实验时应把这四项与文件坐标/单位一起检查，而不应只确认路径可打开。

体注入中的宏粒子权重由密度决定：

$$w_p \approx n(\mathbf{x}) \frac{\Delta V}{N_{ppc}}.$$

动量解析由 SpeciesUtils::parseMomentum() 完成。常见分支包括：

- at_rest: $u=0$;
- constant: 固定 $u_x/u_y/u_z$;
- gaussian: 每个方向正态采样;
- gaussianflux: 通量注入专用，法向速度按 $v_n f(v)$ 加权;
- uniform: 每个方向均匀采样;
- maxwell_boltzmann: 非相对论 Maxwellian;
- maxwell_juttner: 相对论热平衡分布;
- parse_momentum_function: 空间解析函数。

InjectorMomentum 的关键工程实现是手写 tagged union。

源码文件：Source/Initialization/InjectorMomentum.H

对象：InjectorMomentum

```

struct InjectorMomentum
{
    enum struct Type {

```

```

        constant,
        gaussian,
        gaussianflux,
        uniform,
        boltzmann,
        juttner,
        parser,
        gaussianparser
    };

    Type type;

    union {
        InjectorMomentumConstant constant;
        InjectorMomentumGaussian gaussian;
        InjectorMomentumGaussianFlux gaussianflux;
        InjectorMomentumUniform uniform;
        InjectorMomentumBoltzmann boltzmann;
        InjectorMomentumJuttner juttner;
        InjectorMomentumParser parser;
        InjectorMomentumGaussianParser gaussianparser;
    };

```

这不是普通虚函数多态，而是 GPU kernel 友好的平铺对象。后续 AddPlasma() 可以在 device 上用 getMomentum() 采样单粒子动量，用 getBulkMomentum() 得到平均漂移速度。

4.8 3A.8 AddParticles(): 按注入类型进入创建函数

源码文件: Source/Particles/ParticleCreation/AddParticles.cpp

函数: PhysicalParticleContainer::AddParticles(int lev)

```

void
PhysicalParticleContainer::AddParticles (int lev)
{
    ABLASTR_PROFILE("PhysicalParticleContainer::AddParticles()");

    for (auto const& plasma_injector : plasma_injectors) {
        if (plasma_injector->add_single_particle) {
            if (WarpX::gamma_boost > 1.) {
                MapParticleToBoostedFrame(plasma_injector->single_particle_pos[0],
                    plasma_injector->single_particle_pos[1],
                    plasma_injector->single_particle_pos[2],
                    plasma_injector->single_particle_u[0],
                    plasma_injector->single_particle_u[1],
                    plasma_injector->single_particle_u[2]);
            }
            auto const& pos = plasma_injector->single_particle_pos;
            auto const& u = plasma_injector->single_particle_u;
            const auto weight = plasma_injector->single_particle_weight;
            const auto xp = amrex::Vector<ParticleReal>{pos[0]};
            const auto yp = amrex::Vector<ParticleReal>{pos[1]};
            const auto zp = amrex::Vector<ParticleReal>{pos[2]};
            const auto uxp = amrex::Vector<ParticleReal>{u[0]};

```

```

const auto uyp = amrex::Vector<ParticleReal>{u[1]};
const auto uzp = amrex::Vector<ParticleReal>{u[2]};
const auto attr =
    amrex::Vector<amrex::Vector<ParticleReal>>{{weight}};
const amrex::Vector<amrex::Vector<int>> attr_int;
AddNParticles(lev, 1, xp, yp, zp, uxp, uyp, uzp,
              1, attr, 0, attr_int, 0);
return;
}

```

后半段分派如下:

```

if (plasma_injector->gaussian_beam) {
    AddGaussianBeam(*plasma_injector);
}

if (plasma_injector->external_file) {
    AddPlasmaFromFile(*plasma_injector,
                     plasma_injector->q_tot,
                     plasma_injector->z_shift);
}

if (plasma_injector->doInjection()) {
    AddPlasma(*plasma_injector, lev);
}
}

```

这个函数说明: `PlasmaInjector` 中可能同时保存多种初始化信息, 但最终创建时按 `flag` 调用不同路径。

4.9 3A.9 体注入 `AddPlasma()`: 候选粒子、密度、动量和权重

体注入的核心思想是: 先按 `cell` 和 `num_particles_per_cell` 创建候选粒子, 然后用真实 `density/bounds` 筛掉无效粒子, 并把有效粒子写入 `SoA`。

源码文件: `Source/Particles/ParticleCreation/AddParticles.cpp`

函数: `PhysicalParticleContainer::AddPlasma(...)`

```

// count the number of particles that each cell in overlap_box could add
amrex::Gpu::DeviceVector<amrex::Long> counts(overlap_box.numPts(), 0);
amrex::Gpu::DeviceVector<amrex::Long> offset(overlap_box.numPts());
auto *pcounts = counts.data();
amrex::Box fine_overlap_box; // default Box is NOT ok().
if (refine_injection) {
    fine_overlap_box = overlap_box & amrex::shift(fine_injection_box,
    ↪ -shifted);
}
amrex::ParallelFor(overlap_box, [=] AMREX_GPU_DEVICE (int i, int j, int
    ↪ k) noexcept
{
    const amrex::IntVect iv(AMREX_D_DECL(i, j, k));
    auto lo = getCellCoords(overlap_corner, dx, {0._rt, 0._rt, 0._rt},
    ↪ iv);

```

```

auto hi = getCellCoords(overlap_corner, dx, {1._rt, 1._rt, 1._rt},
    ↪ iv);

lo.z = applyBallisticCorrection(lo, inj_mom, gamma_boost,
    ↪ beta_boost, t);
hi.z = applyBallisticCorrection(hi, inj_mom, gamma_boost,
    ↪ beta_boost, t);

if (inj_pos->overlapsWith(lo, hi))
{
    auto index = overlap_box.index(iv);
    const amrex::Long r = (fine_overlap_box.ok() &&
        ↪ fine_overlap_box.contains(iv)) ?
        (AMREX_D_TERM(rrfac[0],*rrfac[1],*rrfac[2])) : (1);
    pcounts[index] = num_ppc*r;
}

```

`applyBallisticCorrection()` 用 bulk velocity 把 boosted-frame 位置反推到 lab-frame 初始面。其公式是:

$$z_{0,lab} = \gamma_b [z_{boost}(1 - \beta_b \beta_z) - ct_{boost}(\beta_z - \beta_b)].$$

随后用 prefix scan 得到写入 offset:

```

const auto max_new_particles =
    amrex::Scan::ExclusiveSum(
        counts.size(), counts.data(), offset.data());

amrex::Long pid;
{
    pid = ParticleType::NextID();
    ParticleType::NextID(pid+max_new_particles);
}

```

这种“两遍法”避免在 GPU kernel 中动态 push 粒子。

真正填充粒子时, 体注入权重系数为:

```

AMREX_GPU_HOST_DEVICE AMREX_FORCE_INLINE
amrex::Real compute_scale_fac_volume (const amrex::GpuArray<amrex::Real,
    ↪ AMREX_SPACEDIM>& dx,
                                     const amrex::Long pcount) {
    using namespace amrex::literals;
    return (pcount != 0) ? AMREX_D_TERM(dx[0],*dx[1],*dx[2])/pcount :
        ↪ 0.0_rt;
}

```

即

$$w_p = n(\mathbf{x}) \frac{\Delta V}{N_{ppc}}.$$

在 boosted-frame 分支中, 密度和纵向动量做 Lorentz 变换:

```

dens = gamma_boost * dens * ( 1.0_rt - beta_boost*betaz_lab );
u.z = gamma_boost * ( u.z -beta_boost*gamma_lab );

```


对应:

$$n' = \gamma_b n (1 - \beta_b \beta_z), \quad u'_z = \gamma_b (u_z - \beta_b \gamma).$$

最后写入粒子 SoA:

```
u.x *= PhysConst::c;
u.y *= PhysConst::c;
u.z *= PhysConst::c;

amrex::Real weight = dens;
weight *= scale_fac;

pa[PIdx::w][ip] = weight;
pa[PIdx::ux][ip] = u.x;
pa[PIdx::uy][ip] = u.y;
pa[PIdx::uz][ip] = u.z;
```

因此 InjectorMomentum 返回的 u 是无量纲 $\gamma \beta$, 粒子数组中存的是 γv 。

4.10 3A.10 Gaussian beam: 显式束流列表注入

Gaussian beam 不使用 InjectorDensity, 而是在 IO rank 上显式随机生成粒子列表。

源码文件: Source/Particles/ParticleCreation/AddParticles.cpp

函数: PhysicalParticleContainer::AddGaussianBeam(...)

```
if (ParallelDescriptor::IOProcessor()) {
    // If do_symmetrize, create either 4x or 8x fewer particles, and
    // Replicate each particle either 4 times (x,y) (-x,y) (x,-y) (-x,-y)
    // or 8 times, additionally (y,x), (-y,x), (y,-x), (-y,-x)
    if (do_symmetrize){
        npart /= symmetrization_order;
    }
    // compute the weight from N_tot if the user specified npart_real =
    // N_tot
    // compute the weight from q_tot if the user specified q_tot
    // note that npart is the number of macroparticles
    const amrex::Real weight_3d = (N_tot > 0._rt) ? (N_tot / npart) :
    (q_tot / (npart*charge));
```

权重来自总真实粒子数或总电荷:

$$w_{3d} = \{N_{\text{tot}}/N_p, Q_{\text{tot}}/(N_p q)\}.$$

随后按高斯分布采样空间位置:

```
#if defined(WARPX_DIM_3D) || defined(WARPX_DIM_RZ)
    const amrex::Real weight = weight_3d;
    amrex::Real x = amrex::RandomNormal(x_m, x_rms);
    amrex::Real y = amrex::RandomNormal(y_m, y_rms);
    amrex::Real z = amrex::RandomNormal(z_m, z_rms);
```

```
#elif defined(WARPX_DIM_XZ)
    const amrex::Real weight = weight_3d/y_rms;
    amrex::Real x = amrex::RandomNormal(x_m, x_rms);
    constexpr amrex::Real y = 0._prt;
    amrex::Real z = amrex::RandomNormal(z_m, z_rms);
```

如果设置 focal_distance, 代码用弹道近似从焦平面束斑反推出初始位置。核心公式是:

$$t = \frac{(\mathbf{x}_f - \mathbf{x}) \cdot \mathbf{n}}{\mathbf{v} \cdot \mathbf{n}}, \quad \mathbf{x} \leftarrow \mathbf{x} - \mathbf{v}_\perp t.$$

同一函数在 focal-plane 分支中计算交叉时刻并反推横向初始位置:

```
// Compute the time at which the particle will cross the focal plane
const amrex::Real v_dot_n = v_x * n_x + v_y * n_y + v_z * n_z;
const amrex::Real t = ((x_f-x)*n_x + (y_f-y)*n_y + (z_f-z)*n_z) /
    ↪ v_dot_n;

// Displace particles in the direction orthogonal to the beam bulk
    ↪ momentum
// i.e. orthogonal to (n_x, n_y, n_z)
#if defined(WARPX_DIM_3D) || defined(WARPX_DIM_RZ)
x = x - (v_x - v_dot_n*n_x) * t;
y = y - (v_y - v_dot_n*n_y) * t;
z = z - (v_z - v_dot_n*n_z) * t;
```

如果设置 symmetrization, 代码为每个样本生成 4 或 8 个镜像粒子, 并把权重除以阶数。这降低横向低阶统计噪声。

4.11 3A.11 openPMD 粒子文件: 文件粒子列表注入

external_file 路径在构造期先打开 openPMD 文件, 读取可选 charge/mass。

源码文件: Source/Initialization/PlasmaInjector.cpp

函数: PlasmaInjector::setupExternalFile()

```
void PlasmaInjector::setupExternalFile (amrex::ParmParse const&
    ↪ pp_species)
{
#ifdef WARPX_USE_OPENPMD
    WARPX_ABORT_WITH_MESSAGE(
        "WarpX has to be compiled with USE_OPENPMD=TRUE to be able"
        " to read the external openPMD file with species data");
#endif
    external_file = true;
    std::string str_injection_file;
    utils::parser::get(pp_species, source_name, "injection_file",
        ↪ str_injection_file);
    // optional parameters
    utils::parser::queryWithParser(pp_species, source_name, "q_tot",
        ↪ q_tot);
    utils::parser::queryWithParser(pp_species, source_name,
        ↪ "z_shift", z_shift);
```

质量和电荷优先级是：

```
input charge/mass > input species_type > openPMD charge/mass record
```

真正读入粒子时，先略去只负责几何维度开关的预处理，再看每个已存在坐标的单位组合：

- 源码文件：Source/Particles/ParticleCreation/AddParticles.cpp
- 函数：PhysicalParticleContainer::AddPlasmaFromFile(...)

```
for (auto i = decltype(npart){0}; i<npart; ++i){
    amrex::ParticleReal const weight = ptr_w.get()[i]*w_unit;

    const auto x = ptr_x.get()[i] * position_unit_x
                  + ptr_offset_x.get()[i] * position_offset_unit_x;
    const auto y = ptr_y.get()[i] * position_unit_y
                  + ptr_offset_y.get()[i] * position_offset_unit_y;
    const auto z = ptr_z.get()[i] * position_unit_z
                  + ptr_offset_z.get()[i] * position_offset_unit_z
                  + z_shift;
```

openPMD 的 position 和 positionOffset 都乘以各自 unitSI, z_shift 是 WarpX 额外偏移。

动量换算：

```
if (plasma_injector.insideBounds(x, y, z)) {
    // The normalized momentum is u = p / m = gamma beta c
    // with m = m_e for photons, m the particle mass otherwise.
    amrex::ParticleReal const mass_eff = (m_mass > 0.0_prt) ? m_mass :
    ↪ PhysConst::m_e;
    amrex::ParticleReal const ux =
    ↪ ptr_ux.get()[i]*momentum_unit_x/mass_eff;
    amrex::ParticleReal const uz =
    ↪ ptr_uz.get()[i]*momentum_unit_z/mass_eff;
    amrex::ParticleReal uy = 0.0_prt;
    if (ps["momentum"].contains("y")) {
        uy = ptr_uy.get()[i]*momentum_unit_y/mass_eff;
    }
}
```

openPMD 文件中的 momentum 是物理动量 p 。除以质量得到 $p/m=\gamma v$ ，这正是 WarpX 粒子数组的 ux/uy/uz 量纲。文件中的 weighting 直接成为宏粒子权重；q_tot 只产生 warning，不会重标定权重。

4.12 3A.12 Projection divergence cleaning: 外部 A/B 场的初始约束修正

如果外部加载的 B 或矢量 A 在离散网格上不满足散度约束，WarpX 可用 projection 方法清理。

数学上, 给定向量场 $\mathbf{F}$, 构造:

$$\mathbf{F}' = \mathbf{F} + \nabla_h \phi, \quad \nabla_h \cdot \mathbf{F}' = 0.$$

于是

$$\nabla_h^2 \phi = -\nabla_h \cdot \mathbf{F}.$$

源码文件:

Source/Initialization/DivCleaner/ProjectionDivCleaner.cpp

函数:

ProjectionDivCleaner::setSourceFromField()

```
WarpX::ComputeDivB(
    *m_source[ilev],
    0,
    {&Bx, &By, &Bz},
    WarpX::CellSize(0)
);

m_source[ilev]->mult(-1._rt);
```

然后 ProjectionDivCleaner::solve() 用 AMReX MLMG 解 Poisson 方程。

配置文件: Source/Initialization/DivCleaner/ProjectionDivCleaner.H

```
amrex::MLMG mlmg(linop);
mlmg.setMaxIter(m_max_iter);
mlmg.setMaxFmgIter(m_max_fmg_iter);
mlmg.setBottomSolver(m_bottom_solver);
mlmg.setVerbose(m_verbose);
mlmg.setBottomVerbose(m_bottom_verbose);
mlmg.setConvergenceNormType(amrex::MLMGNormType::greater);
mlmg.solve({m_solution[lev].get()}, {m_source[lev].get()}, m_rtol,
    ↪ m_atol);
```

最后修正场:

```
Bx_arr(i,j,k) += T::DownwardDx(sol_arr, coefs_x, n_coefs_x, i, j, k);
By_arr(i,j,k) += T::DownwardDy(sol_arr, coefs_y, n_coefs_y, i, j, k);
Bz_arr(i,j,k) += T::DownwardDz(sol_arr, coefs_z, n_coefs_z, i, j, k);
```

这不是演化阶段的 warpx.do_dive_cleaning/do_divb_cleaning。后者是 Maxwell solver 时间推进中的清理变量或修正方程; 本节讲的是初始化或外部场加载后的 Poisson projection。

4.12.1 Laser antenna 与 profile 分派: laser 初始化并不走 PlasmaInjector

species 初始化走的是 PlasmaInjector → Ad-
dPlasma/AddGaussianBeam/AddPlasmaFromFile 这一条链; laser 初始化则完全不同。它的入口是:

- lasers.names
- LaserParticleContainer
- Laser/LaserProfiles.*

LaserParticleContainer 构造函数先统一读取天线几何和公共物理参数:

```
utils::parser::getArrWithParser(pp_laser_name, "position", m_position);
utils::parser::getArrWithParser(pp_laser_name, "direction", m_nvec);
utils::parser::getArrWithParser(pp_laser_name, "polarization", m_p_X);
utils::parser::getWithParser(pp_laser_name, "wavelength", m_wavelength);
```

然后要求 $e_{\max}$ 和 a_0 二选一:

```
const bool e_max_is_specified =
    utils::parser::queryWithParser(pp_laser_name, "e_max", m_e_max);
Real a0;
const bool a0_is_specified =
    utils::parser::queryWithParser(pp_laser_name, "a0", a0);
...
AMREX_ALWAYS_ASSERT_WITH_MESSAGE(
    e_max_is_specified ^ a0_is_specified,
    "Exactly one of e_max or a0 must be specified for the laser.\n");
```

如果给的是 a_0 , WarpX 会立即按

$$E_{\max} = \frac{m_e \omega c}{q_e} a_0$$

换算成真实场强 $e_{\max}$ 。这说明在 profile 实现层, a_0 已经不再存在, 剩下的只是规范化后的公共参数。

profile 类型分派也不是一串手写 if-else, 而是 LaserProfiles.H 中的工厂字典:

```
laser_profiles_dictionary =
{
    {"gaussian",
     [] () {return std::make_unique<GaussianLaserProfile>();} },
    {"parse_field_function",
     [] () {return std::make_unique<FieldFunctionLaserProfile>();} },
    {"from_file",
     [] () {return std::make_unique<FromFileLaserProfile>();} }
};
```

构造函数把 profile 字符串转成小写后直接做字典查找, 再调用统一接口:

```
m_up_laser_profile = laser_profiles_dictionary.at(laser_type_s());
...
m_up_laser_profile->init(pp_laser_name, common_params);
```

所以:

1. gaussian 走解析包络与聚焦/STC/chirp 公式；
2. parse_field_function 直接把 field_function(X,Y,t) 编译成 parser；
3. from_file 则走 lasy_file_name 或 binary_file_name，并用 time_chunk_size / delay 做时间分块读入。

更重要的是，laser pulse 不是直接“写入一个初始场数组”，而是通过人工天线粒子实现。LaserParticleContainer.H 的类注释写得很明确：这些粒子均匀分布在一个平面上，按预设位移沉积电流 J，再由 Maxwell solver 在网格上生成真正的激光场。因此 LaserParticleContainer 需要 current deposition，但不走普通 FieldGather，它也正因如此直接继承 WarpXParticleContainer，而不是 PhysicalParticleContainer。

continuous injection 对 laser 还有额外几何约束：

```
AMREX_ALWAYS_ASSERT_WITH_MESSAGE(
    "...",
    "do_continuous_injection for laser particle only works"
    " if moving window direction and laser propagation direction are the
    ↪ same");
```

如果叠加 boosted frame，目前还进一步要求 boost 方向是 z。因此 laser 的 do_continuous_injection 虽然和普通 species 同名，但它的可用组合更窄，实际上是“moving-window / boosted-frame 下天线何时进入域内”的控制开关。

这还只是 laser 初始化链的入口。真正运行起来后，WarpX 并不会把解析式或文件 profile 直接写进 E/Bfield_fp。它先把 profile 解释成“天线平面上每个人工粒子的目标发射振幅”，再通过标准 J/rho 沉积把激光交给 Maxwell solver。

GaussianLaserProfile::init() 继续在公共参数外读取：

```
utils::parser::getWithParser(ppl, "profile_waist", m_params.waist);
utils::parser::getWithParser(ppl, "profile_duration",
    ↪ m_params.duration);
utils::parser::getWithParser(ppl, "profile_t_peak", m_params.t_peak);
utils::parser::getWithParser(ppl, "profile_focal_distance",
    ↪ m_params.focal_distance);
utils::parser::queryWithParser(ppl, "zeta", m_params.zeta);
utils::parser::queryWithParser(ppl, "beta", m_params.beta);
utils::parser::queryWithParser(ppl, "phi2", m_params.phi2);
utils::parser::queryWithParser(ppl, "phi0", m_params.phi0);
```

因此当前 gaussian profile 并不是“只有纵向和横向高斯包络”的最简形式，而是已经直接包含：

- 聚焦距离 profile_focal_distance
- carrier-envelope phase phi0
- spatial chirp zeta
- angular dispersion beta
- temporal chirp phi2

WarpX 随后还会把 `stc_direction` 归一化，并强制要求它与天线法向 `nvec` 正交：

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(std::abs(dp2) < 1.0e-14,
    "stc_direction is not perpendicular to the laser plane vector");
```

这说明 `stc_direction` 的真实语义是“STC 在激光平面内的作用方向”，不是随手附带的一个参考向量。到了 `fill_amplitude()`，WarpX 再显式构造：

- `diffract_factor`
- `inv_complex_waist_2`
- `stretch_factor`

并按 3D/RZ、XZ、1D 分别选不同 `prefactor`。也就是说，当前 Gaussian 实现已经把 diffraction、Gouy phase、wavefront curvature、spatial chirp、angular dispersion 和 temporal chirp 全都折叠进同一个复 envelope 公式里，而不是后面再给场求解器额外修正。

`from_file` profile 的合同也比“读一个文件”更具体。源码强制要求：

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE (
    lasy_file_name.empty() != binary_file_name.empty(),
    "Exactly one of 'binary_file_name' and 'lasy_file_name' has to be
    ↪ specified");
```

因此 `from_file` 不是同时混用两种后端，而是在：

- `lasy_file_name`
- `binary_file_name`

之间二选一。并且它不是一次性把整份文件全读进来，而是先把 `time_chunk_size` 设成默认全时域，再允许用户把它改小，随后只预读第一块；真正运行时由 `m_up_laser_profile->update(t_lab)` 按需换块。`delay` 也不是写文件时的预处理，而是在 `update()` / `fill_amplitude()` 内统一平移 profile 时间轴。

更关键的是，`LaserParticleContainer::InitData()` 的产物不是“初始化场”，而是人工天线粒子。它先按 finest cell 和天线平面方向计算平面 spacing S_X/S_Y ，再建立实验室坐标与激光平面坐标之间的 Transform/InverseTransform，最后只在注入盒覆盖的区域生成粒子：

- Cartesian / XZ / 1D：每个平面位置生成一对 $+w/-w$ 粒子
- RZ：围绕轴向展开成 spokes，并用 $2\pi r/n_spokes$ 修正权重

到了 `LaserParticleContainer::Evolve()`，运行时主链是：

1. 若在 boosted frame，先把数值时间 `t` 转回实验室系 `t_lab`
2. `m_up_laser_profile->update(t_lab)`，必要时推进 `from_file` 的时间块缓存

3. `calculate_laser_plane_coordinates(...)`, 把真实粒子位置映回激光平面坐标
4. `fill_amplitude(...)`, 为每个天线粒子求目标 E 振幅
5. `update_laser_particle(...)`, 把振幅变成粒子动量和位置
6. `DepositCurrent() / DepositCharge()`, 把人工天线粒子写入 `current_fp/rho_fp`, 必要时也写入 `coarse-fine current_buf/rho_buf`

所以 WarpX 的 laser antenna 不是“边界直接给定场”，而是“人为构造一层发射粒子，通过普通沉积链把 profile 写成 J，再让 Maxwell solver 自己生成传播场”。这也解释了为什么 laser 在 mesh refinement 下照样要走 `fp / buf` 分流，以及为什么 `lasers.deposit_on_main_grid` 其实属于 AMR/沉积合同，而不只是一个 laser 小选项。

最后，laser 的 continuous injection 也应理解得更准确一点。`ContinuousInjection()` 检查的是更新后的 `m_updated_position` 是否第一次进入当前 `injection_box`；一旦进入，就调用一次 `InitData()` 生成那片天线粒子，之后再由普通 `Evolve()` 周期性更新它们的发射振幅。它不是每步“重新生成整束激光”，而是在 moving-window / boosted-frame 下决定“天线什么时候进入域内并开始工作”。

`parse_field_function` 这条分支也需要单独说明，因为它和前两类 profile 的数学合同并不一样。官方参数文档把它定义成：

```
<laser_name>.field_function(X,Y,t)
```

这里给出的不是包络，而是完整电场本身；X/Y 是垂直于激光传播方向的平面坐标，而不是固定的仿真坐标轴。`FieldFunctionLaserProfile::init()` 在源码里只做两件事：

```
utils::parser::Store_parserString(
    ppl, "field_function(X,Y,t)", m_params.field_function);
m_parser =
    ↪ utils::parser::makeParser(m_params.field_function,{"X","Y","t"});
```

随后 `fill_amplitude()` 也没有再加任何 envelope、phase 或 geometry 修正，而是直接逐粒子求值：

```
auto parser = m_parser.compile<3>();
amrex::ParallelFor(np, [=] AMREX_GPU_DEVICE (int i) noexcept
{
    amplitude[i] = parser(Xp[i], Yp[i], t);
});
```

这说明 `parse_field_function` 的 profile 层是三类实现里最薄的一层：用户写什么场函数，天线平面上就得到什么目标场值；后续所有“把目标场值变成可沉积粒子运动”的责任，都落在 `LaserParticleContainer` 的更新 kernel 上。

这几个 kernel 里，`calculate_laser_plane_coordinates(...)` 负责把真实粒子位置减去天线参考点后，投影回 `m_u_X/m_u_Y` 基底，因此 profile 接口始终统一在激光平面坐标上。`ComputeWeightMobility()` 则先用固定的峰值速度上限 `eps = 0.05` 设定：

```
m_mobility = eps/m_e_max;
m_weight = PhysConst::epsilon_0 / m_mobility;
m_weight *= AMREX_D_TERM(1._rt, * Sx, * Sy);
```

也就是说，WarpX 不是先任意给天线粒子权重，再让速度自己长出来；而是先要求峰值场下粒子速度不超过 $0.05c$ ，再反推单粒子权重。到了 `update_laser_particle()`，WarpX 再把 `amplitude[i]` 变成：

1. 沿主偏振方向 p_X 的速度
2. boosted-frame 下额外减去沿传播方向 $nvec$ 的平移速度
3. 相应的 relativistic momentum $ux/uy/uz$
4. 显式路径的整步位置推进，或 implicit 路径基于 $x_n/y_n/z_n$ 的半步 time-centered 位置推进

因此 laser 的人工天线粒子并不是一个只服务显式 solver 的简单边界 hack。它同样要遵守 implicit particle-centering 合同，并继续进入普通 `DepositCurrent()/DepositCharge()` 主链。

在本书引用的 WarpX 源树中，`parse_field_function` 的最明确真实用例位于：

- 目录：Examples/Tests/particle_absorbing_boundary/
- 输入：inputs_test_1d_particle_absorbing_boundary

这个输入把：

- `laser1.profile = parse_field_function`
- `laser1.field_function(X,Y,t) = ...`

嵌进了吸收边界测试里。但对应 `analysis.py` 检查的是边界附近的负向高速电子是否被抑制，而不是直接检查激光场本身。这意味着 `parse_field_function` 目前是“有真实 regression 入口，但没有独立 field-level 解析断言”的状态，书稿里应把这个验证边界明确写出来。

把 laser 模块整体放回 regression 版图后，还能看到另一个重要事实：不同 laser tests 的证据强度差别很大。Examples/Tests/laser_injection/ 的 1D/2D analysis 会直接比较 Gaussian 注入场的包络和主频；implicit 1D/2D 变体也继续复用同一组 analysis，因此并不只是“implicit 能跑通”的 checksum test。Examples/Tests/laser_injection_from_file/ 则继续给 lasy、legacy binary、boosted-frame 和 RZ thetaMode 文件提供 envelope/frequency 双断言。

但这一组还必须再分出一层 helper / prepare 边界。两个目录里的 `analysis_default_regression.py` 都只是 checksum helper：职责是自动识别 plotfile/openPMD 并按测试目录名调用 `evaluate_checksum(...)`，提供历史输出基线，而不是新增 laser 物理断言。更重要的是，`laser_injection_from_file/` 里那批 `inputs_test_*_prepare.py` 并不是“待分析输入”，而是被 `CMakeLists.txt` 先行注册成 dependency 的外部文件生成阶段：

- 普通 1D/2D/3D/RZ lasy 变体统一先写 `gaussian_laser_3d`

- legacy binary 变体先手工写 `gauss_2d`
- RZ thetaMode 变体先写 `laguerre_laser_RZ`

因此这组 regression 的正确结构不是单段输入，而是：

1. prepare

- 生成外部 laser 文件

2. inject

- WarpX 按 `from_file/binary_file_name/RZ` 路径消费这些文件

3. analysis

- 再对最终包络和主频做强断言

这条边界在解释 `Laser/` 模块时很关键，因为它说明“外部 laser 文件格式合同”本身已经是 active regression 的一部分，而不只是示例配套脚本。

选择激光案例：先匹配问题，再读取输入

激光案例不应按目录数量或测试名称判断强弱。读者首先要说明自己想验证的是哪一层：

1. 天线是否给出了预期的场？选 `laser_injection/`，用 Gaussian 注入场的包络和主频作为比较量；
2. 外部 **profile** 文件是否被正确消费？选 `laser_injection_from_file/`，沿“生成外部文件 -> 注入 -> 包络/频率比较”三段检查；
3. 给定几何和驱动下如何搭建尾场加速？选 `laser_acceleration/`，先确认驱动、等离子体、moving window、边界和 diagnostics，再为场、相位或能谱另行定义 reference；
4. 输出在 **boosted frame** 和实验室系之间是否一致？选 `boosted_diags/`，比较同一 Ez 和粒子采样规则，而不把 writer 对齐误读成激光传播的解析验证。

官方 `laser_acceleration/README.rst` 把它定位为 LWFA 的 lab/boosted-frame 例子，并明确指出：接近圆柱对称时，RZ 可用较低成本捕捉相关物理；非圆激光或强 hosing 等明显非对称问题则需要 3D。于是读这些输入的第一步是选几何假设，不是从输出文件开始。

四个基础输入把这个选择具体化：

- 1D: moving window、连续电子注入、Gaussian laser antenna 与 `FieldProbe`；
- 2D: PML、moving window、细化区域、连续背景电子与 `Gaussian beam`；
- 3D: moving window、openPMD full diagnostics 与自定义粒子属性；
- RZ: `n_rz_azimuthal_modes = 2`、beam/plasma 共存与 species 变量输出。

这四类设置说明如何组装不同维度下的应用问题，却不会自动给出一个共同的尾场幅度、dephasing 或能量增益结论。目录中可直接用于局部比较的脚本只有三类：

- `analysis_1d_fluid_boosted.py`: 将 1D boosted fluid WFA 的 Ez/Jz/rho/Vz 与理论 ODE 解比较；

- `analysis_refined_injection.py`: 比较细化注入时的总粒子数和 refinement edge 前方 ρ 均匀性;
- `analysis_openpmd_rz.py`: 比较 RZ openPMD 的 mesh shape、species ordering 与 `rho_<species>` 的物理中心位置。

其余多数输入采用默认输出回归。它适合发现同一输入的输出是否发生意外变化, 却不能替代场幅、相位、beam loading 或激光衍射的独立比较。`README.rst` 仍将 `Analyze` 保留为 `TODD`, 也正好提醒读者: 应用输入提供的是搭建起点, 而不是已经完成的物理说明。

`Examples/Tests/boosted_diags/analysis.py` 对 `test_3d_laser_acceleration_btd` 检查的是 BTD plotfile 与 BTD openPMD 的 E_z 逐点一致, 以及 `random_fraction` 粒子子采样是否生效。它验证的是诊断重建和采样语义; 不能由此推出激光包络、尾场相位或加速器性能。

因此, 阅读激光相关输入时应保持一条清楚的证据链:

问题与几何假设

- > 选择注入、文件 `profile`、LWFA 或 BTD 案例
- > 明确 `producer`、输出和独立 `reference`
- > 只对该 `reference` 覆盖的 `observable` 下结论

这样, `laser_injection`、`from_file`、`parse_field_function`、`laser_acceleration` 与 BTD 不再是一张测试清单, 而是回答不同问题的五类案例入口。

再往运行态交界看一层, `laser` 初始化还必须和 `moving window`、`boosted frame`、`continuous injection`、`external fields` 的更新合同一起理解。`WarpX::MoveWindow()` 在真正平移网格前, 会先做三件互不等价的更新:

```
moving_window_x += ...;
::UpdateInjectionPosition(*mypc, gamma_boost, beta_boost,
    ↪ boost_direction, moving_window_dir, dt[0]);
mypc->UpdateAntennaPosition(dt[0]);
```

这里:

1. `moving_window_x` 是窗口几何本身的位置;
2. `UpdateInjectionPosition(...)` 更新普通 species 的 `m_current_injection_position`;
3. `UpdateAntennaPosition(dt)` 更新 laser antenna 的 `m_updated_position`。

普通 species 的连续注入位置来自 `PlasmaInjector` 的 bulk momentum, 再换成速度并在 `boosted frame` 下做洛伦兹变换; laser 则完全不走 `PlasmaInjector`, 只在 `do_continuous_injection=1` 且 `gamma_boost>1` 时按 boost velocity 平移天线平面。因此两者虽然都叫 `continuous injection`, 但运行态位置更新机制不同。

两者的 `ContinuousInjection()` 语义也不同。普通物理粒子是在 `moving window` 新扫进来的 `level-0 particleBox` 中反复调用 `AddPlasma(...)`; laser 则只在 `m_updated_position` 第一次进入当前 `injection_box` 时调用一次 `InitData()`, 之后靠已有人工天线粒子在 `Evolve()` 中持续更新并沉积 J/ρ 。AMR 下这两个尺度也不同: species 的

runtime 注入盒按 level-0 cell 对齐，而 `LaserParticleContainer::InitData()` 仍然按 `maxLevel()` 的 finest spacing 建立天线粒子。

external field 的 moving-window 合同也在这里锁死。`LoadExternalFields()` 对 `B/E_ext_grid_type == read_from_file` 以及 particle external field 的 `read_from_file` 都直接断言：

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    WarpX::do_moving_window == 0,
    "External fields from file are not compatible with the moving
    ↪ window." );
```

原因不是 openPMD 不能读，而是 `MoveWindow()` 在新进入的 cells 上只能通过 `shiftMF(...)` 用 constant 或 parser 两种方式重建主场背景：

```
srcfab(i,j,k,n) = external_field;
srcfab(i,j,k,n) = field_parser(x,y,z);
```

因此 constant/parser 外场可以跟着 moving window 继续生成，而 `read_from_file` 缺少“窗口每推进一次就按新的 physical coordinates 增量重读”的实现，所以被源码显式禁止。读者若研究文件外场，应选择固定窗口的 `load_external_field*` 案例；若问题是激光与 moving window 的交界，则应转向 `laser_acceleration_boosted`、`refined_injection` 或 `subcycling_mr`，再为目标 observable 定义独立比较。

激光在应用输入中的四种角色

同一个 Gaussian laser 在应用输入中可能是驱动、结果量、AMR 对象或表面等离子体的起点。首先分清这个角色，才能选择正确的 observable。

1. 驱动固体靶：**laser_ion**。输入把 Gaussian laser、solid-density target、full/time-averaged diagnostics、ParticleHistogram、FieldProbe 与 ParticleHistogram2D 放到同一场景。`analysis_test_laser_ion.py` 直接比较的是 `diagInst` 最后 5 个瞬时 `Ez` 的平均与 `diagTimeAvg` 的同一点值。因此它验证 time-averaged diagnostics 的定义；并不由此给出 TNSA cutoff energy、RPA threshold 或离子转换效率。
2. 把辐射当作结果量：**free_electron_laser**。它没有 `lasers.names`，而以刚性注入电子/正电子束、boosted frame、moving window 和 undulator `B_y(z)` 产生辐射。`analysis_fel.py` 用 lab-frame 与 boosted-frame diagnostics 拟合 gain length，并以 FFT 比较 radiation wavelength。这里要解释的是束流辐射，而不是天线注入。
3. 检查细化区域中的天线：**laser_on_fine**。该输入的关键选择是 `max_level = 1`、`fine_tag_lo/hi`、`laser1.prob_lo/prob_hi` 与 PML。默认输出回归只能告诉读者这些 AMR/求解器设置下输出没有意外变化；它不是传播、衍射或靶相互作用的独立物理比较。
4. 建立过密靶表面等离子体：**plasma_mirror**。它组合了 Gaussian laser、solid-density target、前后指数梯度、PML、field filter 和双 species 靶。该输入是研究反射率或高次谐波前的搭建起点；没有独立比较量时，不能把它称作这两类量的 benchmark。

第二项的实现尤其容易被误读。`free_electron_laser` 由 `RigidInjectedParticleContainer`、`particles.B_ext_particle_init_style = parse_B_ext_particle_function` 与 BackTransformed diagnostics 组成。`zinject`

`t_plane` 和 `rigid_advance` 决定注入面前束团如何刚体传播；`undulator` 场不写入主场 `Bfield_fp`，而是在 `gather` 时以 `particle external field` 的 `B_y(z)` 提供。因此它的解释链是“刚性束流 -> 粒子背景场 -> BTM 恢复实验室系 -> `gain length/wavelength`”，不是“激光天线 -> 场注入”。

这条链的三个比较层也不能混用：`analysis_fel.py` 比较 FEL 的 `gain length` 和 `wavelength`；`rigid_injection` 检查刚性传播以及 `plotfile/openPMD` 中的束团状态；`boosted_diags` 检查 BTM writer 对齐和 `random_fraction` 子采样。每一层只为相应的 `producer/consumer` 接口提供证据。

最后，不要把 `laser_ion` 当作“所有多物理同时开启”的总基准。它能作为 `laser-target` 骨架，`field ionization`、`collisions` 与 `QED` 可从此分叉；但这些模块分别在 `InitIonizationModule()`、`CollisionHandler` 与 `InitQED()` 接入，拥有不同的输入前提和推进时序。研究相应机制时，应到各自的 `field_ionization/`、`collision/` 或 `qed/` 案例定义独立 `observable`，而不是把 `time-averaged Ez` 的比较扩大成多物理正确性证明。

4.13 3A.13 初始化验证入口：哪些 regressions 真正在兜底

前面的 3A.1-3A.12 讲的是“源码如何初始化”；若没有可执行 `regression` 对照，这些讲解很容易停留在静态阅读层。WarpX 对 `Initialization` 的验证并没有集中在一个目录里，而是分散在几组物理 `test` 中。

读这张验证地图时，始终先问四个问题：输入创建了什么初态？比较的 **observable** 是什么？**analysis** 真正断言了什么？它没有覆盖哪条分支？目录名、能否运行和 `checksum` 都不能替代这四个问题的答案。下面按读者最常遇到的初始化任务组织入口；同一条路径若只有 `checksum`，便只能作为回归基线，不能推出解析正确性。

第一组是 `Langmuir`。它通常被当成 `evolve` 基准，但对初始化同样关键，因为它直接覆盖：

- `NUniformPerCell`
- `profile=constant`
- `parse_momentum_function`
- 周期边界下的初始粒子/场一致性

例如 `analysis_1d.py` 的主断言不是 `checksum`，而是把输出 `Ez` 与理论 `Langmuir` 波逐点比较：

```
E_sim = data[("mesh", field)].to_ndarray()[ :, 0, 0]
E_th = get_theoretical_field(field, t0)
max_error = abs(E_sim - E_th).max() / abs(E_th).max()
assert error_rel < tolerance_rel
check_charge_conservation(data)
```

这意味着 `Langmuir` 不只是“时间推进跑通”，而是在验证 `parser` 形式的初始扰动确实经过 `PlasmaInjector`、`SpeciesUtils` 和 `AddPlasma()` 正确落到了粒子和场上。

第二组是 `space_charge_initialization`。它最直接对应 `species.initialize_self_fields = 1` 这条初始化支线。输入文件显式打开：

```
beam.injection_style = "gaussian_beam"
beam.initialize_self_fields = 1
beam.momentum_distribution_type = "at_rest"
```

而 analysis 脚本把输出 Ex/Ey/Ez 与高斯电荷团理论场直接比较。因此这组 test 实际上在硬验证：

1. gaussian_beam 初始粒子云生成正确；
2. InitData() 检测到 has_initialize_self_fields；
3. ComputeSpaceChargeField(reset_fields=false) 在第一个时间步前给出了正确的 Coulomb 场。

第三组是 dive_cleaning。它验证的不是 projection cleaner，而是：

- 初始 Gaussian beam 状态进入演化后，
- warpx.do_dive_cleaning = 1 与 PML 能否把 $\text{div}(\mathbf{E}) - \rho / \epsilon_0$ 误差传播并吸收掉，
- 最终场是否回到理论 Gaussian beam 电场。

第四组是 gaussian_beam / external_file。这里要分两半看：

1. analysis_focusing_beam.py 和 analysis_rotated_beam.py 分别验证 focal_distance、束斑统计、旋转位置和旋转动量；
2. inputs_test_3d_focusing_gaussian_beam_from_openpmd_prepare.py + inputs_test_3d_focusing_gaussian_beam_from_openpmd 则覆盖 external_file openPMD 粒子注入合同，包括 weighting、mass、charge、positionOffset 和 momentum.unit_SI = m_e c。

这一组现在还能再细一层：

- inputs_test_3d_focusing_gaussian_beam_photons 不是新物理 benchmark，而是把同一聚焦束斑统计合同重复到 species_type = photon 路径；
- inputs_test_3d_gaussian_beam_picmi.py 则主要覆盖 PICMI GaussianBunchDistribution 前端到 runtime attributes 的接线，当前主要依赖 checksum，而不是独立理论断言。

这里还要诚实记录一个源码边界：gaussian_beam/CMakeLists.txt 为 native test_3d_focusing_gaussian_beam_from_openpmd 指定了 analysis.py，但该目录没有这个文件。因而这条 native 路径只能说明 prepare -> external_file inject -> diagnostics 的输入与输出接口被覆盖，不能被称为官方的束斑物理强验证。相邻的 PICMI 入口复用 analysis_focusing_beam.py，可以用来学习应比较的束斑统计量；它也不能自动补上 native 入口缺失的 analysis。读者若修改 native 输入，应自行对照粒子数、总权重、横向均方根束斑和纵向切片统计，并预先说明容差来自何处。

第五组是 electrostatic / EB 初始化：

- `effective_potential_electrostatic` 用电子径向密度和解析 `adiabatic expansion` 基准比较, 验证 `effective-potential electrostatic solver`;
- `electrostatic_sphere_eb` 则用 `ChargeOnEB reduced diag` 和 `eb_covered` 场, 验证 `InitEB()`、`Poisson` 边界条件和带导体球的初始势问题。

最后一组是 `projection_div_cleaner`, 它对应 3A.12 的 `Poisson projection`, 而不是演化阶段的 `do_dive_cleaning`。相关 tests 已覆盖:

1. RZ openPMD 文件外场版本;
2. 3D PICMI 文件外场版本;
3. Python callback 版本;
4. 2D 解析外场版本。

这些脚本的共同断言都是: 初始化完成后, 从 `raw staggered Bx_aux/By_aux/Bz_aux` 重建的离散 `divB` 必须足够接近零。

这里也要区分强断言的位置:

- `test_rz_projection_div_cleaner` 的强断言在独立 `analysis.py` 里;
- `test_3d_projection_div_cleaner_picmi`、`test_3d_projection_div_cleaner_callback_picmi` 和 `test_2d_projection_div_cleaner_initial_analytical_field_picmi` 则都把 `divB` 断言直接写在输入脚本尾部, 所以 `CMakeLists.txt` 里虽然 `analysis=OFF`, 但并不等于这些条目只是 `checksum-only`。

再往 `species` 入口侧补一组, `Examples/Tests/flux_injection/` 是直接锚定 `setu pNFluxPerCell()` 的 `regression` 家族。这组 tests 分三条:

1. `analysis_flux_injection_3d.py`
 - 对 3D `NFluxPerCell` 场景同时检查总发射量、法向 `Gaussian-flux` 分布和切向 `Gaussian` 分布;
2. `analysis_flux_injection_rz.py`
 - 对 `flux_normal_axis = t` 的 RZ 连续注入检查粒子始终停留在预期 Larmor 半径带, 并保持正确总通量;
3. `analysis_flux_injection_from_eb.py`
 - 对 `inject_from_embedded_boundary = 1` 的 2D/3D/RZ 变体检查发射总数、法向/切向速度统计, 以及粒子不会落入 EB 内部。

因此 `flux_injection` 的意义不是普通 emitter 示例, 而是 `NFluxPerCell`、`Gaussian-flux rejection sampling` 和 `embedded-boundary surface emission` 这三条运行态合同的直接验证入口。

第一组是 `initial_distribution`。它不是普通 smoke test, 而是一组多 `species`、多分布的综合强基准: 同一输入里同时覆盖

- gaussian
- maxwell_boltzmann
- maxwell_juttner
- gaussian_beam
- parser 温度
- parser bulk velocity
- uniform
- parser-Gaussian 动量统计

analysis 脚本把 reduced histogram、束斑统计和解析分布逐条对照。这意味着它真正验证的是 PlasmaInjector、SpeciesUtils 和 momentum-dispatch 层的 built-in / parser 初始化合同，而不只是“粒子能被建出来”。

由于初始化使用随机采样，checksum 的默认容差不应被理解为跨机器、跨 MPI 布局都逐位相同的物理合同。复现实验应优先比较 analysis 脚本定义的直方图、均值、方差、束斑等统计量，并在报告中写清样本量、随机数种子（若可控）、并行布局和容差；只有这些条件一致时，checksum 才能作为补充证据。

4.13.1 3A.13.1 初始化验证卡：分布统计和初始自场是两份合同

初态常被一句“束流已经初始化”概括，但这至少混合了两件不同的事：粒子的位置、权重和动量是否来自预期分布；以及这些粒子是否已经在第一步前建立了应有的自场。前者即使完全正确，也不推出 Poisson 或 space-charge 初始化正确；后者即使场形状正确，也不能替代对粒子分布和总电荷的检查。读者应把它们拆成两条 producer -> observable -> consumer 链。

合同 A：粒子分布。initial_distribution 的输入设定 max_step = 0，并以 ParticleHistogram reduced diagnostics 输出各 species 的动量或位置直方图。例如 h1x/h1y/h1z 对 Gaussian species 统计 ux/uy/uz，h4x/h4y/h4z 对 Gaussian beam 统计空间坐标；同一组还写出 beam 的总电荷。CTest 将 producer 接到 analysis.py，该 consumer 读取这些 reduced diagnostics，将 Gaussian、Maxwellian、Maxwell-Juttner 和 beam 的位置统计分别与解析密度比较，并对归一化误差及相对电荷误差断言容差。

这条设计的关键价值在于 max_step = 0：它把比较对象限定为初始化完成后、尚未由时间推进改变的粒子统计。因此它能支持的结论是“给定输入、采样数、诊断定义和分析容差下，粒子初始化分布与指定解析分布相符”。它不能支持“初始空间电荷场已经正确”或“随后任意步数、任意求解器和任意 MPI 布局的动力学都正确”。若读者修改的是 momentum_distribution_type、parser mean/std、束斑 rms、截断或粒子数，应先复用这一类 histogram/charge observable；不要只看 plotfile 是否写出。

合同 B：初始自场。space_charge_initialization 选择静止 Gaussian beam，并显式给出：

```
beam.injection_style = "gaussian_beam"
beam.initialize_self_fields = 1
beam.momentum_distribution_type = "at_rest"
```

这时 producer 不止是粒子生成。WarpX::InitData() 的 fresh-run 分支会遍历 species; 只要发现任一 initialize_self_fields 为真, 便在第一步推进之前调用 ComputeSpaceChargeField(reset_E_field=false, reset_B_field=false)。输入还请求 Full diagnostics 写出 Ex/Ey/Ez, CTest 将 diags/diag1000001 交给 analysis.py。这个 consumer 按二维或三维 Gaussian charge distribution 构造理论场, 再对输出的 Ex/Ey, 以及三维时的 Ez 做逐数组 allclose 容差检查。

因此这条合同能支持的结论是“在该静止 Gaussian-beam、边界、网格、shape、两 rank 注册布局 and 字段比较容差下, 初始化自场与该理论场一致”。它不把 initial_distribution 的 histogram 通过自动升级为场求解验证, 也不覆盖相对论自场、open-boundary FFT、embedded boundary 或后续 pusher 的正确性; 这些问题应分别进入 relativistic_space_charge_initialization、open_bc_poisson_solver、magnetostatic_eb 或 repelling_particles 的 consumer。

实际操作顺序。当一个新输入同时改了束流分布和 initialize_self_fields 时, 先在零步或等价的初态输出上锁定合同 A, 再在明确的初始 field diagnostic 上锁定合同 B; 随后才用轨迹、能量或束斑演化回答“该初场被 pusher 消费后会怎样”。若 A 失败, 先查 injection style、parser、随机采样、权重和 histogram 定义; 若 A 通过而 B 失败, 转查边界、solver、field diagnostic、initialize_self_fields 与初始场求解。两类失败的定位入口不同, 不能合并成“初始化失败”。

第二组是 initial_plasma_profile。这组当前没有独立 analysis.py, 只有 checksum helper, 但输入本身非常明确:

- injection_style = NUniformPerCell
- profile = parse_density_function

并把横向 parabolic channel 与纵向 ramp / plateau / ramp 组合成二维电子密度。所以它更准确地是:

- parse_density_function 抛物型通道初始化的 checksum-only 基线

而不是应继续留在 general / to classify 的未知条目。

再往 initialize_self_fields 这一支补一组, repelling_particles 是一个更小但更干净的两体基准。它只放两个同号 SingleParticle species, 却同时打开:

- electron1.initialize_self_fields = 1
- electron2.initialize_self_fields = 1

analysis 随后从连续 plotfiles 读取两粒子的间距和速度, 并用两体排斥的非相对论能量守恒关系构造理论 $\beta(d)$ 。因此这组 regression 的真实意义不是“一对粒子大概会分开”, 而是:

1. 初始 electrostatic self-field 确实被建立起来;
2. 后续 pusher 对这份初始场的消费是对的;
3. 两者联立后能回到解析两体减速关系。

接着把另外三组容易落单的入口也补上之后, 初始验证图还能再细一层。

第一组是 load_external_field。它不是普通粒子轨道测试, 而是在验证两套初始化合同:

1. grid external field:

- LoadExternalFields()
- ReadExternalFieldFromFile()
- Bfield_fp_external/Efield_fp_external
- AddExternalFields()

2. particle external field:

- Particles/ExternalParticleFields.cpp
- m_B_ext_particle_s / m_E_ext_particle_s
- B_external_particle_field/E_external_particle_field
- GetExternalFields.cpp

analysis_3d.py / analysis_rz.py 通过磁镜中的单粒子最终位置做硬断言，说明这组 regression 同时验证了“初始化写场”和“后续 gather 消费场”的接口契合。时间依赖变体 analysis_time_scaling.py 则不看粒子，而是直接比较两个时刻 plotfile 上 B 分量的缩放比，从而验证 read_fields_*_dependency(t) parser 和多 field map 的时间缩放合同。

这组 family 里还要再区分一层 restart 保真。当前活跃的一条最小入口是：

- test_3d_load_external_field_particle_time_restart
- test_rz_load_external_field_grid_restart
- test_rz_load_external_field_particles_restart

它们都只是：

- 先继承对应非 restart 输入
- 再通过 amr.restart = ../.../chk000150 从中间 checkpoint 恢复
- 然后复用 analysis_default_restart.py 逐字段比较 restart 与非 restart 输出

因此这三条 regression 验证的不是新的外场 physics，而是：

1. read_from_file 的 grid external field 状态能否在 restart 后保持一致；
2. read_from_file / dependency parser 的 particle external field 状态能否在 restart 后保持一致；
3. 初始化阶段构造出的 external-field 寄存器，不会在 checkpoint/restart 边界上丢失或漂移。

第二组是 relativistic_space_charge_initialization。它的输入和普通 space_charge_initialization 一样也打开了：

```
beam.initialize_self_fields = 1
beam.injection_style = "gaussian_beam"
```

但束流动量改成了 relativistic：

```
beam.uz = 100.0
```

因此这组 regression 实际验证的是 `RelativisticExplicitES::ComputeSpaceChargeField()`，而不再只是静止高斯电荷团的 Coulomb 场。analysis 脚本把 Ex 与理论值比较，并检查 By 是否满足相对论束流自场的 $B_y \approx E_x/c$ 结构。这说明 `initialize_self_fields` 在 relativistic solver 分支下对应的是另一份初始化合同。

第三组是 `open_bc_poisson_solver`。它把四个条件绑在一起：

```
boundary.field_lo = open open open
boundary.field_hi = open open open
warpx.do_electrostatic = relativistic
warpx.poisson_solver = fft
electron.initialize_self_fields = 1
```

同时粒子不是 `gaussian_beam`，而是 `parse_density_function + NUniformPerCell`。analysis 脚本用 Basseti-Erskine 公式逐个 z 截面比较 Ex/Ey，因此它验证的不是一般的 electrostatic 初始化，而是：

- open boundary
- relativistic bunch
- FFT Poisson
- 以及可选 `warpx.use_2d_slices_fft_solver = 1`

共同定义的初始 Poisson 解是否正确。

接下来三组补足了文件驱动分布、带嵌入边界的自场，以及 `collocated` 采样的分支；它们与前面的束流注入和 Poisson 例子不共享同一个可观测量，因而应分别阅读。

第一层是 `load_density`。这组 regression 的输入明确使用：

```
electrons.profile = "read_from_file"
electrons.read_density_from_path =
  ↳ "../test*_load_density_prepare/example-density.h5"
electrons.do_continuous_injection = 1
warpx.do_moving_window = 1
```

源码上它直接对应 `SpeciesUtils::parseDensity()` 里 `profile = read_from_file` 分支，把 openPMD density mesh 装进 `InjectorDensityFromFile`，然后交给 `PlasmaInjector` 和连续注入主链消费。analysis 脚本则逐个 iteration 读取 diagnostics 中的 rho，与 prepare 脚本定义的 ramp / parabolic channel profile 比较。因此 `load_density` 验证的不是一般 I/O，而是“file-driven density profile + moving-window 连续注入”这条初始化合同。

第二层是 `magnetostatic_eb`。原生 inputs 文件把：

```
warpx.do_electrostatic = labframe-electromagnetostatic
beam.initialize_self_fields = 1
warpx.eb_implicit_function = "(x**2+y**2-radius**2)"
warpx.eb_potential(x,y,z,t) = "1."
```

绑在一起，而 WarpXInitData.cpp 的 fresh-run 分支会在 ComputeSpaceChargeField(reset_fields) 之后继续调用 ComputeMagnetostaticField()。所以这组 test 验证的是 embedded boundary、边界 potential、初始 self-field 和 magnetostatic solve 在初始化阶段的联动。这里还必须区分两层证据：原生 inputs_test_3d_magnetostatic_eb 目前主要由 checksum 兜底；两个 PICMI 输入文件则在 sim.step() 之后内嵌了解析 E_r/B_z \theta 误差断言，因此它们不是简单的 checksum-only regression。

第三层是 nodal_electrostatic。这组输入把：

```
warpx.do_electrostatic = relativistic
warpx.grid_type = collocated
beam_p.initialize_self_fields = 1
beam_p.do_qed_quantum_sync = 1
```

放在同一条链上。它的 analysis 不直接比较 E/B，而是用 reduced diagnostics 断言 ParticleExtrema_beam_p 给出的最大 chi 极小，且 ParticleNumber 中 photon 数始终为零。也就是说，这组 regression 验证的是 collocated relativistic electrostatic 初始 self-field 没有制造出会假触发 QED 的非物理场，它更准确地是一个“零触发基准”。

这样 nodal_electrostatic、open_bc_poisson_solver 和 relativistic_space_charge_initialization 就不再需要继续共用一个过粗的 electrostatic / Poisson 桶。它们分别对应的是：

- collocated relativistic electrostatic 零触发基准
- open boundary + FFT/sliced FFT 的 relativistic Poisson 初始化
- relativistic Gaussian beam 的初始 self-field

不过还有一组此前仍容易被写得过粗：electrostatic_sphere。它不该和一般 Poisson 条目混成一桶，因为它真正验证的是“一个静止均匀电子球在自身 Coulomb 场下膨胀”这条自场初始化主链。analysis_electrostatic_sphere.py 的第一层断言是解析电场对照：脚本用最终输出时间 t_max 反解球半径 r_end，再构造内外球解析 E(r)，沿坐标轴比较 E_x/E_y/E_z 或 RZ 下的 E_r/E_z 的相对 L2 误差。这意味着它验证的不只是 solver 跑通，而是：

1. 初始电子球几何是否被正确装填；
2. 初始自场是否正确建立；
3. 后续 electrostatic 演化是否仍与解析膨胀解一致。

这组 test 的第二层断言只在 lab-frame 变体上才打开。只有当输入显式要求：

```
warpx.do_electrostatic = labframe
diag2.electron.variables = x y z ux uy uz w phi
```

analysis 才会利用粒子 phi 重建：

- 动能
- 自场势能 $0.5 \sum w q \phi$

并检查初末总能量。也就是说：

- 所有 `electrostatic_sphere` 变体都做解析电场对照；
- 只有写出 `phi` 的 lab-frame 版本额外验证能量账本。

各输入变体的角色也不同：

- `inputs_test_3d_electrostatic_sphere`
 - 3D relativistic electrostatic 自场膨胀基线
- `inputs_test_3d_electrostatic_sphere_lab_frame`
 - lab-frame 自场膨胀 + 能量守恒
- `inputs_test_3d_electrostatic_sphere_lab_frame_mr_emass_10`
 - lab-frame + MR; `electron.mass = 10` 主要是减慢膨胀，便于短步数比较
- `inputs_test_3d_electrostatic_sphere_rel_nodal`
 - `warpx.grid_type = collocated`，验证 collocated electrostatic 布局
- `inputs_test_3d_electrostatic_sphere_adaptive`
 - adaptive-dt 变体；analysis 仍用实际 `t_max` 做强对照
- `inputs_test_rz_electrostatic_sphere`
 - RZ 自场膨胀 + 能量守恒
- `inputs_test_rz_electrostatic_sphere_uniform_weighting`
 - 再额外覆盖 `radial_numpercell_power = 1.` 的 uniform-weighting 粒子装填

另外，这组目录下还有两个容易误读的文件：

- `analysis_default_regression.py` 只是通用 checksum helper；
- `catalyst_pipeline.py` 只是 ParaView Catalyst 的可视化脚本。

因此，`electrostatic_sphere` 在初始化验证图里更准确的位置应单独写成：

1. electrostatic self-field expansion: `electrostatic_sphere*`

还剩另外两组此前也容易被写得过粗，但它们和 `electrostatic_sphere` 不是一类问题。

第一组是 `electrostatic_dirichlet_bc`。这组不是带电粒子自场 benchmark，而是纯粹在验证时变边界势是否真正进入 electrostatic 解。输入直接设置：

```
warpx.do_electrostatic = labframe
boundary.potential_lo_x = 150.0*sin(2*pi*6.78e+06*t)
boundary.potential_hi_x = 450.0*sin(2*pi*13.56e+06*t)
diag1.fields_to_plot = phi
```


`analysis` 并不读取内部粒子量，而是逐个输出时刻取两侧边界上的平均 `phi`，然后与理论正弦函数比较。因此它测的不是一般 `Poisson` 正确性，而是：

- `boundary.potential_lo_x / potential_hi_x`
- time-dependent parser
- electrostatic Dirichlet boundary condition
- diagnostics 中 `phi` 的边界保真

PICMI 变体只是在 `Cartesian2DGrid(..., lower_boundary_conditions=["dirichlet", ...])`、`warpx.potential_lo_x`、`warpx.potential_hi_x` 这一层重复同一件事，所以它更准确的归类是：

1. time-dependent electrostatic boundary driving: `electrostatic_dirichlet_bc*`

第二组是 `effective_potential_electrostatic`。这组当前只有一个 PICMI test，而且它验证的也不是一般意义上的 electrostatic `phi` 场，而是 `warpx.effective_potential=True` 这条 solver 分叉是否能在导体球约束下复现绝热膨胀 benchmark。PICMI 输入脚本同时做了三件关键事：

1. 用 `GaussianBunchDistribution` 初始化电子和离子球团；
2. 加入导体球 `embedded boundary`；
3. 选择：

```
picmi.ElectrostaticSolver(
    method="Multigrid",
    warpx_effective_potential=True,
    warpx_effective_potential_factor=C_EP,
)
```

`analysis` 则从 `sim_parameters.dpk1` 读回参数，构造 Connor et al. 风格的绝热膨胀近似电子密度，再把 openPMD 中的 `rho_electrons` 变成径向密度曲线，与理论值逐个输出时刻比较 RMS 误差。因此它的真实验证对象是：

- effective-potential electrostatic solver
- PICMI front-end
- 导体球约束下的电子密度膨胀近似

更准确的归类应是：

1. effective-potential electrostatic: `effective_potential_electrostatic`

4.14 3A.14 从 Birdsall 3A ES1 到 WarpX: 历史最小程序骨架的现代映射

Birdsall and Langdon 的 3A ES1 是一份很好的历史参照，因为它把一维静电 PIC 的最小程序压缩成一条容易检查的阶段链：

INIT -> SETRH0 -> FIELDS -> SETV -> ACCEL -> MOVE -> HISTRY

这条链可以帮助读者理解“初始化结束后第一步推进究竟从哪里开始”，但不能把旧程序的子程序名直接当成 WarpX 的函数名。历史参照见 Birdsall 与 Langdon 的 **Plasma Physics via Computer Simulation**；现代实现应从 `Source/Initialization/WarpXInitData.cpp`、`Source/Particles/` 和 `Source/FieldSolver/` 的对应职责回查。

3A ES1 阶段	WarpX 的现代映射与不能直接等同的部分
INIT	建立网格、粒子、权重、边界和初始参数，对应 <code>ReadParameters()</code> 、 <code>WarpX::InitData()</code> 、 <code>InitFromScratch()</code> 、 <code>AllocLevelData()</code> 与 <code>mypc->AllocData()</code> ；WarpX 还要处理 AMR、多几何、solver 分支、PML、外部场、restart 和并行布局。
SETRH0	用初始位置形成源，对应 <code>PlasmaInjector/AddParticles</code> 后的初始 rho 构造及 field-register 路径；rho 是否直接写入、重新沉积或被 solver 消费，取决于 geometry、solver 和初始化选项。
FIELDS	从 rho 求势和场，对应 electrostatic solver 的 <code>InitData()</code> 、 <code>ComputeSpaceChargeField()</code> 、初始场填充和 projection cleaning；不能简化成单一 FFT Poisson 路径，EM、PSATD、RZ、EB 与 external field 会改变对象和约束。
SETV	设置热分布和漂移，对应 <code>SpeciesUtils</code> 、 <code>InjectorMomentum</code> 、temperature/velocity functor 与粒子属性创建；WarpX 还可能创建 relativistic、spin、implicit 或 pusher-specific attributes。
ACCEL	用场更新速度，对应 <code>Evolve()</code> 内 particle push 与 gather；历史静电推进不能覆盖现代 EM、Boris/Vay/Higuera-Cary、implicit 和 subcycling。
MOVE	用新速度更新位置，对应 <code>PushParticlesAndDeposit()</code> 的 position update、边界处理和 current deposition；WarpX 还要处理 AMR tile、moving window、particle boundary、suborbit/crossing 和 MPI 交换。

3A ES1 阶段	WarpX 的现代映射与不能直接等同的部分
HISTORY	记录历史量，对应 full/reduced diagnostics、openPMD/plotfile writer 与 reader-side analysis；现代 diagnostics 是独立 writer/schema 合同，不是旧程序中的一个历史数组。

最容易误读的是 SETRH0 → FIELDS → SETV 的顺序。对历史 ES1 来说，它表达的是“先由初始位置形成源，再求场，再给粒子速度”；对 WarpX fresh run，实际初始化链还要先完成 AMReX level 和 field data 分配，并根据 solver、外场和 initialize_self_fields 等条件决定哪些源/场对象被创建。因而更可靠的读法是：Birdsall 骨架提供物理阶段的语义，WarpX 源码决定这些语义在现代对象图和分支条件中的落点。

这条映射也解释了为什么 Langmuir regression 可以同时作为初始化和演化入口：它检查的不是某个名为 SETRH0 的函数，而是初始粒子/密度、初始场、后续推进和最终解析频率之间的组合合同。相反，initial_distribution、space_charge_initialization、load_external_field 与 projection_div_cleaner 分别覆盖粒子分布、初始 self-field、外场装填和初始散度修正的更窄路径。它们共同支撑的是现代 WarpX 初始化链的分层验证，而不是对 3A ES1 原程序做逐行复现。

4.15 3A.15 本章小结：初始化状态怎样进入第一步推进

到 InitData() 结束时，WarpX 已经完成：

```
inputs
-> WarpX::ReadParameters()
-> WarpX::InitData()
-> InitFromScratch or InitFromCheckpoint
-> AMReX levels and WarpX field data
-> external fields
-> species PlasmaInjector
-> AddParticles / AddPlasma / AddGaussianBeam / AddPlasmaFromFile
-> projection div cleaner if needed
-> initial diagnostics
-> Evolve()
```

这个状态才是 PIC 时间推进的初始条件。后续 Evolve()、OneStep()、PushParticlesandDeposit()、field solver 和 diagnostics 都在这个已经离散化、分布式、带权重和边界条件的状态上工作。

初始化到推进的交界由 Evolve()、OneStep() 与 PushParticlesandDeposit() 承接：前者负责外层时间步，后者负责 solver/AMR 分派，最后才把已经初始化的粒子和场送入实际粒子推进与沉积路径。历史 ES1 的阶段名只提供物理职责的参照，不能替代这些现代对象和分支。

进一步学习可沿以下方向展开：

- 把 InitLevelData() 中每一类 field allocation 展开到 root/fieldsolver 章节；
- 把 Gaussian beam 的 emittance/focal distance 公式结合 accelerator beam optics 文献继续推导；
- 把 openPMD 文件格式与 WarpX 单位约定加入诊断/I/O 章节；

- 为不同初始化路径设计可重复的验证案例，并记录它们各自能说明的边界。

4.16 3A.16 练习与最小复现

1. **fresh/restart** 定位题: 沿 `WarpXInitData.cpp` 追踪 `InitData()`, 说明 `ComputeDt()` 为什么只出现在 `fresh-run` 分支, 以及 `restart` 为什么必须进入 `PostRestart()`。
2. 初始化与复现题: 解释 `AmrCore::InitFromScratch()`、`AllocLevelData()`、`mypc->AllocData()`、`mypc->InitData()` 和 `InitPML()` 的先后关系。再在 `Examples/Tests/initial_distribution/` 选择匹配输入, 核对生效输入、首个 `diagnostics` 和 `species` 数, 并说明输入不匹配导致的 `abort` 为什么既不构成初始化通过, 也不构成其反证。

附录 A 符号、时间层与源码变量

本附录服务于正文阅读和源码检索。公式中的符号采用连续模型或离散模型的常用记号；反引号中的名称则优先对应 WarpX 源码、输入文件或 diagnostics 输出中的实际字段。除特别说明外，SI 制单位适用，粒子权重 w_p 表示一个宏粒子代表的真实粒子数。

使用本附录时，先区分三件事：数学符号描述的物理量、WarpX 内部数组实际保存的量、输入文件或 diagnostics 对同一名称采用的单位。它们常常有关联，但不必完全相同；尤其是 ux , uy , uz 不能只按变量名猜测单位。

A.1 A.1 连续模型符号

符号	含义	常见单位或类型
s	species 索引，例如 electron、ion、photon	无量纲整数
$f_s(\mathbf{x}, \mathbf{p}, t)$	species s 的相空间分布函数	依定义而定
q_s, m_s	species 电荷和静质量	C、kg
$\mathbf{x} = (x, y, z)$	物理空间位置	m
$\mathbf{p}$	物理动量	kg m s ⁻¹
$\beta = \mathbf{v}/c$	无量纲速度	无量纲
$\mathbf{u} = \mathbf{p}/m = \gamma \mathbf{v}$	proper velocity; WarpX 粒子容器内部的 ux , uy , uz 以此量保存	m s ⁻¹
$\mathbf{p}/(mc) = \mathbf{u}/c = \gamma \beta$	无量纲归一化动量；多数 species 输入的 u 参数采用这一约定	无量纲
$\mathbf{v}$	粒子速度	m s ⁻¹
γ	Lorentz 因子, $\gamma = (1 - v^2/c^2)^{-1/2}$	无量纲
c	真空光速	m s ⁻¹
$\mathbf{E}, \mathbf{B}$	电场和磁场	V m ⁻¹ 、T
$\rho, \mathbf{J}$	电荷密度和电流密度	C m ⁻³ 、A m ⁻²
ϵ_0, μ_0	真空介电常数和磁导率	SI 常数
w_p	宏粒子权重，代表的真实粒子数	无量纲或按模型定义
S	粒子到网格的空间形函数	按离散归一化定义

A.1.1 u_x, u_y, u_z 的两层约定

对有质量粒子，三种写法的关系是

$$\mathbf{p} = m\mathbf{u}, \quad \mathbf{u} = \gamma\mathbf{v}, \quad \frac{\mathbf{u}}{c} = \frac{\mathbf{p}}{mc} = \gamma\boldsymbol{\beta},$$

并且

$$\gamma = \sqrt{1 + \left|\frac{\mathbf{u}}{c}\right|^2}.$$

这解释了两个表面矛盾、实际一致的源码约定。

1. `Source/Particles/WarpXParticleContainer.H` 将粒子容器中的 u_x, u_y, u_z 说明为 proper velocity，即 $\gamma\mathbf{v}$ ，内部量纲为 m/s。
2. `Docs/source/usage/parameters.rst` 将 `single_particle_u`、`multiple_particles_u*` 和常见 species momentum distribution 的输入定义为 $\gamma\mathbf{v}$ ，即无量纲的 $\gamma\mathbf{v}/c$ 。初始化时会乘以 c ，再写入内部粒子数组；`PlasmaInjector.cpp` 中的 `multiple_particles_ux` 转换正是这一边界的例子。

因此，阅读输入文件时把 $u_z = 10$ 理解为 $\gamma\beta_z = 10$ ；阅读粒子容器或以 SI 单位计算能量的 kernel 时，把内部 u_z 理解为 $\gamma v_z = 10c$ 。诊断 parser、openPMD 记录和不同输出格式还可能按其文档以 $\gamma v/c$ 暴露动量分量；遇到 u_x 时，先确认它来自输入、容器还是输出 metadata，再进行单位换算。

A.2 A.2 网格、时间层与离散量

符号	含义	在程序中的对应
t^n	第 n 个整数时间层	<code>step</code> 、 <code>istep</code>
$t^{n+1/2}$	半时间层，常用于 leapfrog 电流或磁场	$J^{\{n+1/2\}}$ 等时间层语义
Δt	时间步长	<code>dt</code> 、 <code>warpX.const_dt</code> 或 派生时间步长
$\Delta x, \Delta y, \Delta z$	各方向网格间距	<code>amr.n_cell</code> 、几何 <code>cell_size</code>
i, j, k	网格单元或节点索引	AMReX <code>IntVect</code> 分量
ℓ	AMR level 索引， $\ell = 0$ 为最粗层	<code>lev</code> 、 <code>level</code>
r	RZ/cylindrical 几何中的径向坐标	<code>m</code>
<code>rank</code> 或 r_{MPI}	MPI rank 或 rank 索引；不用裸 r 与径向坐标混写	<code>ParallelDescriptor::MyProc()</code> 等
V_i	网格单元体积	Cartesian 中常为 $\Delta x \Delta y \Delta z$
ρ_i^n	单元/节点 i 在时间层 n 的电荷密度	<code>rho</code> 、 <code>rho_fp</code> 、 <code>rho_buf</code>

符号	含义	在程序中的对应
$\mathbf{J}_i^{n+1/2}$	时间层 $n + 1/2$ 的电流密度	current_fp、current_buf
$\nabla_h \cdot \mathbf{J}$	离散散度	由 staggering 和差分 stencil 决定
$S_i(\mathbf{x}_p)$	粒子位置对网格量 i 的形函数数值	ShapeFactors 中的 shape 权重

在 AMR 语境中, _fp 与 _cp 通常分别标记 fine patch 和 coarse patch, _aux 则标记供粒子 gather 使用的辅助场。它们不是同一物理场的任意别名: 粗细网格覆盖、guard cell 和同步阶段决定某个数组何时有效。第 3、5 和 7 章中的 current_fp/current_buf、rho_fp/rho_buf 与 Efield_aux/Bfield_aux 都应按这个生命周期来读。

A.3 A.3 沉积、推进与场求解记号

符号或名称	含义	阅读提示
$\rho^n \rightarrow \mathbf{J}^{n+1/2} \rightarrow \rho^{n+1}$	电荷守恒 current-deposition 主线的时间层关系	不能把 charge deposition 和 current deposition 当成同一个 kernel
old / new	粒子在一个推进步或 segment 两端的形函数/位置状态	Esirkepov、Villasenor kernel 中常见
relative_time	在同一步内取样粒子位置的相对时间参数	影响 source 的时间层, 而不是额外推进粒子
icomp	charge component 或时间层分量选择	需结合调用入口解释, 不能只按名字猜测
depos_lev	charge 沉积目标 AMR level	coarse-buffer 粒子可能直接沉积到 lev-1
current_fp	fine-patch current buffer	主要对应 fine patch 粒子沉积
current_buf	coarse-buffer current buffer	后续还要经过同步/整理
rho_fp / rho_buf	fine-patch / coarse-buffer charge buffer	是 source route 的观测面, 不等于最终 plotfile 字段
Efield_fp / Bfield_fp	fine-patch 电场/磁场	不能直接假定就是粒子 gather 的数组
Efield_cp / Bfield_cp	coarse-patch 电场/磁场	在 mesh refinement 近边界的 gather/sync 语义要结合 level 与 patch type 判断
Efield_aux / Bfield_aux	供粒子 gather 的辅助场	由 UpdateAuxiliaryData 等路径从 patch 场构造; 只定义 E/B, 不把它误认为 current 或 charge 容器
Direct	直接速度加权电流沉积	简单但不自动满足离散连续性方程

符号或名称	含义	阅读提示
Esirkepov	基于轨迹与形函数差分的 charge-conserving current deposition	重点看 density decomposition 和 prefix accumulation
Villasenor	基于 cell crossing segment 的 charge-conserving current deposition	重点看 crossing、segment 和局部 face flux
Vay	Vay current deposition / spectral 相关路径	常与 PSATD、current correction 组合讨论
FDTD	有限差分场求解器	依赖 staggered grid 和 CFL 限制
PSATD	pseudo-spectral analytical time-domain 求解器	重点看谱空间系数、源项时间模型和周期边界假设
JRhom	PSATD 的多次 J/ρ 时间采样路径	不是多次粒子 push，而是同一轨迹的多时刻源项
PML	吸收边界层	通过 split-field 或等价谱推进吸收出射波

A.4 A.4 诊断、文件与验证术语

名称	含义
plotfile	分析用的网格/粒子输出；不可 restart，重启使用 Full checkpoint
openPMD	粒子、网格或 reduced diagnostics 的结构化输出接口
diagNNNNNNNN	diagnostics 输出的迭代目录，例如 diag1000000
reduced_diags	不保存完整场/粒子，而输出聚合标量或低维量的 diagnostics
FieldProbe	沿指定几何路径采样场并做积分或线探针输出的诊断
BoundaryScrapingDiagnostics producer	记录穿过粒子边界的粒子及其统计量 产生字段、粒子或 reduced output 的运行时路径
consumer	读取产物并执行物理、格式或性能断言的分析脚本
physics gate	对解析解、守恒量、谱或物理量的数值容差断言
writer gate	对文件、字段、维度、iteration、轴和有限非零数据的断言
checksum gate	对最终输出做确定性校验；不能自动等价于物理正确性
reader-side analysis	从已有 plotfile/openPMD 重新读取数据并验证合同的分析层

A.5 A.5 常用缩写

缩写	全称	本书中的语境
PIC	Particle-In-Cell	粒子-网格数值方法
AMR	Adaptive Mesh Refinement	多层网格和 coarse/fine 同步
CFL	Courant-Friedrichs-Lewy	显式场推进稳定性限制
NCI	Numerical Cherenkov Instability	boosted-frame / PSATD 中的数值不稳定性
RZ	cylindrical geometry with azimuthal modes	WarpX 的轴对称/模态几何
EB	Embedded Boundary	嵌入边界和 cut-cell 几何
MPI	Message Passing Interface	rank 间并行通信
GPU	Graphics Processing Unit	device kernel 和 GPU memory 路径
QED	Quantum Electrodynamics	高场量子辐射和 pair-production 模型
LWFA / PWFA	Laser / Plasma Wakefield Acceleration	激光/束流驱动尾场应用

A.6 A.6 使用规则

- 看到 `ux`, `uy`, `uz` 时, 先确认它来自输入、内部粒子容器还是 diagnostics; 分别按 $\gamma\beta$ 、 $\gamma\mathbf{v}$ 或该输出的 metadata 解释, 不能把数值直接混用。
- 看到 `rho`、`current_fp` 或 `current_buf` 时, 先判断它属于 local kernel、level buffer、同步后场, 还是最终 diagnostics 输出; 同名物理量可能处于不同生命周期阶段。
- 看到上标 n 、 $n + 1/2$ 或 `relative_time` 时, 先确认粒子位置、场、current 和 charge 是否处在同一个时间层; 不能只根据变量名推断守恒性。